# Enabling Volumetric Video Conferencing and Live Streaming

by

Rajrup Ghosh

A Dissertation Presented to the
FACULTY OF THE USC GRADUATE SCHOOL
UNIVERSITY OF SOUTHERN CALIFORNIA
In Partial Fulfillment of the
Requirements for the Degree
DOCTOR OF PHILOSOPHY
(COMPUTER SCIENCE)

August 2026

## Acknowledgements

Hello, World!

The journey toward completing this dissertation has been a challenging yet fulfilling experience, filled with both highs and lows. Throughout this journey, I have learned not only how to conduct rigorous research, but also how to navigate uncertainty, reflect on my own strengths and limitations, and grow both professionally and personally. While there were moments of difficulty and self-doubt, this experience has been deeply rewarding, and I am grateful to those who supported and guided me along this process.

First and foremost, I would like to thank my advisor, Prof. Ramesh Govindan, for his support and guidance throughout my PhD. Over the years, he has influenced not only the way I conduct research, but also the way I think, make decisions, and approach challenges. In many situations, I often found myself asking what Ramesh would do, a reflection of how deeply his perspective has shaped my own. He trusted me when I needed confidence, challenged me when I needed clarity, and gave me the freedom to grow in my own direction. His steady support and patience during both productive and difficult periods of my PhD made this journey not only possible, but also genuinely enjoyable. I feel incredibly fortunate to have had him as my advisor and a role model, and the lessons I learned from working with Ramesh will continue to guide me well beyond this dissertation.

I am also grateful to my proposal and dissertation committee members, Prof. Harsha V. Madhyastha, Prof. Antonio Ortega, Prof. Barath Raghavan, and Prof. Laurent Itti for their time, thoughtful feedback and valuable suggestions throughout my PhD.

Throughout the journey, I am grateful to have my collaborators and colleagues who made this work possible. In particular, I am grateful to Fan Bai and Chuan Li for their mentorship and support during my collaboration with General Motors. I also thank all of my labmates in Networked Systems Lab (NSL) for the stimulating discussions and collaborative environment. I am especially thankful to Fawad Ahmad, Weiwu Pang, Rajrup Ghosh, and Chunyu Xia, whom I worked closely with. I deeply appreciate many memories we shared while working together on real-world experiments, from installing LiDARs and deploying systems to running evaluations and driving cars around campus. Although the work was often challenging at the time, it has become a collection of fun and cherished memories that I look back on fondly.

Finally, I owe my deepest thanks to my parents (Dongha Shin and Kyungsook Lee) for their unconditional love and support. I am deeply grateful to my parents, who have supported me in every possible way throughout my life. I was born during my father's PhD journey, and growing up alongside his academic journey naturally sparked my curiosity and interest in pursuing a PhD in computer science. My parents have believed in me from the very beginning, and I carry their love and encouragement with me every day. They deserve far more appreciation than I can express here.

I am also profoundly thankful to my husband (Hang Qiu), whom I met during my PhD journey. He has been my constant source of strength and comfort, supporting me through both research and life. With him, I can find rest and reassurance, and regain perspective when things feel overwhelming. His presence has made this journey full of laughter, and I am deeply grateful to share this journey and my life with him.

# Table of Contents

# List of Tables

# List of Figures

## Abstract

Three dimensional (3D) data have become a key modality for representing and interacting with the physical world. Modern vehicles and roadside infrastructure are equipped with 3D sensors that capture the geometry of traffic participants and surroundings. Meanwhile, the growing deployment of vehicular communication technologies enables these vehicles and infrastructure elements to exchange 3D data with one another and with cloud servers in real time. Together, advances in 3D sensing and vehicular connectivity are transforming how spatial information is captured, shared, and utilized in vehicular systems.

This dissertation explores how 3D data can be elevated to a shared and networked resource that is collaboratively sensed, processed, and delivered across vehicles, infrastructure, and cloud systems. It presents three systems spanning the full networked 3D data pipeline. RECAP addresses the *reconstruction* phase by enabling vehicles to offload 3D sensor data to the cloud for high-fidelity multi-view fusion under dynamic and sparsely overlapping observations. CIP targets the *interpretation* phase, demonstrating real-time cooperative infrastructure perception by fusing data from multiple roadside LiDARs to track and estimate the motion of traffic participants. MARS extends the pipeline to the *delivery* phase by streaming immersive 3D video content from the cloud to in-vehicle augmented reality (AR) displays, adapting to vehicle motion and dynamic network conditions to provide spatially coherent AR experiences.

Together, these contributions demonstrate that networked 3D data can be effectively sensed, reconstructed, interpreted, and delivered in connected vehicular environments, enhancing perception, improving traffic safety and efficiency, and enabling next-generation in-vehicle experiences.

# Chapter 1

# Introduction

The past couple of decades have seen an explosion in Internet-scale video streaming, conferencing, and analytics. Over the next couple of decades, these applications will shift towards *volumetric* videos. In contrast to a regular video, each frame in a volumetric video represents visual information in 3D. Using an augmented/mixed reality headset, a viewer can view each frame of a volumetric video in any direction from any point within the recorded environment, thus offering the viewer 6 degrees of freedom (6DoF). Volumetric videos are typically captured using an array of cameras capable of recording both the color and depth of every point in the scene [87].

This new capability opens up a whole new class of previously infeasible applications. Live volumetric streaming can offer a much more immersive experience of lectures [43], theater performances [7], telesurgery [120], museological storytelling [85], and sports broadcasting with analytics [**volumetric-golf**, **volumetric-sports**]. A major application in this domain is volumetric telepresence, where multiple participants can interact with each other in a shared virtual space or real captured space in 6 degrees of freedom (6-DoF). Participants can have the illusion that they are all in the same room instead of simply seeing 2D renderings of each other (Fig. 1.1). A significant extension of today's video conferencing platforms like Zoom, Microsoft Teams, and Google Meet is to extend them to provide fully immersive experiences to the participants.

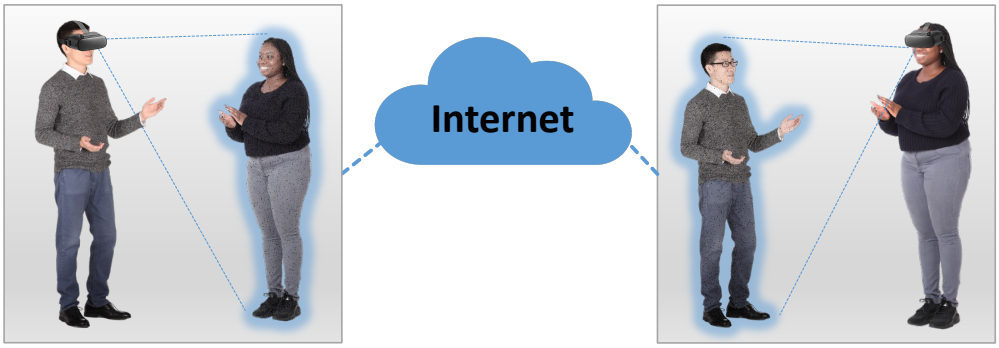



**Figure 1.1:** A telepresence session between 2 participants using volumetric video.

### 1.0.1 Challenges

Despite its promising applications, volumetric video streaming faces significant challenges:

**Bandwidth Constraints.** Transmitting high-quality 3D data in the form of point cloud, mesh, or other representations requires substantial bandwidth. Volumetric video frames inherently contain detailed geometric and color information, resulting in significantly larger data sizes compared to traditional video formats. Effective compression and bandwidth management techniques are essential to handle these large data volumes within existing network constraints. The problem turns worse when streaming full-scene volumetric video compared to a portrait or single person. Moreover, current volumetric video streaming systems [33, 61, 87, 55] are not bandwidth-adaptive, *i.e.,* they do not adapt to the available bandwidth of the network at a given time.

**Latency Constraints.** Achieving low latency of 200-300 ms [**conf-abr1**, **conf-abr2**, 58] (as achieved by today's 2D video conferencing) is particularly challenging in 3D video conferencing where delays can severely impact user's immersive experience. Maintaining real-time interactions demands efficient optimization of encoding, transmission, and decoding processes. 3D compression methods such as G-PCC [29], V-PCC [95], and Draco [22] are still in their infancy and are yet to be optimized for low-latency encoding and decoding of 3D data. Additionally, larger data sizes for 3D due to the low compression ratio achieved by 3D codecs increase transmission latency.

**Visual Quality and Realism.** Ensuring high visual quality and realism is difficult due to potential visual artifacts introduced by current volumetric streaming approaches that primarily use point clouds as their 3D representation. Point clouds have holes and floating artifacts that can reduce realism and overall user immersion, impacting the satisfaction and effectiveness of immersive experiences. Few solutions have been proposed to use meshes instead of point clouds, but they are not yet optimized for live streaming or conferencing. Moreover, the current systems do not use recently introduced advanced rendering techniques such as Neural Radiance Fields (NeRF) [74] or Gaussian Splatting [50] that can achieve photorealism. NeRFs and Gaussian Splatting produce realistic novel view generation at the expense of high computational and memory requirements, which make them infeasible to use out of the box.

### 1.0.2 Contributions

This thesis contributes to addressing these challenges through three distinct yet complementary chapters:

**Chapter 2: LiVo- Toward Bandwidth-adaptive Fully-Immersive Volumetric Video Conferencing.** Chapter 2 introduces LiVo, a novel conferencing system designed to efficiently stream high-quality, full-scene volumetric videos between 2 parties over the Internet. LiVo addresses bandwidth constraints by leveraging advanced 2D video compression techniques, rate-adaptive video codecs, and dynamic bandwidth splitting strategies. It uniquely integrates view prediction and view-culling techniques to minimize transmitted data by adapting to users' real-time viewing frustum, significantly reducing bandwidth requirements without sacrificing quality. LiVo carefully balances the latency for each component to match the line rate of 30 frames per second (fps) and achieves an end-to-end latency of around 250 ms.

# Chapter 2

# LiVo: Toward Bandwidth-adaptive Fully-Immersive Volumetric Video Conferencing

## 2.1 Introduction

**Volumetric Video.** Exploiting visual sensors that capture depth information, research has recently started exploring the capture, transmission, and display of *volumetric* videos. This technology is starting to be used in live streaming sports events and in advertising [39, 112, 13]. Each frame of a volumetric video captures a scene or an object in three dimensions, and is often represented as a point cloud – a collection of points on surfaces of a person or an object – together with each point's position (also known as *geometry*) and color. Volumetric video differs from 2D video and spatial video [39] in two ways. It is significantly more bandwidth-intensive (§2.2). In addition, it permits users to *interact* with the video along 6 degrees of freedom (6-DoF), by viewing a frame from any angle, at any location in the frame. Users can shift perspective by moving in a designated space while wearing a headset that tracks their movement and renders each frame using the current perspective. As such, volumetric video promises a degree of immersion comparable to physical presence. Recent work has explored on-demand and live streaming of volumetric videos (§2.5, Table 2.1).

**Volumetric Video Conferencing.** Relatively less attention has been paid to *conferencing* using volumetric videos. Imagine a two-way conference that streams entire physical spaces (*e.g.*, areas with multiple

participants and objects in the environment) between two locations. This is a capability that can enable immersive collaboration, for example for musical or theatrical productions, remote assistance, and distance education. This *full-scene* volumetric video conferencing requires significant bandwidth: captures of complete scenes are large (upwards of 10 MB per video frame; §2.2, Table 2.3). Fortunately, average broadband bandwidths worldwide are now about 100 Mbps [14], so full-scene volumetric video transmission is on the horizon. But, to enable volumetric video conferencing, it helps to understand why 2D conferencing has been so successful.

**Lessons from 2D Video Conferencing.** 2D video conferencing is also bandwidth-intensive, but owes its success to three factors. First, 2D video codecs are extremely *bandwidth-efficient*, leveraging inter-frame compression and other coding tools to reduce the amount of data sent (this also contributes to the success of on-demand or live streaming). Second, these codec implementations are *rate-adaptive*: they can *directly* compress a video frame to a given available rate [25, 58]. This enables conferencing applications to be *bandwidth-adaptive*. In contrast, on-demand and live streaming systems adapt to bandwidth changes *indirectly*: they pre-encode video chunks (a set of consecutive groups of pictures) into different *qualities*, then send the encoded chunks that best fit available bandwidth [37, 101, 44, 1]. Streaming systems can tolerate more end-to-end delay, so they can afford to pre-encode videos for bandwidth adaptation. Conferencing systems must, however, process and transmit frames at full frame rate and incur no more than 200-300 millisecond latency end-to-end [58, 25, 20, 57]. Third, mature *compute-efficient*, sometimes hardware-accelerated, implementations of codecs have enabled these high levels of performance.

**Shortcomings in Prior Work.** Recent research on volumetric video conferencing either does not support full-scene conferencing or lacks one of these above three properties (Table 2.1, §2.5). Metastream [30], Starline [56], and others [111, 32, 60, 87] transmit constrained views (of a single participant, or of their face or torso). They are, moreover, not bandwidth-adaptive. MeshReduce [45] permits full-scene conferencing, but suffers from two shortcomings. It uses a different volumetric representation (a *mesh*), for which compression

methods cannot yet exploit inter-frame redundancies, so it sacrifices bandwidth-efficiency. Moreover, for meshes, no direct rate-adaptive codecs exist, so MeshReduce *indirectly* adapts to bandwidth. It profiles videos to obtain an empirical mapping between quality and bitrate. Then, given a bandwidth estimate, a sender can compress at a quality obtained from the mapping to (approximately) achieve a target bandwidth. These shortcomings lead to poor perceptual quality (§2.4).

**Goals.** In this paper, we describe the design and implementation of LiVo, a full-scene volumetric video conferencing system. LiVo uses an array of RGB-D cameras, as does prior work in conferencing discussed above, but unlike these, it aims to: (a) support frame rate processing of full-scenes with end-to-end latencies of 200-300 ms and (b) be simultaneously bandwidth-adaptive, bandwidth-efficient, and compute-efficient (in the sense described above).

**Approach.** LiVo re-uses infrastructure developed for 2D video conferencing: efficient compression algorithms, rate-adaptive codecs that can encode and decode at full frame rate, and protocols like WebRTC for transmission of video (§2.3). In this, it is inspired by Starline [56], which also encodes RGB and depth images from each RGB-D camera into separate video streams. As described above, however, Starline does not transmit full-scenes, and is not bandwidth-adaptive.

**Contributions.** In achieving its goals, LiVo makes three distinct contributions (§2.3).

First, it exploits technology trends to multiplex several RGB-D camera outputs into just two video streams: a color stream and a depth stream. This requires less receiver-side synchronization when reconstructing point clouds. Unlike systems that transmit constrained views (*e.g.*, only the face or the torso) [56, 30], full-scene conferencing requires encoding larger physical depth ranges. LiVo develops a novel depth encoding and a video transmission mode that reduces distortion due to depth.

Even though bandwidth-adaptivity comes for free when using 2D video, LiVo needs to carefully split available bandwidth between color and depth streams. Humans are sensitive to distortion in depth, so LiVo must allocate more of the available bandwidth to depth than to color. A static split is sub-optimal

because bandwidth needed to ensure high quality delivery can depend on scene complexity (*e.g.*, number of participants or objects in the scene). LiVo's second contribution is a novel *bandwidth-splitting* strategy that continuously *adapts* to both bandwidth availability and scene complexity changes.

Finally, LiVo employs *view-culling* to increase bandwidth-efficiency. Sender only sends 3D information within the receiver's current field of view. Prior work in on-demand streaming has explored view-culling, but LiVo exploits the tighter end-to-end latency of conferencing to enable cheap, accurate prediction of receiver views and also develops a fast culling technique.

Using a complete implementation of LiVo, which we plan to release publicly, we have conducted a user study and measured objective 3D quality metrics. Both in our user study and by objective quality metrics, LiVo consistently outperforms three baselines: a bandwidth-adaptive *oracle* of Draco [22], a popular point cloud compression technique; an approach that mimics a bandwidth-adaptive version of Project Starline [56]; and MeshReduce [45]. Experiments also demonstrate that LiVo can achieve around 250 ms end-to-end latency at 30 frames per second, comparable to the performance of existing 2D conferencing platforms [58, 20].

**Statement of Ethics.** We obtained Institutional Review Board (IRB) approval for collecting user interactivity traces with 3D videos, and for our user study. As required by the IRB, to protect user privacy, we removed all identifiable information and present analyses using only aggregated statistics.

## 2.2 Background and Motivation

**Volumetric Video.** Volumetric video captures a three-dimensional representation of a continuously evolving scene (*e.g.*, a game, a live performance, a conference call). Each viewer of a volumetric video has 6 degrees of freedom (6DoF) — he/she can view the video from any point in the captured space, in any direction. For example, a viewer can watch a live performance of a string quartet as though sitting next to a violinist, while at the same time, another viewer can take the cellist's perspective. Or, a technician can remotely assist debugging and repairing an aircraft part, moving in space to inspect the part from different

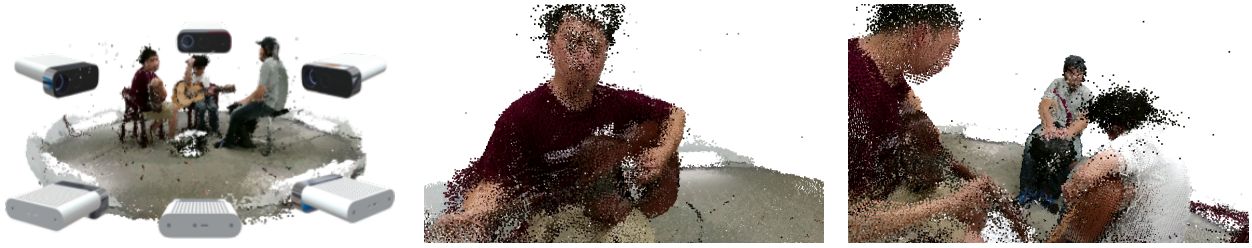


**Figure 2.1: Left:** Capture, **Right:** Two user viewpoints from an actual user trace.

perspectives. Typically, viewers watch volumetric videos on a mixed-reality headset. They *interact* with the video by moving around in a physical space; the headset tracks the viewer's movement and, at any given moment, renders the video from a perspective consistent with the viewer's current position and orientation. Thus, in our example of the musical performance, viewers can move to change perspective during the performance (Fig. 2.1).

A volumetric video consists of a sequence of 3D frames, each representing a three-dimensional capture of a physical space. A *point cloud* [33, 61] is one representation of a frame; we discuss another, the *mesh*, later. Each point in a point cloud corresponds to a 3D location (usually on the surface of an object) that has location coordinates (also called *geometry*) and color of the surface at that location.

In this paper, we consider volumetric videos obtained from an array of off-the-shelf RGB-D cameras encircling a scene. Commodity RGB-D cameras, such as Azure Kinect DK [3] or Intel RealSense [41], can capture both the color (RGB) and depth (D) of points in a scene. By aligning (§2.3.2) RGB-D frames from multiple cameras, one can estimate the position and color of an arbitrary point in the scene, thereby generating a point cloud.

**Truly Immersive Two-Way Conferencing.** We consider the problem of streaming volumetric video between two locations (**A** and **B**) while permitting 6-DoF viewing at both ends. In contrast to prior work [56, 60, 111] on volumetric video conferencing that transmits 3D representations of a single human being (their face or torso), we seek to stream a *full-scene* from **A** that may consist of multiple participants and objects

| Streaming Systems | Type | Compression | Content | BW-adaptive? | FPS | Cull? |
|---|---|---|---|---|---|---|
| ViVo [33] | On-demand | 3D | Multiple Person | Indirect | 30 | ✓ |
| Groot [61] | On-demand | 3D | Full-scene | No | 30 | ✓ |
| Fumos [66] | On-demand | 3D | Single Person | No | 30 | ✓ |
| Vues [69] | On-demand | 2D | Multiple Person | Indirect | 30 | ✓ |
| Holoportation [87] | Conferencing | 3D | Single Person | No | 30 | ✗ |
| LiveScan3D [55] | Live | 3D | Full-scene | No | 15 | ✗ |
| FarFetchFusion [60] | Live | 2D | Portrait | No | 30 | ✗ |
| Starline [56] | Conferencing | 2D | Upper Torso | No | 30 | ✗ |
| Tele-Aloha [111] | Conferencing | 2D | Upper Torso | No | 30 | ✗ |
| VRComm [32] | Conferencing | 2D | One Person | No | 30 | ✗ |
| MeshReduce [45] | Live | 3D | Full-scene | Indirect | 15 | ✗ |
| MetaStream [30] | Live | 3D | One Person | No | 30 | ✗ |
| LiVo | Conferencing | 2D | Full-scene | Direct | 30 | ✓ |

**Table 2.1:** Comparison to related work, more details in §2.5.

surrounding them (*e.g.,* furniture, the floor, walls, etc.). Full-scene conferencing would enable an application in which, for example, groups of actors at **A** and **B** could collaborate jointly to rehearse an upcoming theater performance. As users at **B** receive and view the video, they can move around the scene to change their perspective, permitting true immersion. These movements are captured in video transmitted to **A**, bringing a level of interactivity for human interaction comparable to physical presence.

To realize this interactivity, a conferencing system must (a) transmit full-scene point clouds at *full frame rate* (30 frames per second) (b) with an end-to-end latency of 200-300 ms [58, 25, 20, 57]. Today's 2D conferencing systems satisfy these requirements. Full-scene point clouds can be much larger than those representing more constrained views (single human beings with only their face or torso). For example, in one 3D dataset [46] we use in §2.4, the average point cloud frame size for a participant (single human in the scene) is about 1 MB uncompressed, but for the full-scene (with furnitures, floor, objects, *etc.*) is 10 MB (Table 2.3). As such, transmitting full-scenes will require significant network bandwidth. Today, average global broadband speeds are nearly 100 Mbps and are expected to increase by 20% annually [14]; thus, we expect immersive two-way conferencing to be feasible in the near future, if it is not already.

**Challenge: Compression.** Even with increased broadband speeds, transmitting full-scenes will require compression. In our example above, each full frame needs to be compressed to over a twentieth of its original size to fit the 100 Mbps capacity. The need for compression is unlikely to go away: even if broadband bandwidth increases, point cloud sizes might increase with better cameras.

Point cloud compression techniques such as Draco [22], V-PCC [95], and G-PCC [29] can potentially address this. Indeed, prior work has used one or more of these for on-demand [61, 33] and live [30] streaming of non-full-scene volumetric video.

However, these techniques can be compute intensive and their computational complexity grows linearly with point cloud size. For example, on a machine we use in §2.4, compressing a 1 MB point cloud using Draco takes 25 ms while compressing a 10 MB frame takes over 300 ms. Other compression techniques are slower: 8 minutes using V-PCC for 11 MB point clouds, and 10 seconds for G-PCC. These latencies can make it difficult, if not impossible, to achieve the 200-300 ms end-to-end latency target discussed above. Parallelizing compression [33, 30, 45] can help, but the increase in compute cost can deter or delay adoption.

Point cloud compression techniques also do not achieve as high compression efficiencies as 2D video compression. This is because their *inter-frame* compression algorithms are the subject of ongoing research [122, 36, 35]. For instance, Draco's default setting can compress the 10 MB per full-frame video to about 1.78 MB per frame. In contrast, the approach we describe in this paper does not use point cloud compression and can compress to about 0.66 MB per frame on average.

**Challenge: Bandwidth-adaptivity.** Even with increasing average bandwidths, it will be important for two-way conferencing to be *bandwidth-adaptive.* Bandwidth availability can vary spatially (*i.e.,* across different regions) and temporally (*i.e.,* during a single session). Two-way conferencing can exploit high bandwidth availability to deliver high quality. However, when bandwidth drops, it must deliver volumetric video without stalling.

| Trace Names | Mean Capacity (Mbps) | MeshReduce | | Livo | |
|---|---|---|---|---|---|
| | | Mean TPS (Mbps) | Util. (%) | Mean TPS (Mbps) | Util. (%) |
| *trace-1* | 216.90 | 40.19 | 18.53 | 158.75 | **73.19** |
| *trace-2* | 89.20 | 27.75 | 31.11 | 82.21 | **92.16** |

**Table 2.2:** Throughput (TPS) of LiVo vs. MeshReduce.

2D video conferencing systems (e.g., those that use WebRTC [8]) address this by (a) continuously estimating available bandwidth and (b) using a *rate-adaptive* codec implementation. Such a codec takes a desired bandwidth as input, and attempts to encode the frame at that target bandwidth, thus *directly* adapting to bandwidth changes. Today, compression algorithm implementations that directly compress point clouds or meshes (*e.g.,* Draco [22]) are **not** rate-adaptive: applications cannot specify a target output bitrate when compressing 3D frames. Instead, they contain parameters that control the *quality* of the encoder output. For example, Draco's *quality parameter* (QP) governs the degree of allowable distortion in the output; lower quality output requires lower rate for transmission. However, given a target available bandwidth, an application cannot know *a priori* what QP value to choose to achieve that bandwidth. Recent work, MeshReduce [45], profiles 3D videos offline to develop a mapping from bitrate to one or more parameters (like QP). During a session, given the current available bandwidth, it instructs the compression algorithm to use parameters from the mapping. This approach can only *indirectly* adapt to bandwidth (as opposed to using a codec implementation that can directly encode at a given target rate). It can also be conservative: as Table 2.4 shows, unlike LiVo which uses direct bandwidth adaptation, MeshReduce produces encodings that are significantly lower than the target bandwidth. As a result, it has lower perceptual and objective quality (§2.4).

## 2.3 LiVo Design

LiVo addresses the two challenges discussed above — it efficiently encodes 3D content *and* employs direct bandwidth-adaptation — while ensuring efficient, high quality volumetric video transmission.

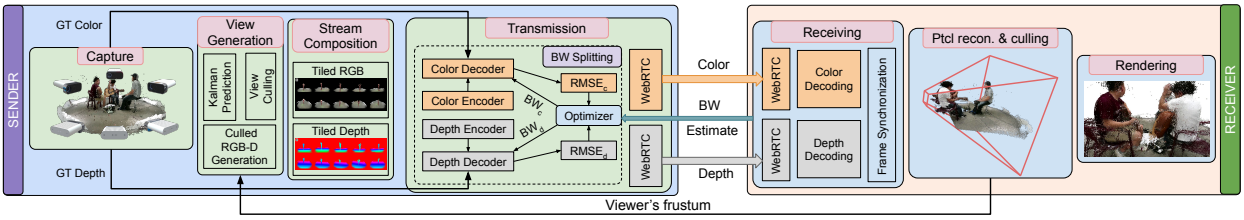



**Figure 2.2:** LiVo Architecture. The green blocks on the left run on the sender and blue blocks on the right run on the receiver.

### 2.3.1 Overview

LiVo enables volumetric video conferencing between two sites. Each site has an *array* of off-the-shelf RGB-D cameras and a desktop-class computing device for processing, sending, or receiving volumetric video from the other site, as well as a viewing device (*e.g.,* a mixed reality headset). Multi-way conferencing can be built using LiVo, but presents opportunities for optimizations (*e.g.,* across receivers from a single sender) that we leave to future work.

Fig. 3.1 pictorially depicts the design of LiVo's sender-to-receiver pipeline; in the deployment model above, each site runs one instance of the pipeline in each direction. LiVo captures a sequence of RGB-D frames from each camera in an array encircling a scene (*e.g.,* a conference table, a small stage or performance area, or a small room). At a given instant, a frame from each camera can be used to generate a point cloud (§2.3.2), and a sequence of successive point clouds constitutes the volumetric video.

LiVo transmits the volumetric video as streams of 2D video. This choice permits LiVo to re-use mature widely-deployed technology: (a) off-the-shelf highly efficient video compression standards such as H.264 and H.265; (b) mature rate-adaptive open-source codec implementations (*e.g.,* GStreamer [109] and FFmpeg [24]); and (c) real-time transport for video conferencing such as WebRTC [8] with its built-in Google congestion control [10].

The second is to *cull* each point cloud to only include points that fall into the receiver's *frustum*. The frustum represents the receiver's 3D field of view (Fig. 3.1). When the receiver is viewing the entire scene,

her frustum includes the entire point cloud. But, for example, if she moves closer to objects or participants in the scene, her frustum will likely include a much smaller part of the point cloud (Fig. 3.1).

Together, these address the two challenges described in §2.2. 2D codecs employ inter-frame compression, resulting in higher bandwidth-efficiency relative to point-cloud compression. Culling further reduces the bandwidth requirements. Moreover, 2D codecs can encode to a target bandwidth, so LiVo can directly adapt to bandwidth changes. Finally, there exist mature, compute-efficient implementations of 2D codecs that can, with low latency, process video streams at full frame rate.

Even though it relies maximally on 2D video technology, LiVo's design poses two significant challenges: (a) how to encode RGB-D frames in 2D videos and how to adapt these to available bandwidth; (b) how to determine receiver frustum and cull views. We describe how LiVo addresses these challenges in the rest of this section.

### 2.3.2 2D Video Encoding

**Background: Point Clouds from RGB-D Camera Array.** Each camera produces RGB color and depth frames at 30 frames per second. Consider a static RGB-D camera array (Fig. 3.1) with $N$ frame-synchronized cameras.* Then, for every inter-frame interval (30th of a second), we can generate a point cloud from $N$ frames, one from each camera. To do this, we need to determine the position and orientation of each RGB-D camera in a common frame of reference. There exist standard one-shot calibration techniques for this [127] that produce a transformation matrix to convert each camera's local coordinate system to a global one. Generating a point cloud is then simple: for each pixel of each RGB-D frame, first determine the pixel's position in the camera's local coordinate frame (using camera parameters such as its center and focal length [30]), and then convert it to global coordinates (using the transformation matrix) to obtain one point in a point cloud.

*There exist techniques to synchronize Kinect cameras [108].

**The Challenge.** Prior work in volumetric video generates a point cloud at the sender then applies point cloud compression (*e.g.*, Draco [22]) [55, 30]. LiVo is qualitatively different: it encodes 2D RGB-D video frames *directly* at the sender, and the receiver decodes and constructs a point cloud for viewing. An array of $N$ cameras produces $N$ color frames and $N$ depth frames at 30 fps.

For these frames, LiVo must solve three distinct problems: *stream composition*, *depth encoding*, and *bandwidth splitting*. Stream composition refers to the problem of multiplexing the $2N$ images into one or more video streams. Multiplexing all images, depth and color, onto a single stream defeats inter-frame prediction because successive frames might be from different cameras or might interleave color and depth frames. At the other extreme, independently encoding each camera's output into a color stream and a depth stream increases complexity in two ways. First, the receiver must reassemble related color and depth frames from different streams to reconstruct the point cloud. We have found this reassembly tricky, especially when different streams incur different delays. Second, if LiVo composes frames into more than one stream, it must determine how to split available bandwidth to each stream (§2.3.3). The more streams there are, the more complex the splitting algorithm.

Depth encoding converts pixel depth values into a format that video codecs recognize. For example, one can encode depth in one of the three color channels (R/G/B) in a color image. In general, this approach can introduce significant distortions, since video compression algorithms exploit smoothness in natural images to achieve compression, but depth information can exhibit discontinuities [91, 53, 118]. LiVo must attempt to minimize depth distortion, especially since humans are very sensitive to point cloud depth errors [123].

In this subsection, we describe LiVo's stream composition and depth encoding, then discuss bandwidth splitting in §2.3.3.

**Stream Composition.** LiVo leverages technology trends to compose streams in a novel way. Off-the-shelf RGB-D cameras have lower depth resolution than color resolution. For example, the Azure Kinect DK offers $640 \times 576$ *depth* image resolution, and the Intel RealSense D457 supports up to $1280 \times 720$, but their color

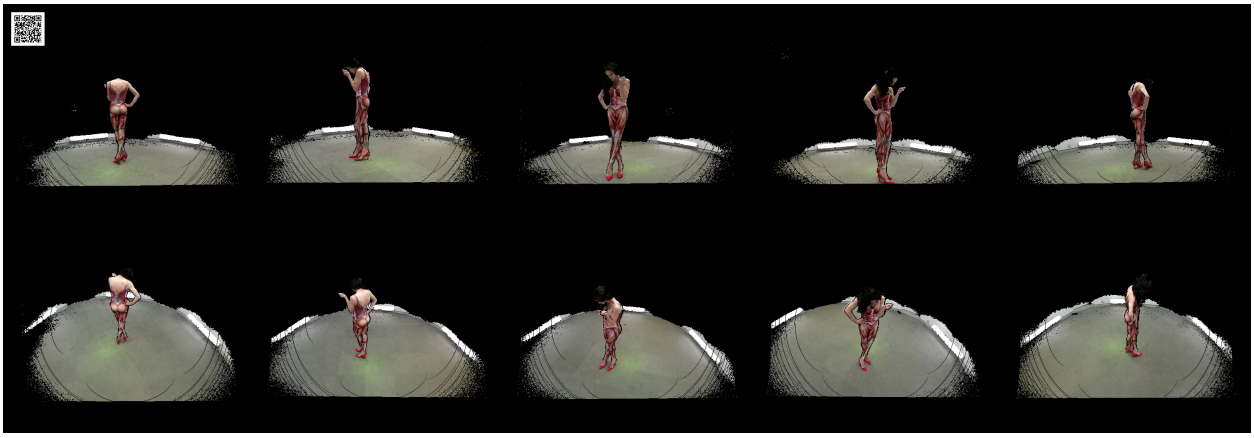


**Figure 2.3:** Tiled color view for 10 kinect cameras. A Similar view is generated for depth.

images can have up to 4K resolution. These cameras usually include a time of flight sensor that measures, for each pixel, the depth of that pixel. The cost of these sensors have resulted in the slower growth of RGB-D camera depth resolution.

LiVo exploits this observation to multiplex the $2N$ images into *two* videos: a color video and a depth video.

Specifically, it tiles the $N$ depth images into a 4K frame (Fig. 2.3). RGB-D cameras generate color images at a much higher resolution (up to 4K), but it would be wasteful to transmit this, since the receiver would first downsample the color image to match the depth image resolution to construct the point cloud. Upsampling the depth to match color resolution is not followed in practice to avoid introducing distortion in geometry [5, 88, 55]. Accordingly, LiVo downsamples color images to the depth resolution, then tiles these downsampled color images into a 4K color frame. Using this approach, LiVo can fit 9 Intel RealSense cameras or 16 Azure Kinect DK cameras into two 4K frames (for color and depth). Given that these cameras have a modest range (5-6 m) and can only cover small spaces, we do not foresee deployments with significantly more RGB-D cameras (other work also makes similar assumptions about the number of cameras [19, 56, 60, 87]). With more cameras, we may need to multiplex images onto more streams and add more complex receiver synchronization, which we have left to future work.

Tiling camera images consistently on a 4K frame does not impact video compression efficiency. 2D video codecs predict macroblocks (8x8 or 16x16 pixel blocks) within and between frames. In successive frames, the location of the macroblocks is fixed (images from the same camera are located at the same spot in the tiled image). Inter-frame prediction is thus relatively unaffected by tiling since the macroblocks preserve locality, improving compressibility.

After tiling, LiVo passes each color 4K image to an H.265 encoder (see below for depth). These can encode 4K frames at full frame rate.

**Depth Encoding.** Prior work has explored two different techniques to encode depth into images. One approach encodes a 16-bit depth value into a 3-channel RGB pixel [100, 91, 32]. This attempts to ensure that the depth encoding respects color *smoothness* so that video codecs can compress the resulting image without significant distortions. This works well for shorter depth ranges (of about 1.5 m [32]) but not for larger ranges ([91], §2.4).

Other work [56, 60] encodes 10-bit depth into the Y-channel of a YUV [124] (a different way to represent color) encoding. Codecs compress the Y-channel (representing luminance) at higher bit rates to avoid loss since humans are sensitive to luminance distortions. For this line of work, this depth encoding works well because they target portrait 3D videos (of faces or upper torsos), which require a lower depth range (1-2 meters). Our RGB-D cameras output 16-bit depth values at millimeter resolution. Quantizing these to 10 bits can sacrifice either depth or resolution, impacting perceived quality and impairing full 6-DoF capabilities.

LiVo leverages a rarely-used H.265 mode, which supports a 3-channel 16-bit YUV encoding [81]. LiVo stores the 16-bit depth pixel value in the Y channel and sets the U and V channels to the same fixed value. This reduces depth distortion since codecs distort the Y-channel less.

However, simply storing the depth value in 16 bits on the Y channel introduces significant artifacts (Fig. 2.16). Current depth cameras have a maximum depth range of 5–6 meters [4, 42], so the depth values

can range from $0 - 6000$ at millimeter resolution. This uses only part of the full 16-bit range. Instead, we scale the depth value to occupy the entire 16-bit range, *i.e.,* scaled depth value for 0 mm remains at 0 while it is $2^{16} - 1$ for 6000 mm. This approach incurs lower depth distortion: codecs quantize depth values, and, for a given quantization step size, more unscaled depth values fall into one quantization bin than scaled depth values.

### 2.3.3 Bandwidth Splitting

**Background: WebRTC and Rate-adaptive Codecs.** Unlike on-demand HTTP-based video streaming, 2D video conferencing systems use a real-time transport protocol (*e.g.,* WebRTC) with rate-based congestion control (*e.g.,* GCC [10]). The sender feeds the available bandwidth from congestion control to a rate-adaptive video encoder, which compresses the frame to fit within the target bandwidth.

**The Challenge.** LiVo transmits depth and color videos as two separate WebRTC streams. Suppose, at a given instant, the sender's rate control algorithm estimates an available bandwidth of $B$. LiVo cannot assign $B/2$ to each stream. It encodes depth (§2.3.2) to reduce depth distortion, but in order to get higher quality, it must *also* assign more bandwidth to the depth stream than the color stream. Figure 2.4 shows the variation in quality for depth and color at different splits of a target bandwidth of 80 Mbps for one of the videos we use in §2.4. When the depth stream gets 90% of the available bandwidth, the error in depth and color is lowest. This is because humans are much more sensitive to depth than to color errors [123]; allocating more bitrate to depth reduces depth distortion [130]. Moreover, distortion in depth can also distort objective measures of color quality [130].

To ensure high quality, LiVo must determine the *bandwidth split* $s$: the fraction of available bandwidth allocated to the depth stream. A strawman approach might profile off-line, a collection of volumetric videos to determine an optimal *static* split at a given bandwidth. Intuitively, this split jointly minimizes distortion in depth and color. For example, Fig. 2.4 suggests a split $s$ of 90%. It is an open question whether such an approach can generalize easily; the optimal split can vary from frame to frame, as *scene complexity* — the

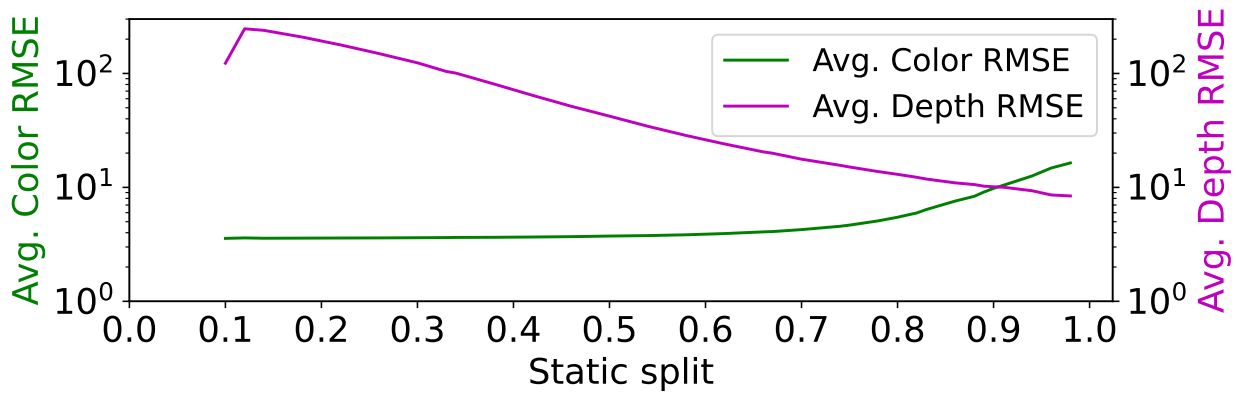



**Figure 2.4:** Color and depth RMSE for different splits at target bandwidth of 80 mbps. Log-scale y-axis. Video sequence: *band2*

number of participants, or the degree of motion — changes. It might be possible to design learned split predictors, but these might need a lot of data with varying degrees of scene complexity to generalize well, and we have left this to future work.

**Adaptively Splitting Bandwidth.** Instead of determining the split statically, LiVo *adaptively* and continuously determines splits to capture bandwidth and scene complexity changes. It repeatedly encodes a frame with a given split $s$, determines the resulting quality of the encoded frame, and then either increases or decreases $s$ to improve depth and color quality. To do this, it must solve two problems: (a) how to estimate encoding quality and (b) how to adapt.

***Estimating encoding quality.*** LiVo's approach encodes a frame at the sender, immediately decodes it, and then computes distortion with respect to the ground truth frame (available at the sender). Today, GPU-accelerated video encoders and decoders, like NVIDIA's `nvenc` [84], can decode multiple frames while also encoding other frames. So, while the GPU is encoding a frame, it can decode a previously encoded frame to determine quality. Parallel decoding minimally impacts GPU utilization.

To estimate the quality of the decoded frames, LiVo could use a point cloud quality metric such as PointSSIM [2], but this requires the sender to reconstruct the point clouds before and after encoding. LiVo instead uses the root-mean-square error (RMSE) in pixel values between the original (depth or color)

frame and the decoded frame. This choice is far more compute-efficient. LiVo further reduces the compute overhead by computing RMSE every $k$ frames ($k = 3$, chosen empirically, in our evaluations, §2.4), instead of every frame.

***Adapting the split.*** LiVo continuously adapts the split using the following steps (Fig. 3.1):

- It uses two video decoders (running parallely) which decode the compressed tiled frames to generate the distorted tiled color and depth frames.
- For each decoded color (resp. depth) frame, it calculates the color RMSE $RMSE_c$ (resp. depth RMSE $RMSE_d$) by comparing the distorted tiled frame ($F'$) to the ground truth tiled frame ($F$).
- LiVo does not modify the split if $|RMSE_d - RMSE_c| \leq \epsilon$, where $\epsilon$ is a parameter to the algorithm.
- It runs multi-dimensional line search [34, 80] to find this optimal split. This additively increases or decreases $s$. If $RMSE_d - RMSE_c > \epsilon$, then $s$ increases by $\delta$ (the step size). Else, $s$ decreases by $\delta$.

At the beginning of a session, $s$ is set to an initial split $s_i$. This can be estimated empirically from video data (*e.g.,* Fig. 2.4) or can be set to a fixed initial value.

Multi-dimensional line search is sensitive to $\delta$. A smaller value can lead to delayed convergence when the scene complexity changes. A larger step size can result in oscillations around the optimal split. We have empirically chosen a step size of $0.005$ to balance these two concerns.

We also choose $0.5 \leq s \leq 0.9$. The lower limit ensures depth always gets more bandwidth than color. When available bandwidth is too low, $RMSE_d - RMSE_c$ may always exceed $\epsilon$, which will drive $s$ to 1, and this can degrade color significantly. To prevent this, we clamp $s$ at 0.9.

### 2.3.4 View Prediction and Culling

In addition to encoding RGB-D frames in 2D video, LiVo *culls* points that are not in the receiver's view. To do this, it must (a) determine the receiver's *frustum* and (b) efficiently remove points outside the frustum.

**Challenge: Frustum Prediction.** The frustum is determined by the pose (position and orientation) of the receiver's headset, as well as the parameters of the viewing device (such as the focal length and aspect

**Figure 2.5:** The red boxes are culled regions with black pixels in two successive tiles that video encoders can compress efficiently.

ratio). Today's headsets can provide continuously evolving values for these, and the receiver can transmit these to the sender. Even so, given the feedback delay between the receiver and the sender, the latter must *predict* the frustum of the receiver at the time when the receiver views the frame.

On-demand 360-degree video systems [93, 27] train viewport predictors from user data. In that setting, if many users watch the same video, learned predictors can be accurate. In a conferencing setting, it is less clear if learned predictors can be accurate, since each conference call is unique and not enough user traces might be available for the same content. We show (§2.4) that predictors trained from a few traces perform poorly relative to LiVo's approach.

**Challenge: Efficient Culling.** Given a frustum and a point cloud, the *culled point cloud* consists of points inside the frustum. However, RGB-D cameras do not provide point clouds; each camera produces an image, with color and depth of associated pixels. LiVo must efficiently determine which color and depth pixels correspond to points within the frustum.

**Frustum Prediction.** When culling a frame at time $t$, LiVo's sender must predict the receiver's frustum at $t + \Delta t$, where $\Delta t$ is the one-way delay from sender to receiver (including both network and processing delays at the two ends). LiVo obtains $\Delta t$ by halving a smoothed application-level RTT estimate.

LiVo predicts frustums using a Kalman Filter [48] on the 6 dimensions of receiver pose (position and orientation) [31]. Still, prediction errors can arise as a result of sudden user movement or errors in one-way delay estimates. To counter these, LiVo expands the predicted frustum by a guard-band of $\epsilon$. We have found an $\epsilon$ of 20 cm to be the sweet spot in the trade-off between compression efficiency and reconstruction accuracy (§2.4). Future work can explore other techniques to ensure better tracking accuracy.

**View Culling.** This step, performed *before* stream composition and depth encoding (§2.3.2), is invoked every inter-frame interval, and takes as input the viewer's frustum and the $N$ RGB-D frames. For each pixel in each image, it must determine if that pixel falls within the frustum or not, and replace culled pixels by a zero value (both for color and depth).

It can do this by (a) generating the point cloud, (b) determining whether a point is within the frustum or not and (c) back-project points within the frustum to the relevant camera's pixel. This can be slow.

LiVo speeds up culling by avoiding two coordinate transformations required for each point: from local to global coordinates before culling and vice-versa after. Instead, it *converts the frustum into the local coordinate system* of each RGB-D camera. Then, for each pixel, it obtains that pixel's local coordinates and determines if it lies within the frustum; if not, it zeroes out the color and depth of the pixel.

### 2.3.5 Other Details

Our LiVo implementation combines techniques from various sources to achieve high-quality, low-latency volumetric video streaming. We describe these briefly.

**Stream Synchronization.** WebRTC does not permit embedding frame numbers in video streams. Following prior work [25], the LiVo sender embeds a (pre-generated) QR code in each 4K depth and color tiled frame that encodes the frame sequence number (Fig. 2.3). The receiver decodes the QR code to obtain frame

sequence numbers. If both depth and color frames have not been decoded by the time necessary to render the point cloud, LiVo simply skips the frame.

**Receiver-side rendering.** The receiver must reconstruct point clouds from received 4K frames. To do this, it needs the parameters and positions of the RGB-D cameras; these are exchanged once during connection setup or whenever they change. The receiver uses these to transform the position of each pixel from each 4K frame into the global coordinate frame. Together with the corresponding color attributes, these constitute the reconstructed point cloud at the receiver. This point cloud may be more dense than necessary and may have more points because LiVo transmits a guard-band around the frustum (§2.3.4). To speed up rendering, LiVo first voxelizes the point cloud, then culls the point cloud further to the actual (current) frustum, following prior work [33, 61].

**Pipelining and parallelism.** To ensure frame-rate processing, LiVo consists of several stages that run in parallel, and each stage incurs a delay per frame of less than one inter-frame interval. The sender has 4 stages: capture, view generation, tiling, and transmission. The receiver also has 3 stages: receiving and synchronization, point cloud reconstruction, and rendering. Thus, the total end-to-end *processing* latency is within 180 ms. Each stage has a dedicated thread, which is connected to the next stage by a small inter-stage buffer (implemented using a thread-safe queue). We also employ parallelism within some stages when possible: *e.g.*, view generation, and point cloud generation.

**Minimizing the impact of packet loss.** We enable several WebRTC features, including negative acknowledgments, Picture Loss Indication (PLI), and Full Intraframe Request (FIR). Because 4K videos are large, the default Linux UDP socket buffer (213 KB) proved insufficient, so we increased it.

## 2.4 Evaluation

We compare LiVo to other alternatives, using both a user study and a trace analysis on real bandwidth traces. We also quantify its compute-efficiency.

| Videos | Duration (s) | People + Objects | Avg. Frame Size (MB) |
|---|---|---|---|
| band2; Musical performance | 197 | 9 | 11.1 |
| dance5; Dance | 333 | 1 | 10.8 |
| office1; Person working | 187 | 7 | 10.6 |
| pizza1; Food and party | 47 | 14 | 13.8 |
| toddler4; A child playing games | 127 | 3 | 10.6 |

**Table 2.3:** Summary of 5 videos in Panoptic Dataset.

### 2.4.1 Methodology

**Implementation.** Our LiVo implementation (Fig. 3.1) contains 15K lines of C++ and few hundred lines of Unity. It implements pipelined capture, view generation and tiling using Boost [9] and OpenMP [86]. It also uses: Eigen [23] for linear algebra operations for point cloud transformation; the Kalman Filter implementation in OpenCV; WebRTC and `NVENC` (NVIDIA Video Codec) in Gstreamer [81] for color (H.265, BGRA) and depth (H.265, Y444_16LE); and Open3D [133] for rendering on PC or on a headset using Unity. LiVo captures RGB-D frames using the Azure Kinect SDK [5], and synchronizes them as described in [108].

**Evaluation testbed.** Our evaluation uses two machines, both with 32 GB RAM and connected using 1 Gbps Ethernet. Both have modest computing resources, by today's standards: one is an Intel i7-8700K @ 3.70GHz, with one NVIDIA GeForce GTX 1080 Ti; the other is an Intel Core i9-10900KF @ 3.70GHz and one NVIDIA GeForce RTX 3080.

**Traces and Datasets.** To compare LiVo against other alternatives while subjecting all of them to the same workload, we use volumetric video *replay* instead of live transmission.

***Volumetric videos.*** We use videos from the Panoptic Dataset [46], which are captured at 30 fps using 10 Kinect v2 RGB-D cameras. We select 5 videos (Table 2.3) of duration 1–5 minutes that capture 1–6 participants performing activities which are excellent for a fully immersive live experience, *e.g.,* dance, musical band, party, and games. The average raw frame size in these videos is 10.5–13.5 MB, much larger than that considered in prior work.

***User traces.*** When a user interacts with a volumetric video by moving to change perspective, the sequence of her instantaneous poses (position and rotation) constitutes a user trace. There are no publicly available user traces for the videos in Panoptic dataset, so we collected these under an IRB-approved study. In a 30–45 minute session, we asked each user to view 2–3 videos selected randomly. Users could freely move around in a sufficiently empty room while viewing the videos and their headset recorded their instantaneous poses. We collected three traces for each video.

***Network traces.*** We replay the videos on two traces of network bandwidth. **trace-1** (Table 2.4) captures the throughput variation measured in a home WiFi setting [65]. **trace-2** captures a WiFi connection while moving around in a shopping mall [64]. These traces likely exhibit variability (§2.4.7) comparable to broadband connections, but not their capacity. As such, by themselves, these traces cannot support volumetric video live streaming. For this reason, we scaled *trace-2* by 15× and *trace-1* by 10× to reach mean throughputs around 90 Mbps and 217 Mbps, respectively; the former corresponds to current broadband bandwidth, while the latter is representative of broadband bandwidth in 3–4 years [14].

***Trace replay.*** This reads RGB-D frames from disk at 30 fps and feeds them into the LiVo pipeline at the sender (Fig. 3.1). Using the frame sequence number, the receiver calculates the corresponding user frustum from the selected user trace file, then culls and renders the point cloud. We replay the network traces using Mahimahi [77] to emulate the bandwidth conditions between sender and receiver.

**Comparison Alternatives.** We compare the performance of LiVo with other alternatives inspired by live volumetric video streaming systems such as Holoportation [87], Project Starline [56], LiveScan3D [55] and MetaStream [30] (Table 2.1). None of these schemes are bandwidth-adaptive, and all of them achieve live transmission on a network with sufficient bandwidth by sending more constrained views (*e.g.,* single person or torso, Table 2.1). Recall that these constrained views require an order of magnitude less bandwidth than full-frame transmission. Most of these do not provide open-source implementations, so we resort to

mimicking their essential video compression and transmissions strategies, but adapt these for full-frame transmission, so we can do a more even comparison with LiVo.

***Draco-Oracle.*** Volumetric video on-demand systems such as ViVo [33] and GROOT [61] and live systems such as MetaStream [30] use Draco [22], an open-source point cloud compression library. They have shown that Draco either provides better compression ratio, or lower encoding latency than other point cloud compression schemes like PCL [92], G-PCC [29], and V-PCC [95]. Draco by itself is not rate-adaptive, which is why prior work that uses Draco is not bandwidth-adaptive. To understand what would happen if Draco were to develop rate-adaptivity, and yet transmit the information that LiVo does in our evaluations, we designed a Draco-Oracle: given a target bandwidth and a perfect estimate of a receiver's frustum (*perfect culling*), it picks the highest quality compression for the point cloud that fits within the target bandwidth.

To do this, we compute *offline* a table from each video frame and each user trace. This maps, for each video frame and user frustum and each Draco compression level (Draco has 10 of these) and quantization parameter (Draco supports 31 of these), the *time to compress* the perfectly-culled frame, and the *compressed size* of this frame. Then, during playback, we find that quantization parameter and compression level whose compressed size fits the bandwidth estimate at that instant, and whose time to compress respects the frame rate. If no such entry exists, we record a *stall* for Draco-Oracle.

At 30 fps, Draco-Oracle exhibits over 90% stalls for most of our video sequences. So, our evaluations use a lower frame rate, 15 fps: prior work [55] has also used 15 fps for volumetric video streaming when using point-cloud compression.

***LiVo-NoCull.*** This baseline runs LiVo without culling. In addition to quantifying the importance of culling, LiVo-NoCull is intended to mimic one small aspect of Project Starline [56], which also streams RGB-D captured from multiple depth cameras over 2D video streams using WebRTC and performs 3D reconstruction in the receiver. Starline streams 2D videos at a fixed quality, but in LiVo-NoCull we include bandwidth splitting (§2.3.3) and bandwidth-adaptivity (in **??**, we also quantify what happens if we disable

bandwidth-adaptivity). So, this baseline seeks to understand what would happen if Starline were to be bandwidth-adaptive. Starline is an impressive system that differs in *many* other respects from LiVo, so we do not intend LiVo-NoCull to represent Starline's performance. We run LiVo-NoCull and LiVo at 30 fps on all our videos.

***MeshReduce.*** MeshReduce [45] is a mesh-based live volumetric video streaming system, which represents any 3D surface as a collection of interlocking triangles, not a collection of points. The sender captures a RGB-D frame from off-the-shelf RGB-D cameras, reconstructs a per-frame mesh, encodes the geometry and color separately, and transmits over 2 TCP socket connections. It compresses the mesh geometry using Draco, and the mesh texture using H.264.

MeshReduce employs *indirect* bandwidth adaptation (§2.2): using a profile obtained from an offline analysis, it determines the best compression parameters for a given level of available bandwidth. The publicly available MeshReduce implementation decides on these parameters based on the average bandwidth availability in a trace, so we use this approach in our experiments.

**Metrics.** Aside from end-to-end latency and frame rate, we also measure objective point cloud quality. We use PointSSIM [2] (or PSSIM), which is analogous to SSIM for 2D images, but extends it to 3D by exploring a higher dimensional feature space that includes geometry, normals, curvatures and color attributes. PSSIM separately captures the quality of depth and color. It ranges from 0 to 100; higher values are better, and values in or above the high 80s are generally considered good.

Prior work such as ViVo [33], Vues [69], GROOT [61] has used 2D metrics such as SSIM and PSNR on rendered 2D image of the point cloud. However, the point cloud coding community considers these metrics inadequate for several reasons: 3D to 2D projection introduces errors; 2D pixel size influences quality but points in a point cloud do not capture size; a slight change in point position due to depth coding loss results in a large quality drop for 2D metrics though the geometrical structure of the object in the

| Trace Names | Bandwidth (Mbps) | | | | |
|---|---|---|---|---|---|
| | **Mean** | **Max** | **Min** | **90$^{th}$** | **10$^{th}$** |
| *trace-2* | 89.20 | 106.37 | 36.35 | 98.09 | 80.52 |
| *trace-1* | 216.90 | 262.19 | 151.91 | 234.41 | 191.52 |

**Table 2.4:** Statistics of the bandwidth traces.

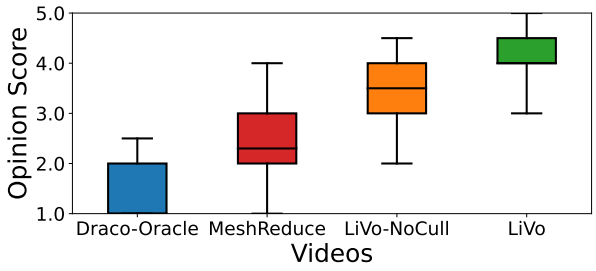



**Figure 2.6:** Aggregated opinion scores for 4 methods.

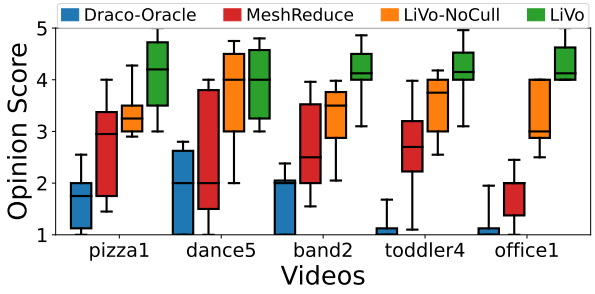



**Figure 2.7:** Opinion scores across 5 videos.

scene is preserved; 2D metrics depend on the distance of the viewer from the point cloud; and 2D metrics are influenced by color of background pixels.

PSSIM is not defined for meshes, so to compare MeshReduce against LiVo, we sample points from the rendered mesh to generate a point cloud, then compute PointSSIM.

### 2.4.2 User Study

**Setup.** We conducted an IRB-approved user study to measure the perceptual quality of LiVo against other alternatives. Each participant viewed 2-3 videos from Table 2.3 over 45–60 minutes. For each video, we randomly selected one user trace and one network trace, and the user passively observes the video as seen by the user who contributed the user trace (§2.4.1). Each participant rated the videos across 4 schemes on a Likert scale of 1 (worst) to 5 (best) with respect to the ground truth and (optionally) left comments. These experiments used the replay infrastructure described in §2.4.1.

**Perceptual Quality Results.** Fig. 2.6 shows the distribution of perceptual quality scores for each of the 4 schemes across 20 participants. Each scheme received a total of 57 ratings across all combinations of <video, user trace, network trace>. Draco-Oracle performs poorly with a Mean Opinion Score (MOS) of 1.5 due to 3

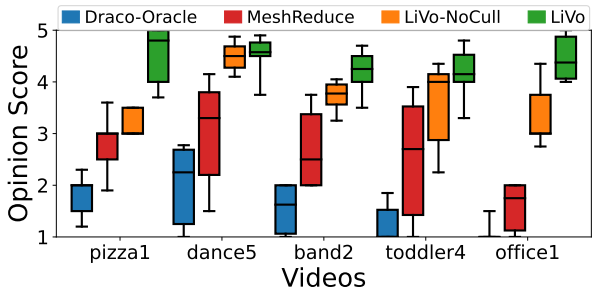



**Figure 2.8:** Opinion scores for *trace-1*.

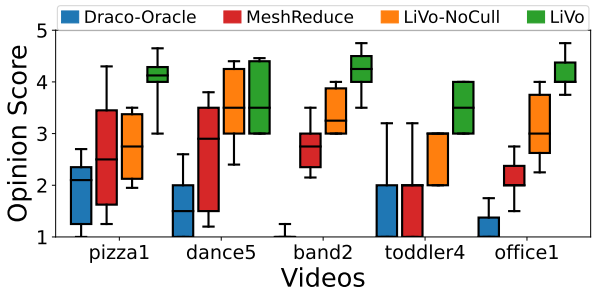



**Figure 2.9:** Opinion scores for *trace-2*.

factors: Draco is streamed at 15 fps; even at 15 fps, it experiences $36-98\%$ stalls; and Draco-Oracle settles for lower quality to remain within compute and bandwidth budgets. While other work on volumetric video streaming has used Draco to achieve good quality, in our setting with large point clouds from full-scenes, Draco proves to be a poor choice as it is both compute-intensive and compression-inefficient.

MeshReduce has a median opinion score of around 2.3 (and MOS of 2.5), higher than Draco-Oracle. In theory, MeshReduce should perform worse than Draco-Oracle, since it doesn't use a bandwidth oracle. It performs better likely because it uses meshes, which have better perceptual quality than point clouds in general and also because MeshReduce incurs fewer stalls.

By comparison, LiVo-NoCull has a median opinion score of 3.5 (and MOS of 3.4), higher than Draco-Oracle and MeshReduce; its use of 2D video encoding and *direct* bandwidth-adaptivity lead to higher perceptual quality. LiVo has the highest MOS of 4.1 and median opinion score of 4.0, $14-20\%$ better than LiVo-NoCull and $64-74\%$ better than MeshReduce. LiVo's use of culling results in less data, so it encodes at higher quality and incurs fewer stalls.

Across 5 videos (Fig. 2.7), LiVo and LiVo-NoCull perform significantly better than Draco. LiVo also outperforms MeshReduce by $48-135\%$ in MOS, and it outperforms LiVo-NoCull by $10-33\%$ in MOS across all the videos. For the *dance5* video, the median opinion score for LiVo and LiVo-NoCull is comparable. This video contains only one person and no objects, so requires less bandwidth, and culling does not provide any benefits.

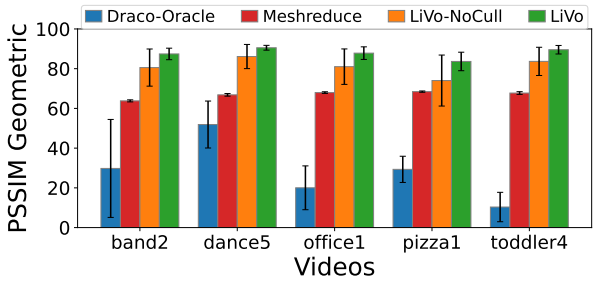



**Figure 2.10:** Geometry PSSIM.

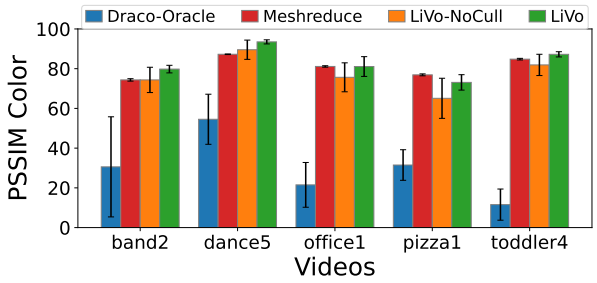



**Figure 2.11:** Color PSSIM.

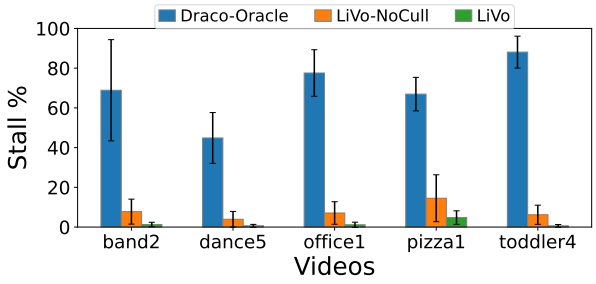



**Figure 2.12:** Stalls in 3 methods.

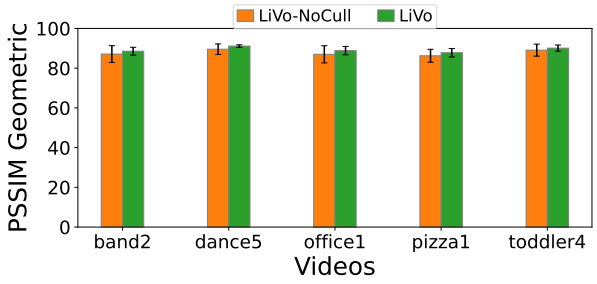



**Figure 2.13:** PSSIM, no stalls.

Across our bandwidth traces, quality improves with increasing bandwidth. In *trace-1* (Fig. 2.8), LiVo's MOS is over 3.9, going up to 4.5 for the *pizza1* video. The performance improvement over LiVo-NoCull is $6.6 - 43.9\%$, except for *dance5* on trace-1 for the reasons mentioned above. The *direct* bandwidth adaptation in LiVo-NoCull shows $2.0 - 113.3\%$ improvement over MeshReduce, which uses an *indirect* adaptation. When bandwidth is low (*trace-2*, Fig. 2.9), LiVo's MOS is 4.1 due to the loss of quality at lower bitrate that affects both color and geometry. At lower bandwidth, multiple competing factors influence opinions — lower overall quality and higher stalls vs. quality improvement due to culling.

**Qualitative feedback.** Participants were optionally asked to provide comments summarizing their experience. We received 184 responses across 20 participants. We manually examined these and classified the responses into 3 categories (quality, frame rate, and stalls). For each category, we assigned a high/medium/low rating to each response (Table 2.5). Descriptions such as "smooth" or "seamless" indicate low stalls and high frame rate, "more distorted" indicate low quality, and "some glitches" indicate medium stalls.

| Schemes | Frame Rate (in %) | | | Stalls (in %) | | | Quality (in %) | | |
|---|---|---|---|---|---|---|---|---|---|
| | L | M | H | L | M | H | L | M | H |
| Draco-Oracle | 94.4 | 5.6 | 0.0 | 0.0 | 12.5 | 87.5 | 35.0 | 45.0 | 20.0 |
| MeshReduce | 73.3 | 26.7 | 0.0 | **90.9** | 9.1 | 0.0 | 61.3 | 34.1 | 4.6 |
| LiVo-NoCull | 15.0 | 25.0 | 60.0 | 25.0 | 50.0 | 25.0 | 12.5 | 53.1 | 34.4 |
| LiVo | 0.0 | 0.0 | **100.0** | **70.8** | 25.0 | 4.2 | 6.1 | 33.3 | **60.6** |

**Table 2.5:** Percentage of comments providing a Low (L), Medium (M), or High (H) rating for frame rate, stalls, and quality.

Draco-Oracle performs worst in stalls (high=87.5%); every response describes "glitches" or "blanks" for Draco-Oracle. None of the responses complain about high stalls for LiVo; about 96% of responses discussing stalls for LiVo agree that LiVo had less stalls (low=71%, medium=25%). MeshReduce rated best in terms of stalls: this is because it conservatively uses a lower frame rate (no response rated it high in terms of frame rate, compared to 100% for LiVo), and encodes at a lower quality (only 4.6% of responses rated it high, compared to 60.6% for LiVo). Other alternatives also perform poorly in terms of quality: only 34% or responses rated LiVo-NoCull high, only 20% rated Draco-Oracle high. 95% of the participants said MeshReduce has low to medium quality, with many responses of the kind "triangles are disturbing", "block of black mass", and "blobs". LiVo and LiVo-NoCull are considered superior in terms of frame rate than MeshReduce and Draco-Oracle. However, for LiVo, some participants (6 out of 20) wrote "corners of the video loads slowly", "delay when camera changes", and "blanks after sharp motion", likely because of artifacts resulting from prediction error.

### 2.4.3 Objective Quality Comparisons

**Setup.** We use the same experimental setup as described above but now the receiver logs the received point cloud. We compare each point cloud against ground truth for the same 5 videos, 3 user traces per video and 2 network traces.

**Results.** Fig. 2.10 and Fig. 2.11 separately plot PSSIM for depth and color across 4 schemes. For each video sequence, we aggregate the geometry and color PSSIM values over all user traces and network traces. We

use a metric value of 0 for frames that experience stalls. Draco-Oracle achieves the lowest geometry and color score with mean value of 28.3 (std. 19.1) and 29.9 (std. 19.8) respectively, which correlates with the lower opinion scores in our user study.

LiVo outperforms LiVo-NoCull, achieving a mean PSSIM geometry of 87.8 (std. 3.7) compared to 81.0 (std. 9.5) for LiVo-NoCull. This is consistent with our user study and demonstrates the benefits of culling. LiVo's benefits relative to LiVo-NoCull are higher on videos with more subjects (Table 2.3); in such videos, users often focus on a few subjects at any given instant, so culling is very effective. MeshReduce has significantly lower PSSIM geometry of 67.0 (std. 1.8) because it uses Draco, which is less bandwidth-efficient, and because it adapts indirectly to bandwidth.

LiVo is slightly better than LiVo-NoCull by the PSSIM color metric (80.9, std. 5.06 vs. 82.9, std. 7.59). This is because our bandwidth splitting allocates only a small fraction of available bandwidth to color, so any bandwidth reductions from culling for color are proportionally lower. Interestingly, MeshReduce compares better with LiVo for color (77.30, std. 10.6). We conjecture that in meshes, color is less affected by depth distortion than for point clouds.

Stalls impact objective quality, and Fig. 2.12 shows the aggregate stalls across videos for 3 schemes. We omit MeshReduce since it doesn't experience stalls: it uses reliable transmissions and its parameters are designed to match average bandwidth, so rather than stalls it exhibits varying frame rates. Draco-Oracle suffers from high mean stall rate of 69.3%. Even on the *dance5* video which has only 1 participant and no objects in the scene (Table 2.3), it has a stall rate of 37.8%. LiVo-NoCull incurs an average stall rate of 7.9% (std. 7.5%) while LiVo incurs 1.7% (std. 2.3%); culling accounts for this difference. LiVo's infrequent stalls occur when the rate-adaptive codec overshoots the bandwidth target.

Culling by itself improves quality: LiVo improves over LiVo-NoCull by an average difference of 2% for PSSIM geometry (Fig. 2.13) and 1% for color (omitted for brevity).

| Component | LiVo-NoCull (in ms) | | LiVo (in ms) | |
|---|---|---|---|---|
| | **Mean** | **Std** | **Mean** | **Std** |
| Capture Frame | 32.75 | 6.81 | 29.85 | 5.09 |
| View Generation | 19.50 | 8.93 | 30.59 | 2.91 |
| Tile Creation | 7.16 | 5.08 | 5.49 | 0.86 |
| Synchronization | 31.74 | 14.24 | 32.33 | 4.60 |
| Point cloud Reconstruction | 21.80 | 2.47 | 15.83 | 3.64 |
| Rendering | 5.16 | 1.32 | 4.91 | 1.23 |
| WebRTC Send-Recv | 133.19 | 2.13 | 132.82 | 0.65 |
| **Total latency** | **251.30** | **23.32** | **251.82** | **6.93** |

**Table 2.6:** Component-wise latency.

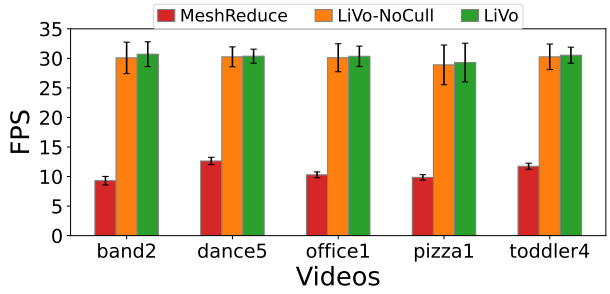



**Figure 2.14:** FPS comparison for *trace-1*.

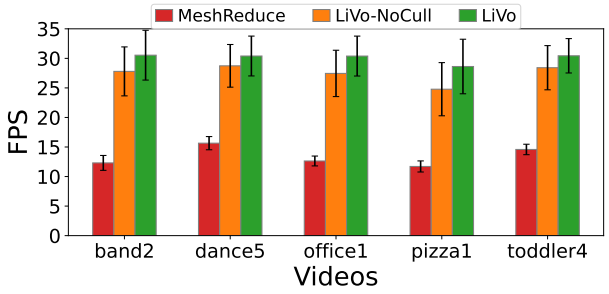



**Figure 2.15:** FPS comparison for *trace-2*.

Overall, both by objective and perceptual measures, LiVo consistently outperforms all baselines for our videos.

### 2.4.4 Performance Validation

Table 2.6 lists the per-component latency for LiVo and LiVo-NoCull. Both achieve end-to-end latency within 300 ms, in line with production 2D video conferencing systems such as Zoom, Meet, *etc.* [58, 25]. For LiVo, the sender processing latency is around 64 ms while the receiver processing latency is 53 ms. In contrast, LiVo-NoCull incurs lower processing latency at the sender since it does not perform view culling, but incurs the culling cost at the receiver. The largest latency is incurred by WebRTC transmission (color and depth transmissions are parallel), which is about 137 ms. Much of this is attributable to the jitter buffer in WebRTC: we use 100 ms [97], consistent with current practice.

**Figure 2.16:** Block artifacts arising from unscaled depth encoding when we use naive H.265 YUV 16-bit variant.

Fig. 2.14 plots the rendering frame rate for LiVo-NoCull and LiVo for *trace-1*. LiVo maintains 30 fps at a lower standard deviation than LiVo-NoCull. For *trace-2* (Fig. 2.15), LiVo maintains close to 30 fps (std. 0.7), while LiVo-NoCull achieves 28 fps (std. 1.8). At lower bandwidth, LiVo-NoCull experiences more stalls (mean fps drops to 24 fps for *pizza1*) than LiVo, as compressed frames occasionally exceed the bandwidth budget. MeshReduce achieves a mean frame rate of 12.1 fps, 2.5x lower than LiVo, even though it utilizes all cores on the sender side at 100% to encode frames. MeshReduce's frame rate for *trace-2* is slightly higher than *trace-1*, because it decimates the mesh more to fit the lower bandwidth, and the encoder needs to do less work. MeshReduce has been shown to outperform LiveScan3D [55] and Holoportation [87], so we have not included these in our evaluation.

### 2.4.5 Validating Design Choices

**Depth Encoding.** LiVo encodes in Y-channel (luminance) of 3 channel 16-bit variant of H.265 encoding (Y444_LE). If the depth values aren't scaled to the full range, they produce artifacts as seen in Fig. 2.16. These artifacts arise since nearby depth values are mapped to same quantization bin within the 16-bit range. So LiVo scales the depth values. We also experimented with a strategy proposed by prior work [100, 91, 32]

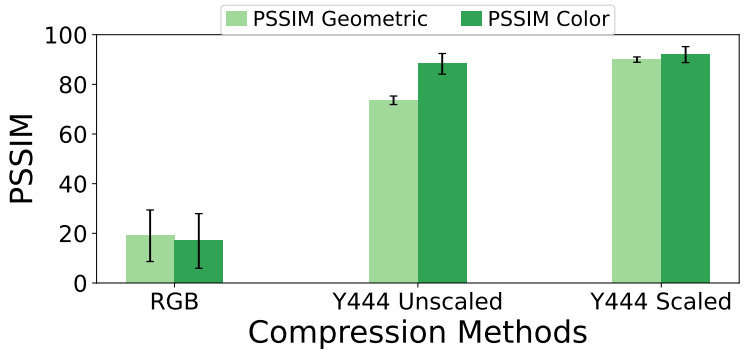



**Figure 2.17:** Comparison between realsense vs our approach.

| Guard Band | Prediction Window Size | | | |
|---|---|---|---|---|
| | W=5 | W=10 | W=20 | W=30 |
| 10 | 98.44 (0.57) | 96.69 (0.54) | 94.37 (0.57) | 91.76 (0.57) |
| 20 | 99.26 (0.62) | 98.37 (0.62) | 96.13 (0.62) | 93.95 (0.62) |
| 30 | 99.61 (0.66) | 98.97 (0.66) | 97.22 (0.67) | 95.41 (0.67) |
| 50 | 99.89 (0.73) | 99.56 (0.73) | 98.46 (0.73) | 97.18 (0.73) |

**Table 2.7:** Accuracy of culling using Kalman Filter by varying the guard band (in cm) for *band2*. The numbers in brackets represent fraction of points within frustum.

which colorizes each 16-bit depth pixel into a 3-channel RGB pixel using a depth to color conversion table. Fig. 2.17 shows that LiVo's scaled depth encoding achieves the highest geometry and color PSSIM relative to this approach and to unscaled depth encoding.

**Kalman Filter buffer size.** Kalman filter prediction error depends on the prediction horizon. LiVo targets prediction horizons of about 10 frames (300 ms). Predictions help culling, which can reduce data volume, but erroneous predictions can impact quality. LiVo uses a static guard-band $\epsilon$ of 20 cm around the frustum to minimize quality loss due to prediction errors. Table 2.7 shows that for guard-bands up to 30 cm, LiVo's accuracy and data volume are relatively insensitive to the choice of guard-band, hence a static guard-band works well. We have verified this for other videos as well.

**Bandwidth Splitting.** Fig. 2.18 plots compares PSSIM for different static splits with LiVo's dynamic split (§2.3.3) for bitrates ranging from 60-120 Mbps for *office1* (similarly, color in Fig. 2.19). Dynamic splitting is with 0.5 PSSIM points for geometry and 3 PSSIM points for color relative to the best static split. As available bandwidth increases, both geometry and color PSSIM values are within 0.5 of the best static split. This

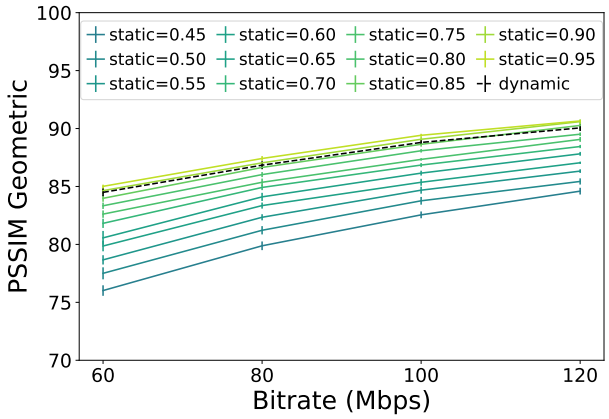



**Figure 2.18:** PSSIM Geometric for static vs dynamic split.

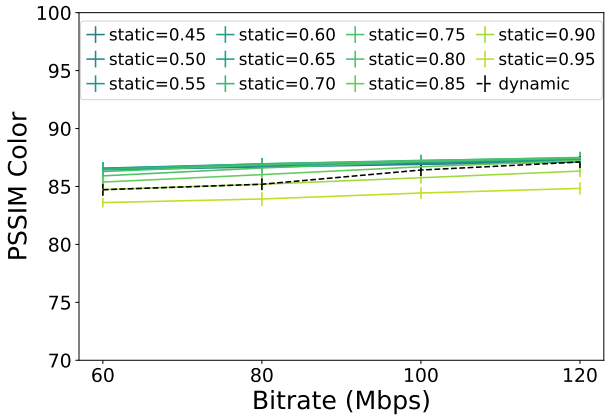



**Figure 2.19:** PSSIM Color for static vs dynamic split.

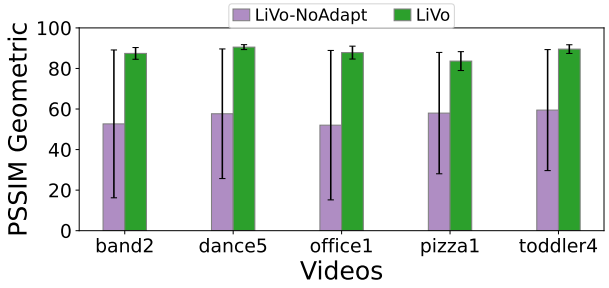



**Figure 2.20:** PSSIM Geometric, LiVo-NoAdapt vs LiVo.

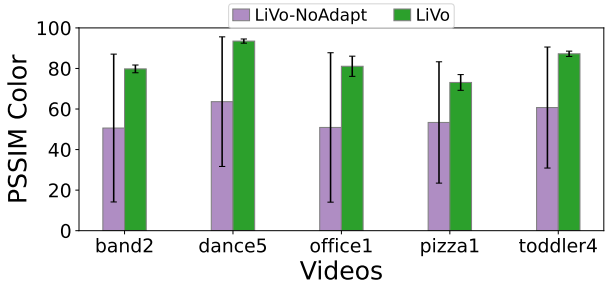



**Figure 2.21:** PSSIM Color, LiVo-NoAdapt vs LiVo.

demonstrates the effectiveness of dynamic splitting. Moreover, dynamic splitting converges quickly, within 5-10 frames (figure omitted for brevity).

**Bandwidth Adaptation.** What if LiVo did not use bandwidth-adaptation and view culling at all, but instead used fixed quality parameters? We set fixed color QP (quantization parameter) to 22 and depth QP to 14; Starline [56] uses these values. Fig. 2.20 and Fig. 2.21 show that this LiVo-NoAdapt has significant quality drop of $30-41\%$ for geometry and $27-37\%$ for color, with PSSIM going below 60.

**Frustum Prediction.** To estimate quality impairment as a result of our predictive culling, we compare it against LiVo with perfect culling (using predictions obtained from the user trace). The average quality difference in PSSIM geometry and color is $1\%$ across all 5 videos when run on *trace-1*.

| Method | # Hidden Units | Position (m) | Rotation (degree) |
|---|---|---|---|
| MLP Trained on 1 user trace | 3 | 1.01 | 135.86 |
| | 32 | 0.59 | 80.46 |
| | 64 | 0.24 | 58.24 |
| MLP Trained on 2 user trace | 3 | 0.40 | 33.34 |
| | 32 | 0.09 | 3.69 |
| | 64 | 0.07 | 2.17 |
| Kalman Filter | - | 0.04 | 7.19 |

**Table 2.8:** Comparing prediction errors in Kalman Filter-based to learning-based prediction methods.

Prior work has trained frustum predictors from user traces [69, 33] for on-demand volumetric videos. This doesn't quite apply to conferencing, since user interactions depend on content, which can be different for different content. That said, we evaluate (Table 2.8) whether an MPL with 3 hidden layers used in ViVo [33] could learn from a small number of our traces. We find that it may be possible to match LiVo's predictor only with a large number of hidden layers (64); with the three that ViVo uses, prediction errors are unacceptably high.

### 2.4.6 Depth vs Color Distortion

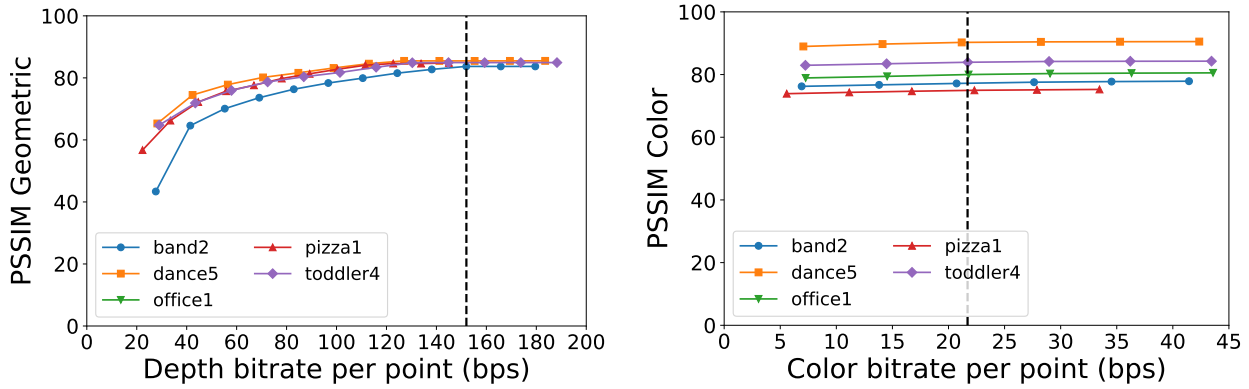
Fig. 2.22 studies variability in PSSIM quality metric in depth (sim. color) when we fix color (sim. depth) bitrate and vary depth bitrate. Fig. 2.22a shows how PSSIM Geometry metric varies when we increase depth bitrate (normalized as depth bitrate per point); similarly report PSSIM color in Fig. 2.22b. We observe that depth varies a lot with increase in bitrate before it flattens, while variation in color quality is minimal. Also, the bitrate that needs be allocated for depth is almost 7× higher before it saturates (around vertical dotted line). This confirms that depth is much more sensitive to color and needs careful bitrate allocation.

### 2.4.7 Variability in Bandwidth Traces

We show the variability in the 2 traces we pick - *trace-1* and *trace-2* in Fig. 2.23.

## 2.5 Related Work

Table 2.1 explains how LiVo differs from related work. We discuss salient differences below.




**(A)** Depth distortion vs depth bitrate.

**(B)** Color distortion vs color bitrate.

**FIGURE 2.22:** PSSIM distortion across videos when bitrate is varied

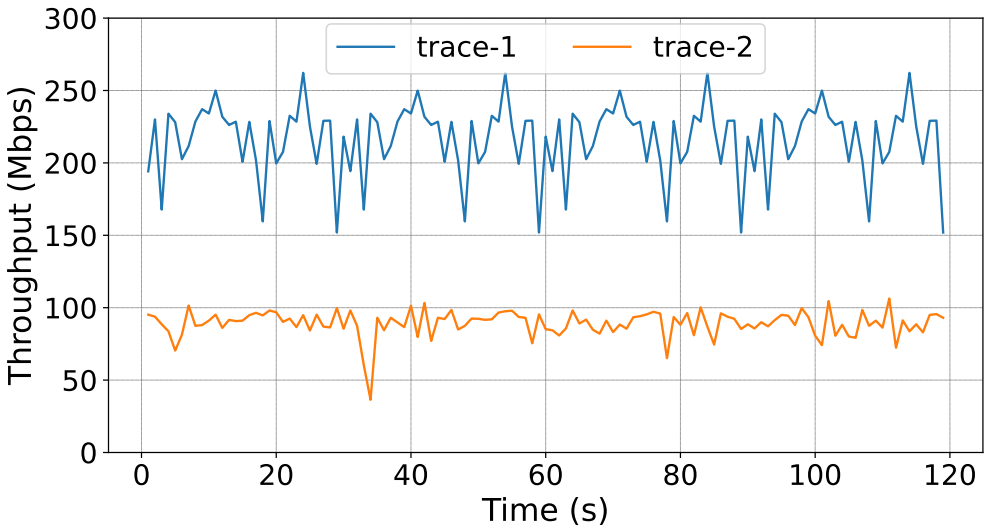



**FIGURE 2.23:** Variability in the 2 network traces we consider.

**On-demand Streaming.** A line of work has explored on-demand volumetric video streaming [33, 61, 69, 125, 116, 12], and focused on reducing data rate and improving decoding efficiency. ViVo [33] culls occluded points or those outside the predicted frustum. Groot [61] employs parallelism to improve Draco decoding efficiency and achieve 30 fps. Fumos [66] improves data rate by exploiting inter-frame redundancy using a neural codec and supports high frame rate decoding. In contrast, LiVo avoids point cloud compression to achieve performant two-way live streaming.

**Live Streaming and Conferencing.** LiveScan3D [55] uses a fast binary compression method, zstd, which achieves low compression ratios. Holoportation [87] achieves conferencing by compressing a 3D representation of a single person using LZ4 binary encoding, which doesn't scale to larger scenes. Starline [56], a conferencing system, streams upper body views in 3D over fixed quality 2D video streams; §2.4 describes how it differs from LiVo. FarfetchFusion [60] streams only the face, and is optimized for mobile devices. MeshReduce [45] supports full-scene conferencing using a mesh, but achieves a lower frame rate and lower quality (§2.4). MetaStream [30] uses Draco encoding to stream a single person at 30 fps by reducing the number of points in the capture point cloud. Unlike LiVo, none of the live volumetric streaming systems support full-scene 2-way live streaming with bandwidth adaptation at 30 fps.

**Other 3D Formats and Compression Standards.** Besides point clouds, prior work attempts to capture 3D scenes as textured meshes [78, 15, 87, 21, 79, 45]. Meshes are generally higher quality representations that are much more bandwidth-intensive; future work can explore mesh-based two-way bandwidth-adaptive live streaming. Besides Draco, prior work has proposed other point cloud compression techniques: G-PCC [29], V-PCC [95], and PCL [92]. Draco achieves faster compression with higher compression ratios than these, so we base our evaluations on it. Recently, Hermes [116], patchVVC [12], and DeformStream [62] have attempted to exploit inter-frame redundancy in volumetric videos, but these have high encoding latency so can only be used for on-demand streaming.

**Learned 3D Representations.** Recent work uses learned 3D representation such as Neural Radiance Fields (NeRFs) [74] and Gaussian Splatting (GSplat) [50] for streaming and conferencing. Many papers [98, 117, 105, 67, 115, 103, 28, 63] have explored learned 3D representations for on-demand streaming. Tele-Aloha [111] has used Gsplat for conferencing but constrains only to upper torso of a person like Starline [56]. Though these representations can generate high-quality rendering superior to point clouds or meshes, they suffer from high training and rendering overheads rendering them currently unsuitable for live or conferencing systems.

**Quality Metrics.** Other work has explored different 3D objective quality metrics, point2point-PSNR [110], PCQM [73], GraphSIM [123]. These compare point positions, and attributes such as colors, normals and curvatures in 3D. We choose PointSSIM for measuring quality since it can measure both geometry and color distortions by directly extending the popular SSIM metric to 3D. Future work can evaluate LiVo using other metrics.

## 2.6 Conclusions

LiVo enables bandwidth-adaptive conferencing of full-scene volumetric video by maximally leveraging 2D video compression, rate-adaptive codecs, and real-time transport. Both user studies and objective quality assessments show it outperforms baselines while maintaining its performance goals. Future work can explore better frustum prediction techniques, extend the work to support multi-way conferencing, and explore adapting LiVo for mobile devices.

# Chapter 3

# GS-NFS: Bandwidth-adaptive Streaming of Dynamic Gaussian Splats and Point Clouds

## 3.1 Introduction

Video over the Internet has effected societal and economic transformations. A technology with potentially similar transformative power, 3D video, is on the horizon. In 3D video, each frame depicts a representation of a scene in three dimensions, and viewers can view the scene from any perspective. This technology will enable entirely new ways of viewing sporting events, and new modes of instruction delivery, as well as new visual story-telling paradigms.

### 3.1.1 Gaussian Splatting

At the core of any 3D video technology is a technique to represent a 3-D scene. While many representations exist, 3-D Gaussian Splats [51] (or 3DGS) have been the subject of recent research interest. 3DGS represents a scene as a collection of 3-D Gaussians. Each Gaussian represents a small, ellipsoidal volume, and is defined by its position in 3-D and a set of attributes. The Gaussians and their attributes are learned by training on a set of camera images capturing a scene from multiple perspectives. For this reason, we refer to the set of Gaussians and their attributes as a 3DGS model.

More precisely, a 3DGS model is a collection of $N$ 3D Gaussians, each with the following *attributes*:

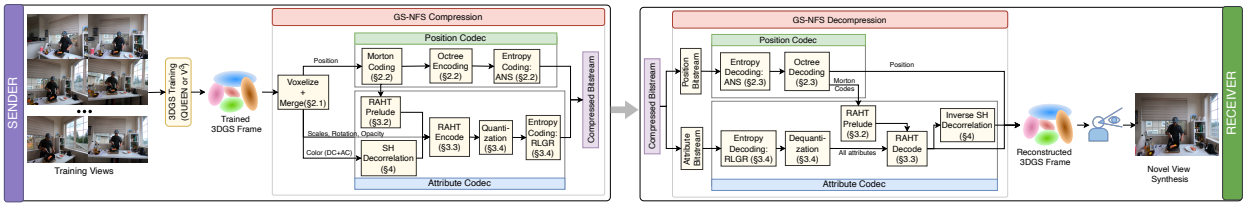



**Figure 3.1:** GS-NFS Architecture. Blocks in yellow are GPU-resident components.

- Mean ($\in \mathbb{R}^3$) represents the position of the Gaussian in 3D space (*i.e.*, the center of the ellipsoid).
- Scales ($\in \mathbb{R}^3$) define the size of the ellipsoid along its three axes.
- Rotation ($\in \mathbb{R}^4$) defines the orientation of the ellipsoid in 3D space, typically represented as a quaternion.
- Opacity ($\in \mathbb{R}$) represents the transparency of the Gaussian.
- Colors: In 3DGS, spherical harmonics (SH), which are functions defined on a surface, represent view-dependent color, capturing realistic lighting effects, material properties, and reflections. Each SH function is represented by three coefficients corresponding to three colors (RGB). The coefficients of a degree $d = 0$ SH—the DC coefficients—represent the overall color of the Gaussian. Higher degree ($d > 0$) AC coefficients capture more complex lighting and shading effects. A common choice of SH degree is 3, which results in 48 SH coefficients per Gaussian.

Given a 3DGS model and a viewpoint, a rendering algorithm [51] produces an image of the view of the scene from the given perspective. While training a 3DGS model can take minutes, rendering only requires milliseconds.

**Dynamic 3DGS.** A sequence of 3DGS frames constitutes a *dynamic* 3DGS video, often referred to as 4DGS (time being the fourth dimension). In this paper, we focus on delivery of 4DGS content. This delivery is challenging, because each 3DGS frame can be large. Each Gaussian requires about 236 bytes to represent its attributes. 3DGS frames with a million Gaussians are not uncommon, so a single frame can be 236 MB in size. By contrast, a single frame of 4K UHD video requires 24-30 MB, an order of magnitude less.

### 3.1.2 4DGS Compression

To reduce the size of 4DGS, recent work has explored compression techniques. In addition to reducing size, compression can be used to encode 4DGS at different bitrate levels for streaming, as for 2D video. Two *complementary* approaches have emerged (§3.6 discusses related work in detail).

*During-training* compression techniques reduce information in a variety of ways. QUEEN [28], 4DGC-Pro [129], and CompGS++ [68] reduce the number of Gaussians by using temporal prediction between key frames and their immediately following frames. A frame following a key frame is represented using *residuals*—differences with respect to the key frame. Other approaches exploit human tolerance for small color distortions. Vega [52] compresses SH (color) coefficients aggressively by first grouping semantically-related Gaussians, then training an MLP for each Gaussian group. QUEEN [28] and 4DGC-Pro [129] also train a quantizer that can learn optimal quantization levels for SH coefficients. Quantization reduces the number of bits required to represent SH coefficients at the expense of information loss.

*Post-training compression* techniques take a sequence of 3DGS frames as input and encode Gaussian attributes using 2-D or 3-D video codecs. For example, $V^3$ [114] encodes Gaussian attributes in images and compresses the image sequence using a H.264 codec. MesonGS [121] and LTS [104] extend point-cloud compression techniques, such as G-PCC [29] or Draco [22], to compress 4DGS. Post-training compression has two advantages: (a) it can be faster than during-training compression ([28] reports about a 20% increase in training time with a during-training quantizer), and (b) it can help encode 4DGS videos at different bitrates without re-training.

### 3.1.3 Approach, Challenges, and Contributions

**Goal.** We present GS-NFS (or GS-NFS)*, a post-training compression method for 4DGS which can encode and decode at full frame-rate, 30 frames per second (fps). It is also mobile-friendly; on a mobile GPU, it

*NFS, or Need for Speed, is a popular car racing game.

can decode some 4DGS sequences at 25 fps. No prior work on post-training compression has achieved full frame rate encode and decode.

Full frame-rate decoding is obviously useful; without that, it will not be possible to play 4DGS videos at frame rate. We argue that full frame-rate *encoding* is useful for three important reasons. First, for on-demand streaming, it will be necessary to encode 4DGS videos at different bitrates. At scale, an efficient encoder can reduce dollar costs significantly for encoding in the cloud. Second, today, 2D video providers use per-title *encode optimization* [40]. This approach seeks to find the best quality to encode a video, by repeatedly encoding and decoding frames at different qualities. A fast encoder/decoder like GS-NFS's can enable similar optimizations for 4DGS. Third, recent work in feed-forward 3DGS networks [131] can generate Gaussian frames within tens of milliseconds. This can generate Gaussian frames at nearly full frame rate, and GS-NFS can help encode and transmit these frames in real-time.

**Approach.** To achieve this goal, GS-NFS uses a compression approach similar to a point-cloud compression technique, G-PCC [29]. This is a more natural starting point than the 2D compression used in $V^3$ [114], since a 3DGS frame's representation resembles that of a point-cloud. The latter contains a set of points, each with one or more attributes; 3DGS is similar, except that each Gaussian has more attributes (§3.1.1). Next, we describe the G-PCC pipeline for encoding 3DGS frames; MesonGS [121] has used such an approach.

G-PCC [29]'s encoder takes a set of positions and associated attributes as input, then serializes (produces a bitstream) this set for transmission. Its decoder reconstructs the positions and attributes from the bitstream. To do this, G-PCC (a) *encodes* positions and associated attributes in such a manner that they can be decoded efficiently, and (b) compresses these encodings (either with or without information loss) to ensure bandwidth and storage efficiency.

G-PCC encodes position and attributes using qualitatively different approaches. It represents positions of Gaussians using a spatial data structure, an *octree*. Usually, 3DGS Gaussians are spatially sparse, so

not all parts of the octree are occupied. G-PCC serializes the octree such that the bitstream only contains information for occupied octree parts.

For attributes, G-PCC provides several alternatives. We use Region-Adaptive Hierarchical Transform (RAHT [90, 18]), which organizes the Gaussians into a hierarchy of levels and predicts attributes at finer levels from attributes at coarser levels. The output of RAHT is a set of coefficients, which can be *quantized* (this introduces loss) and then entropy-coded (a lossless step that removes redundancy).

G-PCC's decode pipeline inverts these operations, de-serializing Gaussians from the encoded bitstream. Unfortunately, G-PCC and MesonGS have high encode and decode times (§3.5.2). GS-NFS achieves frame-rate encode/decode by ***running the entire encoding and decoding pipeline on a GPU***. Beyond accelerating encoding, executing the encoder on a GPU is efficient when the encoder takes input from a training method that emits 3DGS frames. These frames are already in GPU memory, since 3DGS training requires a GPU, thereby avoiding memory copy costs.

**Challenges and Contributions.** Unfortunately, accelerating G-PCC's algorithms on a GPU is non-trivial. Unlike 2D video frames, where pixels exhibit regularity, 3DGS training can produce irregularly-spaced Gaussians. Mapping this to a single-instruction, multi-threaded (SIMT) GPU is difficult.

CPU-resident algorithms for octree encoding and RAHT manage this irregularity by imposing a spatial hierarchy on the data. Unfortunately, GPUs are a poor fit for computations on hierarchical data. Traversing the hierarchy requires following pointers; this *pointer chasing* is known to be highly inefficient on a GPU, since if the referenced memory is not in the cache, all warps in a GPU thread must wait for the data to be loaded (sometimes from relatively slow DRAM), resulting in poor performance. This problem is well-known for GPU-based computations on irregular data in general [96, 126], and for parallelizing point-cloud algorithms in particular [61, 132].

Against this backdrop, GS-NFS is, to our knowledge, the first post-compression encoder/decoder for 4DGS in which *all* components (Fig. 3.1) execute on a GPU and can encode and decode at full frame rate. To achieve this, the paper makes the following novel contributions:

- A GPU-based parallelization of octree encoding (§3.2).
- A GPU-based parallelization of RAHT (§3.3).
- A technique to improve the compression of 4DGS (§3.4).

Our evaluations (§3.5) on two different popular 4DGS datasets demonstrate that GS-NFS: (a) can encode an order of magnitude faster than the state-of-the-art; (b) dominates MesonGS [121], a GPCC [113] variant modified for 4DGS, and LTS [104] both in quality and compression performance, and performs comparably or better than $V^3$ [114] on large scenes; (c) can decode 4DGS at up to 25 fps on a Jetson Orin; and (d) can encode and decode point clouds and live 4DGS videos at full frame rate, a capability that, to our knowledge, has not been demonstrated in the literature.

## 3.2 GPU-Accelerated Octree Encoding/Decoding

We first describe our GPU-accelerated octree encoding algorithm, which encodes Gaussian position attributes efficiently.

### 3.2.1 Background

**Voxelization.** Gaussians can be positioned irregularly in space, and all 3-D compression algorithms first *voxelize* Gaussian positions to regularize the data. Voxelization discretizes the cuboid bounding all the Gaussians into a regular grid of $2^J \times 2^J \times 2^J$ cubes, where $J$ is a voxelization parameter which controls the *resolution* of the 3DGS frame. Then, it maps each Gaussian's 3-D position to one of the cubes, by quantizing the position coordinates to use $J$ bits. Multiple Gaussians might fall into a single voxel; GS-NFS replaces those Gaussians with a single Gaussian whose position is at the center of the voxel and other attributes are the average of the original Gaussians.[†]

[†]This may not be ideal for small $J$, but suffices for the range of $J$ we consider in this paper.

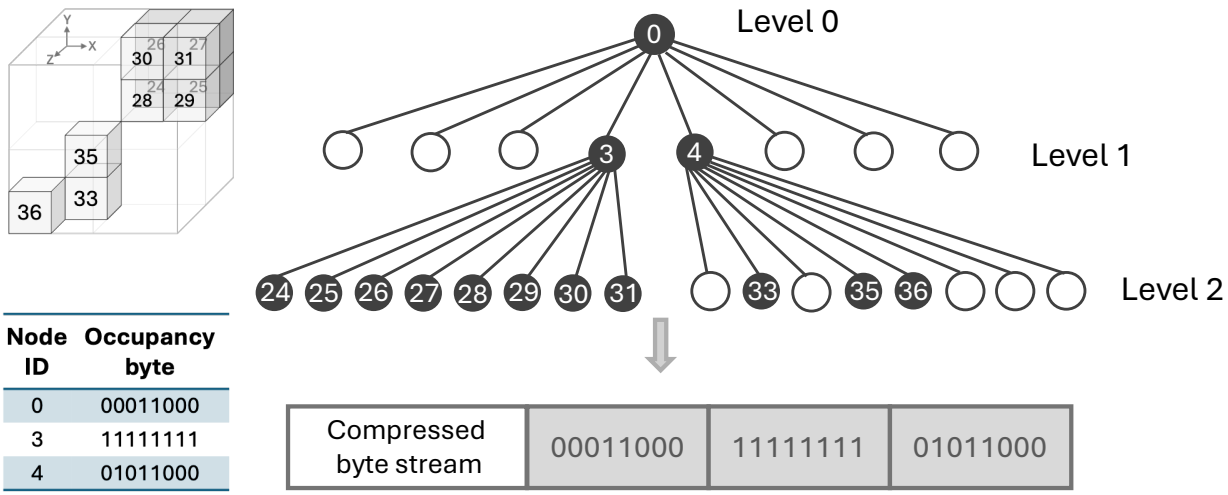



**Figure 3.2:** An example of an octree with some occupied voxels.

**The Octree Data Structure.** After voxelization, many voxels are often empty and only a small fraction of voxels are occupied by Gaussians. An octree [72] efficiently encodes the positions of occupied voxels to exploit sparsity and spatial locality. It recursively partitions the 3D space into octants (8 equally-sized child nodes), creating a tree structure where each node represents a cubic region of occupied space.

Consider the octree in Fig. 3.2. The root node represents the entire 3D space, and its 8 child nodes represent the 8 equally-sized children. The child nodes that are occupied (*i.e.*, contain at least one Gaussian) can be further subdivided into their own child nodes. The process continues until we reach a leaf node that contains a single occupied voxel.

This structure can efficiently encode occupancy. Each internal tree node uses a byte, where each bit indicates whether the corresponding child node is occupied (1) or empty (0). For example, if the fourth and fifth bits are set to 1, it means that the fourth and fifth child nodes are occupied, while the rest are empty (Fig. 3.2).

**Recursive Octree Encoding.** 3-D compression must *construct* the octree (*i.e.*, determine octants and their occupancy), then *encode* (*i.e.*, serialize) the octree into a bitstream. Both of these can be accomplished with a single sequential *level-order* traversal from the root to the leaves; a reference implementation of G-PCC [75]

| Works | Quote |
|---|---|
| Zhou [132] | "Creating an octree for point clouds directly on the GPU, however, is very difficult, mainly because of memory allocation and pointer creation." |
| GROOT [61] | "While the tree structure efficiently handles the sparsity of 3D data and subdivides only the volumes where the point exists, the irregularity requires traversing the serialized byte stream and recursively calculating the child node geometry. The exact positions that represent intermediate nodes depend on the occupancy of their ancestors, which cannot be parallelized." |

**Table 3.1:** Parallelization challenges for octree encoding reported in prior work.

uses this approach. This algorithm starts from the root, determines the occupancy of its child octants, then writes out the occupancy byte of the root. It then recursively repeats the process for each child in a fixed sequence (*i.e.*, if children are numbered $0..7$, it visits the children in that order), and stops when all occupied voxels form the leaves.

The output of this algorithm is a sequence of bytes representing occupancy at various nodes of the octree. The total number of bytes in this sequence is proportional to the number of occupied voxels $\hat{N}_v$, which is often more compact than recording the occupancy of all $2^{3J}$ voxels.

In Fig. 3.2, for example, only two children of the root node (the 4th and the 5th) contain at least one occupied voxel, so its occupancy byte is $00011000$. This forms the first byte of the output bitstream. The second byte corresponds to the root's 4th child, all of whose children are occupied, so its occupancy byte is $11111111$, and so on.

### 3.2.2 GPU-accelerated Octree Encoding

GS-NFS parallelizes octree construction and encoding on the GPU. Before describing this algorithm, we explain why this is a challenge, and how prior work has attempted to accelerate octree construction and encoding. The next subsection describes GPU-accelerated octree decoding.

**Challenges.** Recursive octree construction is inherently sequential. State-of-the-art point cloud compression algorithms [29, 22, 92] implement sequential CPU-based octree construction and encoding. Constructing the octree requires traversing the tree top-down and creating nodes and pointers. This is difficult to

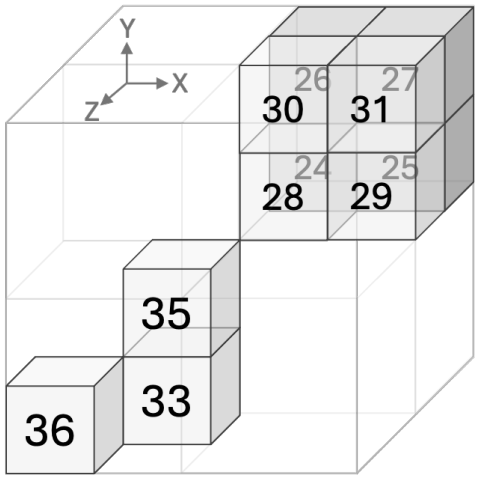
efficiently parallelize on a GPU since the memory layout of the tree can be irregular, and tree-traversal might require pointer-chasing, resulting in GPU thread stalls while waiting for memory accesses to complete [132, 54, 61]. Generating the occupancy bitstream also requires a tree traversal in level-order and pointer-chasing to determine the occupancy byte of a node depending on existing children, which can be inefficient on a GPU. Table 3.1 includes quotes from prior work [132, 61] that illustrate the difficult of parallelizing octree encoding.

**Prior Work on Parallelization.** One line of work has explored CPU parallelism for octree construction and encoding. ViVo [33], MeshReduce [45], and MetaStream [30] use coarse-grained CPU parallelism by partitioning the 3D points into separate blocks, generating the octree encoding independently for each block and concatenating these encodings. Others [54, 61] employ slightly more sophisticated hybrid CPU-GPU strategies. For example, Groot [61] sequentially constructs the octree top-down up to a certain depth $d$, then generates encodings in parallel for each occupied voxel (on a GPU) by traversing the tree bottom-up up to level $d$.

Another line of work [107, 132, 49] has achieved GPU-accelerated parallelization of octree construction. They linearize the positions in 3D to a 1D list using a space-filling curve (the Morton code, described below) while preserving spatial locality [6]. Thus, the generated octree is not explicitly represented as a tree structure, but rather as arrays of indices in the space-filling curve for each node at a certain level. In these approaches, however, *encoding is still sequential*, since that requires an in-order tree traversal. GS-NFS uses a similar space-filling curve, but parallelizes both construction and encoding on a GPU.

**Morton Codes.** The key to our GPU-accelerated octree encoder is to never build explicit tree pointers. Rather, we represent each occupied voxel by its *Morton code* [6], which is a bit-interleaving of its integer voxel coordinates. For a voxelized coordinate $\hat{v} = (x, y, z) \in \{0, \ldots, 2^J - 1\}^3$, the Morton code is a bit string $z_{J-1}y_{J-1}x_{J-1} \ldots z_0y_0x_0$ where $x_b, y_b, z_b$ are the $b$-th least-significant bits of $x, y, z$.

| Octant ID | Morton Code (z y x) |
|---|---|
| 24 | 011 000 |
| 25 | 011 001 |
| 26 | 011 010 |
| 27 | 011 011 |
| 28 | 011 100 |




| Octant ID | Morton Code (z y x) |
|---|---|
| 29 | 011 101 |
| 30 | 011 110 |
| 31 | 011 111 |
| 33 | 100 001 |
| 35 | 100 011 |
| 36 | 100100 |

**Figure 3.3:** This figure shows the Morton codes of the occupied voxels in Fig. 3.2.

For example, consider the voxelization shown in Fig. 3.3, which depicts a $4 \times 4 \times 4$ voxel grid ($J = 2$) with 11 occupied voxels. The occupied voxel with ID 33 has coordinates $(1, 0, 2)$ so its Morton code is 100001. The Morton code is obtained by first taking the most significant bits of the 3 coordinates (100) in the order from $z - x$, followed by the next most significant bits (0 for the z-coordinate value of 2, 0 for the y-coordinate value of 0, and 1 for the x-coordinate value of 1), and so on.

The Morton code has three properties crucial for GPU-acceleration of octree construction, encoding, and decoding. First, it preserves spatial locality: when we sort the voxels by their Morton code, voxels near each other in space will be near each other in the sorted order [6]. For example, if in Fig. 3.3, the voxel with ID 33 and coordinate $(1, 1, 2)$ is also occupied, it would have the Morton code 100011. This differs from the Morton code of its neighboring voxel $(1, 0, 2)$ only in the last 3 digits. This property ensures memory locality if voxels are laid out in GPU memory in Morton order, and avoids non-local memory access overheads.

Second, the Morton code for the parent of a voxel is a prefix of the voxel's Morton code. In Fig. 3.3, the parent of $(1, 0, 2)$ (ID 33) has the code 100, which can be obtained by shifting the voxel's Morton code

three bits to the right. Conversely, to obtain the Morton code of, say, the 5th child of an octree node, we simply append 101 to the node's Morton code.

Third, given the Morton code for a voxel, we can obtain its voxelized coordinates using the definition of the Morton code. In our example, given the code 100011, the first coordinate is obtained by concatenating every third bit starting from position 3, the second coordinate by doing so from position 2, and so on, resulting in the coordinate $(1, 1, 2)$.

Thus, if given the Morton code for an occupied voxel, it is an $O(1)$ operation to determine its parent. Conversely, if all octree nodes are sorted in Morton order, then a parent can in an $O(1)$ operation determine its child's Morton code, and look up its occupancy status. Finally, in $O(1)$, given a node's Morton code, we can obtain its voxel coordinates.

Before explaining how we use these properties to construct, encode and decode octrees, we describe how we parallelize voxelization (§3.2.1). This is important since voxelization precedes octree construction. We require all steps in GS-NFS to be GPU-resident to avoid data movement costs (§3.1.3).

**Voxelization.** Voxelization (a) discretizes Gaussian positions and (b) merges Gaussian attributes (by averaging them) of Gaussians at the same discretized positions. The input to this step is an $N \times C$ tensor for a 3DGS frame with $N$ Gaussians, with each Gaussian having $C$ attributes. Step (a) is simple to parallelize on a GPU by launching one thread per Gaussian, which computes the voxel coordinates (§3.2.1) of the Gaussian. This step also computes the voxel's Morton code, and creates an array sorted by the Morton code that contains a pointer to the Gaussian. We use a 64-bit Morton code to support octree depths up to $J = 21$, which is sufficient for all scenes of interest [121]. If $J \leq 21$, we set to zero the unused most-significant bits in the Morton code.

In this array, Gaussians belonging to the same voxel are contiguous; we identify these *clusters* using standard PyTorch operations. Then, we launch one GPU thread for each cluster to average the attributes

of Gaussians in that cluster. The output of this process is an $N_v \times C$ tensor, where $N_v$ is the number of occupied voxels. This is sorted by Morton order, and forms the input to the next step.

**Octree construction.** In contrast to the recursive top-down approach (§3.2.1), our construction identifies, *bottom-up*, occupied parents of occupied voxels, and occupied internal nodes up the octree (Fig. 3.2). Let $L_J$ be an array of occupied voxels, sorted by Morton order, obtained directly from the input tensor. We launch one GPU thread per occupied voxel; this thread computes the parent's Morton code and writes this to a new list $L_{J-1}$. This array contains parents of occupied voxels in Morton order, but can have duplicates, since multiple occupied voxels can have the same Morton code. We use CUDA's `unique` primitive to compact duplicates in parallel, so $L_{J-1}$ has exactly one instance of an occupied parent.

We can now repeat this process to find the next-level occupied parents (*i.e.*, parents of occupied parents) up to the root level. The output of this process is a list of Morton codes for the occupied nodes at each level, *i.e.,* $L_0, \ldots, L_J$, where $L_0$ has the root node, $L_J$ has the leaf nodes, and $L_d$ has the occupied nodes at depth $d$.

**Bitstream generation.** From these lists, the encoder must emit the octree occupancy bitstream. Each internal node contributes exactly one occupancy byte, whose $i$-th bit is set if child $i$ exists. This step requires us to compute the occupancy byte for each internal node. To compute this, a node must: (a) determine the Morton codes of its children and (b) identify which children are occupied.

Given a parent code $p_m \in L_d$, the Morton prefix of its children can be obtained by $c_m = p_m \ll 3$. The eight possible children are therefore the contiguous code range $[c_m, c_m + 7]$. To compute the occupancy byte for $p_m$, we need to check which of these eight candidate child codes exist in the child list $L_{d+1}$. This requires searching for the child codes in $L_{d+1}$.

We make 2 observations that make this search efficient. First, all children with Morton codes in the range $[c_m, c_m + 7]$ must appear contiguously in the sorted list $L_{d+1}$. Second, we can perform a binary search for the first occupied child code $\geq c_m$ in $L_{d+1}$, since that listed is sorted.

To compute the occupancy bytes for all octree nodes, we use a *top-down* approach. We compute occupancy bytes for nodes in $L_d$ before doing so for nodes in $L_{d+1}$. At level $L_d$, our octree encoder launches a custom CUDA kernel[‡] with one thread per parent node $p_m$ in $L_d$. It initializes $p_m$'s occupancy byte to 0. Then, the thread for $p_m$ performs the following steps:. (i) compute $c_m = p_m \ll 3$; (ii) binary-search $L_{d+1}$ to find the first occupied child code $\geq c_m$; (iii) scan forward over at most 8 entries until the child code exceeds $c_m + 7$; (iv) set bit for every child code $c_m$ found. The thread then writes its occupancy byte to a known location in the GPU buffer.

At the end, all the occupancy bytes are written in order, similar to Fig. 3.2. To improve compression efficiency, GS-NFS then runs a GPU-based entropy encoder (Fig. 3.1), ANS [16].

**Detailed Algorithm.** Algorithm 1 summarizes our GPU octree encoder. Starting from voxelized integer coordinates, we first compute Morton codes for all occupied leaf voxels, then radix-sort and deduplicate them to obtain the leaf set $L_J$. The upper octree levels are constructed bottom-up by repeatedly mapping child codes to their parents and removing duplicates, yielding level-wise node sets $\{L_d\}_{d=0}^{J}$.

After the hierarchy is built, we serialize a compact occupancy bitstream. The header stores the number of levels and the node count of each level. The payload is then generated in breadth-first order: for each parent in level $L_d$, we produce one occupancy byte indicating which of its eight children are present in $L_{d+1}$. As shown in Algorithm 1, this is done by locating the base child Morton code and setting the corresponding bits for all valid children in the next level.

This construction avoids explicit pointer-based octree data structures and operates directly on sorted Morton codes, which is well suited to parallel primitives such as radix sort, deduplication, and batched searches. The resulting bitstream is a level-synchronous octree representation that can be decoded with the same breadth-first traversal order.

[‡]For efficiency, when necessary, we replace PyTorch operations with custom CUDA kernels to improve encoding speed.

**Algorithm 1:** GPU Octree's Encoding from Voxelized Positions

**Input :** Voxelized integer positions $\hat{V} = \{(\hat{x}_i, \hat{y}_i, \hat{z}_i)\}_{i=1}^{\hat{N}_v}$ on GPU; octree depth $J$
**Output:** Serialized octree bitstream $B_{occ}$

// Step 1: Morton codes for leaf voxels
**for** $i \leftarrow 1$ **to** $\hat{N}_v$ **do in parallel**
  $m_i \leftarrow \texttt{Morton}(\hat{x}_i, \hat{y}_i, \hat{z}_i, J)$
**end**
$L_J \leftarrow \texttt{Unique}(\texttt{RadixSort}(\{m_i\}))$ // leaf nodes
// Step 2: Level-synchronous octree construction
**for** $d \leftarrow J-1$ **to** $0$ **do** // bottom-up parents
  $L_d \leftarrow \texttt{Unique}(\{\, \texttt{Parent}(c_m) \mid c_m \in L_{d+1} \,\})$ // $\texttt{Parent}(c_m) = c_m \gg 3$
**end**
// Step 3: Bitstream header
$num\_levels \leftarrow J + 1$
$level\_sizes \leftarrow (|L_0|, \ldots, |L_J|)$
$B_{occ} \leftarrow \texttt{Append}(B_{occ}, num\_levels, level\_sizes)$
// Step 4: Breadth-first occupancy bitstream.
**for** $d \leftarrow 0$ **to** $J-1$ **do in parallel**
  $B_d \leftarrow \texttt{OccKernel}(L_d, L_{d+1})$ $B_{occ} \leftarrow \texttt{Append}(B_{occ}, B_d)$ // contiguous bytes for level $d$
**end**
**return** $B_{occ}$
**Kernel** `OccKernel` **(**$L_d, L_{d+1}$**):**
**for** $p \in L_d$ **do in parallel**
  $c_m^b \leftarrow p \ll 3$ // base child code
  $occ \leftarrow 0$ // occupancy byte
  $c_m \leftarrow \texttt{BinarySearch}(L_{d+1}, c_m^b)$ **for** $k \leftarrow 0$ **to** $7$ **do**
    **if** $L_{d+1}[c_m + k] \le c_m^b + 7$ **then**
      $occ \leftarrow occ \mid \left(1 \ll (L_{d+1}[c_m + k] - c_m^b)\right)$
    **end**
  **end**
  Write $occ$ into $B_d$ at parent's index
**end**

### 3.2.3 GPU-Accelerated Octree Decoding

**Inputs and Output.** The input to this algorithm is a sequence of occupancy bytes in the octree. We obtain this from the output of the ANS entropy decoder. The output of the algorithm is a list of voxelized positions of occupied voxels. Each such position corresponds to a Gaussian, whose attributes we obtain as described in §3.3.

This algorithm also needs to know $J$, the depth at which the 3DGS frame was encoded, and the number of occupied nodes $n_d$ at each level $d$ in the tree. Such information is encoded in the bitstream as *metadata*.

**Prior work.** GROOT and [54] try to decode lower levels of the octree in parallel on a CPU, but most tree levels are still decoded sequentially on the CPU top-down. We know of no other work that has GPU-accelerated decoding.

**Octree decoding.** Our octree decoding algorithm relies on properties of the Morton code. We note that the first byte is the occupancy byte of the root. From this, we can determine (a) which children are occupied, and (b) the Morton code of the children (both $O(1)$ operations). We can repeat this process to obtain the Morton codes of all nodes in the tree, and eventually, the Morton codes of all occupied voxels. From these, we can obtain the voxelized positions of the occupied voxels using the definition of the Morton code.

To parallelize this, we use a top-down level-synchronous approach (similar to the encoder), where, at each level $d$, we launch one thread per octree internal node. For level $d$, we can determine from the metadata where the occupancy bytes for that level start and end. But each node at that level can have a variable number of occupied children. To determine the offset of the $k$-th occupied node, we first create an array per internal node that counts the number of occupied children (using the population count CUDA primitive), then do a CUDA prefix sum on that array to find the offset of this node's children in the occupancy bytes at level $d + 1$.

The $k$-th occupied node writes the Morton codes of its occupied children into a separate array starting at the corresponding offset. Then, the decoder launches one thread for each occupied thread at level $d + 1$, and the process repeats. After processing all levels, the decoder's output is a list of Morton codes for the leaf nodes, which correspond to the occupied voxels. The decoder launches a final CUDA kernel to decode the Morton codes back into integer voxel coordinates by reversing the bit interleaving.

**Detailed Algorithm.**

**Algorithm 2:** GPU Octree Decoding

**Input :** Compressed octree buffer $B$ on GPU; octree depth $J$
**Output:** Voxelized positions $\hat{V}$ on GPU; leaf Morton codes $L_J$ on GPU

// Step 1: Parse header.
$(num\_levels,\ level\_sizes,\ ptr) \leftarrow$ `ReadHeader`$(B)$
// Step 2: Initialize root list (implicit root Morton code).
$L_0 \leftarrow [0]$
// Step 3: Level-synchronous decoding.
**for** $d \leftarrow 0$ **to** $J-1$ **do** // top-down levels
  $M_d \leftarrow |L_d|$
  $Occ_d \leftarrow$ `ReadOccBytes`$(ptr, M_d)$ // read $M_d$ bytes from $B$ into a GPU array
  $ptr \leftarrow ptr + M_d$
  // Count children per parent.
  **for** $p \leftarrow 1$ **to** $M_d$ **do in parallel**
    $cnt[p] \leftarrow$ `Popcount`$(Occ_d[p])$
  **end**
  // Find total children at $d+1$.
  $off \leftarrow$ `ExclusiveScan`$(cnt)$ $T \leftarrow off[M_d] + cnt[M_d]$
  Allocate $L_{d+1}$ of length $T$
  // Expand children.
  `ExpandChildrenKernel` $(L_d, Occ_d, off, L_{d+1})$ with $M_d$ threads
**end**
// Step 6: Get voxel coordinates at leaf.
$\hat{N}_v \leftarrow |L_J|$
$shift \leftarrow J_v - J$
Allocate $\hat{x}[1{:}\hat{N}_v], \hat{y}[1{:}\hat{N}_v], \hat{z}[1{:}\hat{N}_v]$
Launch `ReconstructPointsKernel` $(L_J, shift, \hat{x}, \hat{y}, \hat{z})$ with $\hat{N}_v$ threads
$\hat{V} \leftarrow$ `InterleaveXYZ`$(\hat{x}, \hat{y}, \hat{z})$ // Reverse Morton Code.
**return** $(\hat{V}, L_J)$

Algorithm 2 summarizes our GPU octree decoder. The GPU decoder reconstructs voxelized geometry by replaying the serialized occupancy stream in a level-synchronous manner, without building any explicit tree pointers. Starting from the implicit root Morton code ($L_0 = \{0\}$), for each depth $d$ it reads the $|L_d|$ occupancy bytes, counts the number of occupied children per parent (population count), and performs an exclusive prefix-sum to compute disjoint write offsets for the next-level array. It then launches `expand_children_kernel` (Alg. 3), which generates each occupied child Morton code in $O(1)$ via ($parent \ll$

**Algorithm 3:** `expand_children_kernel`: Expand occupied children and generate Morton codes

**Input :** Parent Morton codes $L_d[1{:}M_d]$; occupancy bytes $Occ_d[1{:}M_d]$; write offsets $off[1{:}M_d]$
**Output:** Child Morton codes $L_{d+1}[1{:}T]$

**for** $p \leftarrow 1$ **to** $M_d$ **do in parallel**
  $parent \leftarrow L_d[p]$
  $occ \leftarrow Occ_d[p]$
  $base \leftarrow parent \ll 3$ `// base child Morton code`
  $w \leftarrow off[p]$ `// exclusive-scan start offset for this parent`
  **for** $i \leftarrow 0$ **to** 7 **do**
    **if** $occ \wedge (1 \ll i)$ **then**
      $L_{d+1}[w] \leftarrow base + i$
      $w \leftarrow w + 1$
    **end**
  **end**
**end**

$3) + i$ and writes all children contiguously into $L_{d+1}$, avoiding atomics despite variable fan-out. Repeating this from $d = 0$ to $J-1$ yields the leaf Morton codes $L_J$, which are finally decoded into integer voxel coordinates (and optionally interleaved) in a parallel post-pass. Overall, decoding preserves the same breadth-first structure as the encoder while turning the key sequential dependency (unknown next-level size) into a standard GPU pattern: count $\rightarrow$ prefix-sum $\rightarrow$ parallel scatter.

### 3.3 GPU-Accelerated Attribute Encoding/Decoding

In this section, we describe how we GPU-accelerate the encoding and decoding of Gaussian attributes, such as opacity, scale, rotation, and spherical harmonics.

#### 3.3.1 Background: RAHT

Region-Adaptive Hierarchical Transform (RAHT) is an algorithm for encoding and decoding the attributes of sparse, spatially non-uniform data (such as point clouds [18] and 3DGS frames). The algorithm is complex to explain in its entirety, so we use an example to explain the algorithm; details can be found in [18, 90]. To further simplify the description, we describe how it encodes a *single attribute*, say opacity.

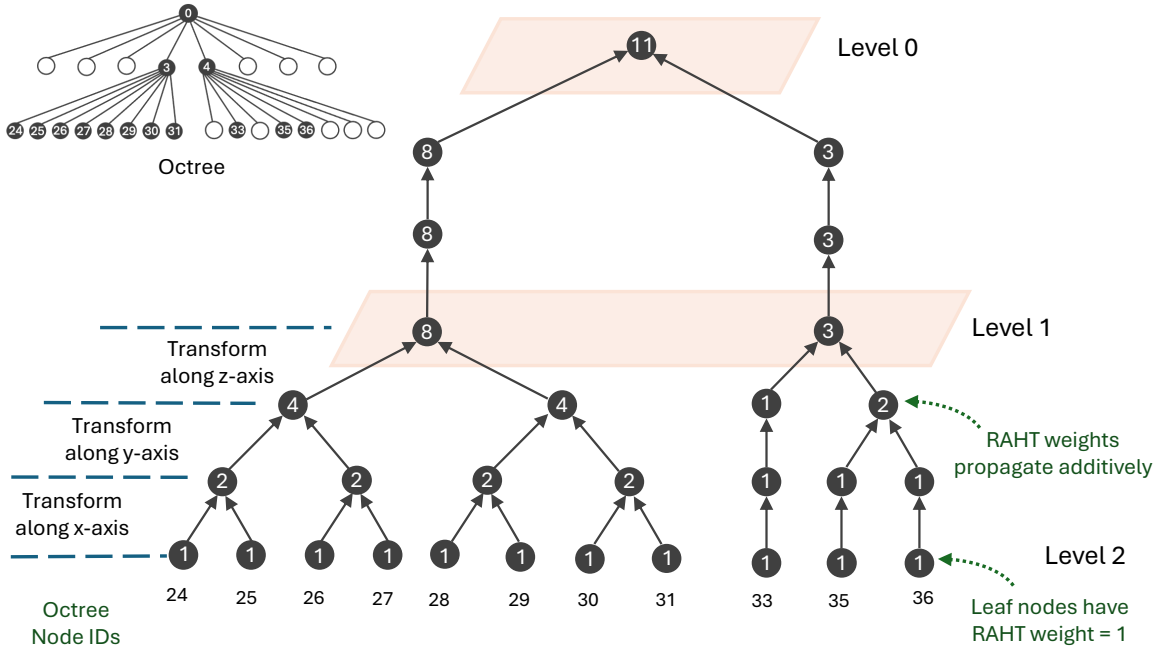



**Figure 3.4:** Example to illustrate the RAHT algorithm.

---

RAHT exploits the octree structure derived from voxel positions (Fig. 3.4). It operates on a binary tree derived from the octree; in this binary tree, each octree level is represented by 3 levels in the binary tree, each of which corresponds to one of the directional axes. The leaves of the tree correspond to voxels, and in Fig. 3.4 we only show branches in the binary tree leading to occupied voxels, for simplicity.

If a node has a sibling (*e.g.*, nodes **35** and **36** in Fig. 3.4), RAHT decomposes the attribute into:

1. **Low-frequency Coefficient:** A weighted average representing coarse signal information.
2. **High-frequency Coefficient:** A weighted difference representing the signal details.

More precisely, if two siblings $l$ and $r$ have attribute values $a_l$ and $a_r$, then computing these coefficients requires a $2 \times 2$ matrix-vector multiplication:

$$\begin{bmatrix} a^0 \\ a^1 \end{bmatrix} = \frac{1}{\sqrt{w_l + w_r}} \begin{bmatrix} \sqrt{w_l} & \sqrt{w_r} \\ -\sqrt{w_r} & \sqrt{w_l} \end{bmatrix} \begin{bmatrix} a_l \\ a_r \end{bmatrix}, \tag{3.1}$$

In this equation, $a^0$ represents the low-frequency (coarse) coefficient, $a^1$ the high-frequency coefficient, and $w_l$ and $w_r$ represents *weights* associated with $a_l$ and $a_r$ respectively. The weight at any node is the number of occupied voxels in the octant below that node; attributes at leaf nodes have unit weights. Fig. 3.4 shows node weights next to the node. RAHT propagates the low-frequency coefficient to the parent, while storing the high-frequency coefficients at the right sibling. (If a node does not have a sibling such as node **33**, it passes its attribute value up to its parent). RAHT then repeats this procedure level by level, bottom-up, on this tree.

At the end of this computation, the octree root stores the low-frequency component, and internal nodes and leaves store high-frequency coefficients.[§] Smoothly varying spatial signals (*e.g.*, point clouds can have smoothly varying color, just like images) have near-zero high-frequency coefficients, especially close to the leaves, which can be quantized and entropy-coded, resulting in significant compression. In GS-NFS, we use RAHT to encode all 3DGS attributes (§3.1.1) other than positions. RAHT decoding inverts this operation; for brevity, we discuss only the RAHT encoder and leave the decoder (inverse-RAHT) to **??**.

### 3.3.2 Background: A Re-formulated RAHT Algorithm

Pavez *et al.* [90] describe a re-formulation (henceforth P-RAHT) of the original RAHT algorithm [18] that exploits properties of the Morton code. P-RAHT takes as input occupied voxels and their associated Morton codes. It pre-computes, in a step called the *RAHT* prelude, the constructs needed for the hierarchical RAHT computation, the sibling relationships and the node weights, since these are a function of voxel occupancy alone, not of the attribute. Then, it re-uses these when encoding each attribute.

**The RAHT prelude.** This step computes three lists at every level $\ell \in \{1, \ldots, 3J\}$: (i) an index list $I_\ell$ indicating those nodes in the binary tree at level $\ell$ that are either left siblings, or singletons (those with no occupied siblings); (the binary tree in Fig. 3.4 shows only the nodes in these lists) (ii) a weight list $W_\ell$ for

[§]This computation can be intuitively thought of as an extension of the Haar wavelet transform to a sparse 3D grid [18].

those nodes (indicated by numbers inside nodes in Fig. 3.4, and (iii) a flag list $F_\ell$ indicating which nodes are left siblings (not shown in the figure for brevity).

P-RAHT computes these lists bottom-up. Consider a node $n$ at level $\ell$. Its weight is the sum of the weights of its children at level $\ell + 1$. If either of its children is in $I_{\ell+1}$, node $n$ must be in $I_\ell$. Finally, if $n$'s sibling at level $\ell$ is also in $I_\ell$, it is the left sibling if it has a lower Morton code. For example, the rightmost node at the second level from the bottom has a weight of 2, because its children are both occupied. For that reason as well, it belongs in the $I_\ell$ list for that level. But, it does not belong in the corresponding $F_\ell$ list, since it is *not* the left sibling at that level.

**Computing RAHT Coefficients.** Using these lists, P-RAHT computes the RAHT coefficients using another bottom-up pass for each attribute. If a node $n$ at level $\ell$ is a left sibling (*i.e.*, its flag is set in $F_\ell$), then it computes the low-frequency and high-frequency coefficients using Eq. (3.1). Nodes **24** and **35** in Fig. 3.4 are examples of left siblings. It then passes the low-frequency coefficient to its parent and stores the high-frequency attribute in its right sibling. For this, it uses the pre-computed weights in $W_\ell$. If $n$ is a singleton, such as node **33** in Fig. 3.4, it simply passes its attribute to its parent.

### 3.3.3 GPU-Accelerated RAHT

**Inputs and Outputs.** GS-NFS's GPU-accelerated RAHT takes as input occupied voxels, their Morton codes, and the attributes of the merged Gaussian in the voxel. It outputs RAHT coefficients for all attributes.

**Key Ideas.** Our GPU-accelerated RAHT relies on three observations. First, we observe that the basic RAHT algorithm is similar to the octree construction, encoding and decoding algorithms (§3.2). RAHT proceeds bottom-up on a tree, performing one or more computations at some but not all nodes (*e.g.*, only siblings), at each level in the tree. The same is true for octree construction (§3.2.2), for example, which performs computations only occupied nodes at each level.

This suggests that we can compute RAHT coefficients using the following design elements from those algorithms. (a) Those algorithms use properties of Morton codes to mitigate compute stalls due to pointer

chasing. (b) They proceed sequentially level-by-level bottom-up or top-down, but within a level, they launch one GPU thread for every node in the tree involved in the computation. (c) The set of nodes at a level involved in the computation is of variable length, so these algorithms employ PyTorch [89] operations to convert fixed length vectors to variable length ones. For example, the octree construction algorithm first creates a list of all occupied parents, then uses the `unique` operation to identify exactly the set of nodes at the next higher level involved in determining occupied nodes.

Our second observation is that P-RAHT is an ideal starting point for parallelization. In particular, the pre-computed lists from the RAHT prelude greatly simplify applying the design elements described above.

Our third observation is that GPUs allow us to trivially *parallelize RAHT coefficient computation across attributes*. We can simply vectorize the attributes, and compute the coefficients in one bottom-up pass.

**Computing RAHT coefficients on the GPU.** GS-NFS computes RAHT coefficients bottom-up, exactly like P-RAHT. Unlike P-RAHT, at each level, it launches multiple GPU threads. Specifically, at level $\ell$, it launches a GPU kernel with one thread per entry in the list $I_\ell$. For example, at the lowest level in Fig. 3.4, GS-NFS will launch **11** threads, and at the next level up, **7**. Only threads whose flag $F_\ell$ is valid execute the $2 \times 2$ transform; each such thread reads indices of a sibling pair (current and next element in $I_\ell$), looks up their weights in $W_\ell$, and then applies Eq. 3.1 to the attributes.

**Parallelizing the RAHT Prelude.** At each step $\ell$, GS-NFS maintains the current active list $I_\ell$ as a flat GPU tensor of indices. We launch one GPU thread per element of $I_\ell$ to compute $W_\ell$ and $F_\ell$. Each thread compares the Morton code of its current element with the next element (adjacent in Morton order) to decide whether they form a sibling pair.

After $F_\ell$ is computed, GS-NFS obtains the next active list $I_{\ell+1}$ by removing right siblings. It implements this efficiently using boolean shift and masked select of tensors which are available in PyTorch (Alg. 5). The output of this is a compacted list containing exactly the set of nodes at $\ell + 1$ that should run the RAHT coefficient computation at that level.

**Algorithm 4:** RAHT prelude (non-parallel)

**Input :** Morton codes $M[0{:}N_v{-}1]$, Num voxels $N_v$, octree depth $J$
**Output:** $[(I_\ell, W_\ell, F_\ell)]_{\ell=1}^{3J}$

**for** $\ell \leftarrow 1$ **to** $3J$ **do**
  **if** $\ell = 1$ **then**
    $I_1 \leftarrow (0{:}N_v{-}1)^T$ `// active indices at level 1`
  **else**
    $I_\ell \leftarrow I_{\ell-1}\big(\neg[0; F_{\ell-1}]\big)$ `// keep left siblings + singletons`
  **end**
  $M_\ell \leftarrow M[I_\ell]$ `// Morton codes at level` $\ell$
  $W_\ell \leftarrow [I_\ell(2{:}\text{end});\ N_v] - I_\ell$ `// run-length weights (sentinel` $N_v$`)`
  $D \leftarrow M_\ell(1{:}\text{end}{-}1)\ \oplus\ M_\ell(2{:}\text{end})$ `// path diffs`
  $F_\ell \leftarrow \big(D\ \wedge\ (2^{3J} - 2^\ell)\big) = 0$ `// left-sibling flags`
**end**
**return** $[(I_\ell, W_\ell, F_\ell)]_{\ell=1}^{3J}$

Especially for accelerating RAHT on a mobile GPU, we have found it essential to implement some of these steps using CUDA (§3.5).

**Detailed Algorithm.** We implement RAHT as a two-stage GPU pipeline: a *prelude* that builds the merge schedule from Morton-sorted occupied voxels, and a *transform* stage that applies the weighted orthonormal rotations using that schedule. This separation isolates the octree-dependent control flow from the per-attribute transform, allowing the latter to be executed with simple parallel kernels.

Algorithm 4 gives the reference non-parallel prelude. For each of the $3J$ binary merge steps induced by a depth-$J$ octree, it constructs the active index list $I_\ell$, the corresponding run-length weights $W_\ell$, and the sibling flags $F_\ell$. Here, $W_\ell$ records the number of original voxels represented by each active entry, and $F_\ell$ marks whether an entry is the left element of a valid sibling pair. The sibling relation is tested directly from Morton codes using XOR and a level-dependent mask, without explicitly materializing the octree.

Algorithm 5 shows the GPU version of this schedule construction. At each level, one thread computes the weight and sibling flag for one active entry. The next active list is then obtained by removing right siblings while keeping left siblings and singletons. This produces the schedule $[(I_\ell, W_\ell, F_\ell)]_{\ell=1}^{3J}$ used by the transform.

**Algorithm 5:** RAHT Prelude on GPU

**Input :** Morton codes $M[0{:}N_v{-}1]$, Num voxels $N_v$, octree depth $J$
**Output:** $[(I_\ell, W_\ell, F_\ell)]_{\ell=1}^{3J}$

$I_1 \leftarrow (0{:}N_v{-}1)^T$
**for** $\ell \leftarrow 1$ **to** $3J$ **do**
  $\mu_\ell \leftarrow (2^{3J} - 2^\ell)$ `// mask for sibling test`
  **for** $k \leftarrow 1$ **to** $M_\ell$ **do in parallel** `// 1 GPU thread per` $k$
    $curr \leftarrow I_\ell(k)$
    $next \leftarrow \begin{cases} end & \text{if } k = M_\ell \\ I_\ell(k{+}1) & \text{otherwise} \end{cases}$
    $W_\ell(k) \leftarrow next - curr$
    **if** $k = M_\ell$ **then**
      $F_\ell(k) \leftarrow 0$
    **else**
      $F_\ell(k) \leftarrow ((M(curr) \oplus M(next)) \wedge \mu_\ell) = 0$
    **end**
  **end**
  $P_\ell \leftarrow$ `torch.shift`$(F_\ell)$ `//` $[0; F_\ell(0{:}\text{end}{-}1)]$ `marks right siblings`
  $I_{\ell+1} \leftarrow$ `torch.masked_select`$(I_\ell, \neg P_\ell)$
  **if** $|I_{\ell+1}| = 1$ **then**
    **break**
  **end**
**end**
**return** $[(I_\ell, W_\ell, F_\ell)]_{\ell=1}^{3J}$

Algorithm 6 applies the forward or inverse RAHT on the GPU. In the forward pass, levels are processed bottom-up; in the inverse pass, they are processed in reverse order. For each sibling pair, the transform uses the standard RAHT weights

$$a = \sqrt{\frac{w_0}{w_0 + w_1}}, \qquad b = \sqrt{\frac{w_1}{w_0 + w_1}},$$

to perform the corresponding orthonormal rotation independently for each attribute channel. Since pairs within a level are disjoint, the computation can be parallelized across active entries.

**Algorithm 6:** RAHT Transform (Forward / Inverse) on GPU

**Input :** Attributes $A \in \mathbb{R}^{N_v \times C}$ (GPU), prelude schedule $[(I_\ell, W_\ell, F_\ell)]_{\ell=1}^{3J}$, `inverse` flag
**Output:** Coefficients $Y \in \mathbb{R}^{N_v \times C}$ (GPU)

$Y \leftarrow A$ `// clone for in-place updates`
**if** *inverse = 0* **then**
  **for** $\ell \leftarrow 1$ **to** $3J-1$ **do** `// bottom-up`
    $M_\ell \leftarrow |I_\ell|$
    Launch `raht_forward_level_kernel` with $M_\ell$ threads
      **for** $k \leftarrow 0$ **to** $M_\ell - 2$ **do in parallel**
        **if** $F_\ell[k]$ **then**
          $i_0 \leftarrow I_\ell[k]$;
          $i_1 \leftarrow I_\ell[k+1]$;
          $w_0 \leftarrow W_\ell[k]$;
          $w_1 \leftarrow W_\ell[k+1]$;
          $a \leftarrow \sqrt{w_0/(w_0+w_1)}$;
          $b \leftarrow \sqrt{w_1/(w_0+w_1)}$;
          **for** $c \leftarrow 0$ **to** $C-1$ **do**
            $y_0 \leftarrow Y[i_0, c]$;
            $y_1 \leftarrow Y[i_1, c]$;
            $Y[i_0, c] \leftarrow a \cdot y_0 + b \cdot y_1$
            $Y[i_1, c] \leftarrow -b \cdot y_0 + a \cdot y_1$
          **end**
        **end**
      **end**
  **end**
**end**
**for** $\ell \leftarrow 3J-1$ **to** $1$ **do** `// top-down`
  $M_\ell \leftarrow |I_\ell|$
  Launch `raht_inverse_level_kernel` with $M_\ell$ threads
    **for** $k \leftarrow 0$ **to** $M_\ell - 2$ **do in parallel**
      **if** $F_\ell[k]$ **then**
        $i_0 \leftarrow I_\ell[k]$;
        $i_1 \leftarrow I_\ell[k+1]$;
        $w_0 \leftarrow W_\ell[k]$;
        $w_1 \leftarrow W_\ell[k+1]$;
        $a \leftarrow \sqrt{w_0/(w_0+w_1)}$;
        $b \leftarrow \sqrt{w_1/(w_0+w_1)}$;
        **for** $c \leftarrow 0$ **to** $C-1$ **do**
          $y_0 \leftarrow Y[i_0, c]$;
          $y_1 \leftarrow Y[i_1, c]$;
          $Y[i_0, c] \leftarrow a \cdot y_0 - b \cdot y_1$
          $Y[i_1, c] \leftarrow b \cdot y_0 + a \cdot y_1$
        **end**
      **end**
    **end**
**end**
**return** $Y$

### 3.3.4 Quantization and Entropy Coding

After obtaining the RAHT coefficients for each attribute, GS-NFS *quantizes* these coefficients. Quantization trades off visual quality by reducing the number of bits required to represent a coefficient. We quantize coefficients of different 3DGS attributes to different levels, similar to [121, 113]. GS-NFS uses dead-zone quantization [102], which is simple to implement on a GPU, using one thread per coefficient, and vectorizing the attributes.

GS-NFS uses an entropy coder to efficiently serialize quantized coefficients into a bitstream. We use the Run-Length Golomb-Rice (RLGR) [71] entropy coder, which uses run-length coding for consecutive zeros, and Golomb-Rice coding for non-zero magnitudes using an adaptive parameter. RLGR is inherently sequential, but GS-NFS parallelizes this by dividing the vector of coefficients, and entropy coding *blocks* (*e.g.*, 2K or 4K successive coefficients) using one GPU thread per block. This ensures fast entropy coding at the expense of a slight loss in compression efficiency.

We have omitted a discussion of de-quantization and entropy decoding for brevity; GS-NFS also parallelizes this. Thus, in GS-NFS, *all steps* in octree and attribute encoding are GPU-accelerated.

## 3.4 Improving 4DGS Compression

In many 4DGS videos, especially those that generate SH coefficients of degree 3, SH coefficients dominate the size of each 3DGS frame. Some post-training compression techniques have realized this, and have developed specialized techniques to compress SH coefficients. For example, MesonGS [121] devises a vector quantization codebook for each frame, in order to compress SH coefficients. This is extremely compute-intensive, so GS-NFS uses a different approach.

In 3DGS, each SH function has 3 RGB color channels. GS-NFS exploits the fact that RGB colors can be correlated [11]. It first transforms RGB to YUV for each SH function. In YUV, since Luma (Y) has more energy and it is more perceptually important than Chroma (UV), this transformation improves compression.

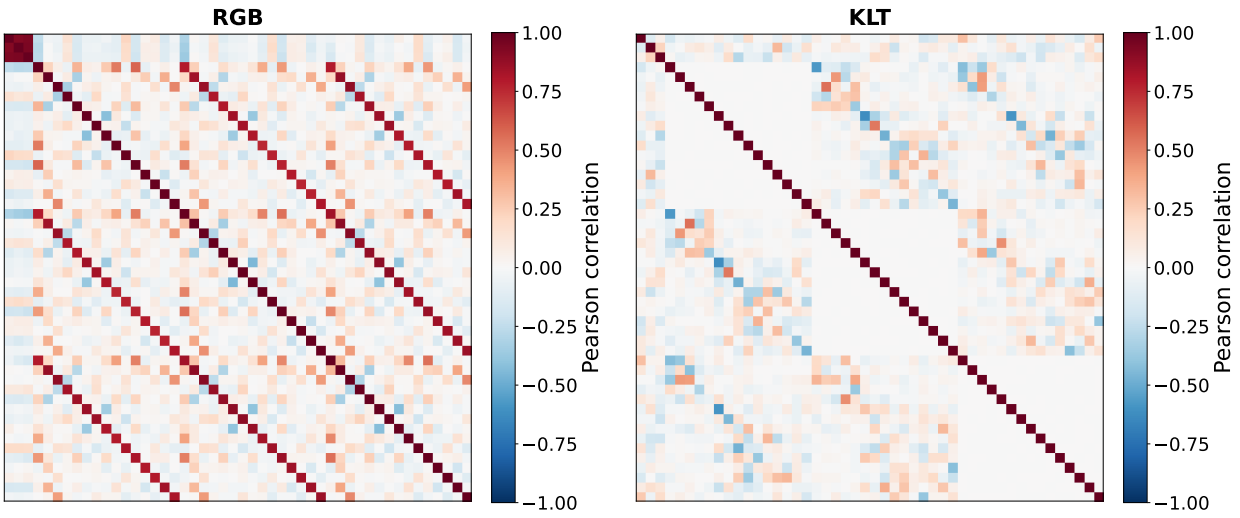



**Figure 3.5:** Heatmaps that plot Pearson correlation across SHs in RGB domain (left) and after YUV conversion and KLT (right).

---

GS-NFS applies this RGB to YUV transformation, a simple matrix multiplication, which is fast on a GPU, to all SH coefficients (both DC and AC).

While RGB to YUV conversion is common in image and video compression, because 3DGS represents color with high dimensional SH coefficients, there are additional opportunities to remove redundancy. After YUV conversion, GS-NFS applies a decorrelating transform, Karhunen-Loève transform (KLT [94]), *only to the AC coefficients* (*i.e.*, to SH coefficients of degree greater than 0). More specifically, we apply three different KLTs, one to each channel in YUV color space. The KLT effectively concentrates signal energy on the first few coefficients. Fig. 3.5 illustrates this. On the left is a heatmap of the Pearson correlation coefficients for the original RGB SH coefficients. On the right is the corresponding heatmap after applying RGB to YUV conversion followed by KLT; notice how the figure on the right has significant white-space (white corresponds to zero correlation).

By decorrelating color across AC SH coefficients, KLT reduces redundancy, concentrates energy, and thereby improves compression. We apply KLT *before* RAHT (Fig. 3.1). KLT requires a matrix multiplication, which we implement on the GPU with minimal overhead.

## 3.5 Evaluation

In this section, we compare GS-NFS against the state of the art, using two popular 4DGS data sets.

### 3.5.1 Methodology

**Implementation.** We have implemented GS-NFS in Python. It uses custom CUDA kernels for the octree encoder/decoder (§3.2), RAHT (§3.3), and RLGR (§3.3.4). It compresses position using the ANS entropy coder from NVIDIA's nvCOMP library [16] and implements KLT decorrelation and quantization in PyTorch [89]; these run on the GPU. Most of the data and intermediate tensors use PyTorch.

**Hardware.** Most experiments run on a PC equipped with an NVIDIA RTX 4500 Ada Generation GPU (24 GB VRAM), 128 GB system RAM, and Intel(R) Xeon(R) w7-2475X with 20 physical cores (40 threads with hyperthreading). Mobile decoding runs on an NVIDIA Jetson.

**Baselines.** We compare GS-NFS against four post-training compression baselines:

***V³-2D [114]*** maps Gaussian attributes to 2-D image stacks, one per attribute channel, and compresses the resulting images with a H.264 codec using `x264` from FFmpeg [24]. It compresses each channel of attributes into a single MP4 stream, where frames for that attribute form a group of pictures (GOP), and a single quantization parameter (QP) controls the quality-size tradeoff. It splits the position of Gaussians into MSB and LSB, where QP for MSB is 0 (lossless) and QP for LSB is the QP passed to the encoder (default: 25). V³-2D caps QPs for rotation and scale attributes to 22, and quantizes all other attributes (position LSB, color, opacity) with the same QP (default: 25). It compresses each attribute channel using one call of the H.264 encoder, resulting in 62 MP4 files per GOP (default: 20).

***MesonGS [121]*** is a partly GPU-accelerated codec that uses an octree for geometry, RAHT and block quantization for attributes, vector quantization for higher-degree SH coefficients, and LZ77 for entropy coding. We use the authors' default parameters.

***G-PCC [29]*** Wang *et al.* [113] use G-PCC to compress static 3DGS, and adapt octree depth for voxelization to preserve quality. This requires fine-tuning of Gaussians after voxelization to reach high quality. To obtain a baseline that directly extends G-PCC to 4DGS videos, we implement a simplified version of [113] (without adaptive voxelization and fine-tuning). The approach uses RAHT for color and opacity (lossy, depending on QP) and hierarchical neighborhood prediction for rotation and scale (lossless).

***LTS-Draco [104, 106]*** uses modified Draco to independently compress each 3DGS frame on the CPU. The number of quantization bits per attribute is configurable (default: 16 bits for all attributes, compression level 10).

In each case, we use the implementations provided by the authors. Moreover, these baselines use qualitatively different compression techniques for 4DGS: 2D codecs [114], Draco [104], and G-PCC [121, 113].

**Datasets.** We evaluate on two widely-used 4DGS video datasets spanning diverse content types:

***HiFi4G (7 sequences)*** contains person-centric captures of actors performing various actions, with 200 frames per sequence. We use 3DGS models trained using $V^3$ [114] with SH coefficient degrees $0..3$ ($\sim$120K–300K Gaussians per frame).

***Neural 3D Video (N3DV) (6 sequences)*** contains full-scene captures of indoor table-top activities with day and night lighting, each with 300 frames. We use models trained with QUEEN [28] (without its in-training compression) with SH degrees $0..2$ ($\sim$150K–400K Gaussians per frame).

**Metrics.** For each approach, we report measures of: (1) *visual quality* based on PSNR computed on rendered test views of the decompressed frames against the uncompressed ground truth (we use the test views used by $V^3$ and QUEEN); (2) *per-frame encode* and *decode latency* in milliseconds (ms); and (3) *compression efficiency* in terms of 3DGS frame size after compression. We describe the precise metrics below.

| Pipeline | N3DV (QUEEN) | | HiFi4G ($V^3$) | |
|---|---|---|---|---|
| | Enc. (ms) | Dec. (ms) | Enc. (ms) | Dec. (ms) |
| **GS-NFS** | **23** | **21** | **18** | **14** |
| $V^3$-2D | 370 | 290 | 1098 | 341 |
| LTS-Draco | 400 | 158 | 156 | 91 |
| G-PCC | 8804 | 6928 | 6101 | 3650 |
| MesonGS | 28996 | 1096 | 144953 | 535 |

**Table 3.2:** Per-frame encode and decode latency (ms).

### 3.5.2 Baseline Comparison

**Methodology.** This section compares GS-NFS against all baselines at a single operating point. For competing approaches, we select the default parameters for compression, such as QP, octree depth, bit-depth, *etc.*, that we either found in the corresponding paper or source code. For GS-NFS, we use parameters that give us the same quality (PSNR) on the first frame of a sequence as the default setting for each approach. For instance, if, on the first frame of sequence $S$, $V^3$-2D has a PSNR of $p$, we find a parameter setting for GS-NFS (using the technique described in §3.5.3) whose PSNR for the first frame on $S$ is close to $p$. This results in a pair-wise comparison between GS-NFS and each baseline.

We use these settings to encode, decode, and estimate PSNR and compression sizes for every 10th frame[¶] of every sequence in the two datasets listed above. We do this primarily because some of the baselines [121, 113] have exceedingly high encode/decode latencies, rendering an exhaustive evaluation intractable. Then, we compute, for each sequence: (a) the average per-frame encode latency, (b) the average per-frame decode latency, (c) the average per-frame PSNR difference between GS-NFS and the baseline (the $\Delta$PSNR), (d) average per-frame size ratio between GS-NFS and the baseline (the *relative compression ratio*, or RCR).

[¶]For $V^3$-2D, we compute the size of a frame as the average size of the GOP that it belongs to.

| Sequence | V$^3$-2D | | MesonGS | | LTS-Draco | | G-PCC | | GS-NFS (Ours) | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Enc | Dec | Enc | Dec | Enc | Dec | Enc | Dec | Enc | Dec |
| Actor1 | 443.1 | 312.5 | 144950.2 | 487.4 | 136.8 | 84.4 | 5063.9 | 2986.5 | 17.4 | 12.8 |
| Actor2 | 480.9 | 362.8 | 144363.5 | 368.8 | 103.5 | 60.8 | 3679.6 | 2205.4 | 15.6 | 12.0 |
| Actor3 | 542.5 | 331.9 | 145057.5 | 529.3 | 156.3 | 95.1 | 6027.0 | 3474.0 | 17.6 | 13.2 |
| Actor4 | 554.1 | 358.9 | 145351.6 | 622.6 | 182.8 | 104.6 | 8070.4 | 4858.2 | 19.1 | 15.0 |
| Actor5 | 435.7 | 330.0 | 145003.4 | 559.3 | 158.2 | 91.9 | 5992.8 | 3693.7 | 17.8 | 13.4 |
| Actor6 | 483.5 | 367.4 | 145344.3 | 674.9 | 201.9 | 113.1 | 8579.4 | 5163.9 | 19.6 | 15.7 |
| Actor7 | 421.3 | 323.4 | 144602.4 | 504.7 | 151.8 | 88.2 | 5294.2 | 3174.0 | 17.7 | 13.2 |
| Coffee Martini | 473.6 | 352.0 | 30228.0 | 1415.6 | 1059.4 | 215.8 | 12540.2 | 10090.6 | 24.9 | 23.9 |
| Cook Spinach | 316.8 | 262.8 | 28719.5 | 1012.1 | 229.7 | 127.2 | 7467.1 | 5866.7 | 21.7 | 20.1 |
| Cut Roasted Beef | 313.3 | 261.0 | 28781.4 | 978.2 | 229.4 | 128.5 | 8162.4 | 6438.3 | 21.5 | 19.6 |
| Flame Salmon | 488.0 | 340.3 | 29692.5 | 1335.9 | 413.8 | 214.0 | 11327.0 | 9236.8 | 27.7 | 23.4 |
| Flame Steak | 316.9 | 262.3 | 28877.8 | 1080.6 | 222.4 | 129.3 | 6364.7 | 4548.4 | 22.2 | 20.3 |
| Sear Steak | 312.8 | 263.2 | 28826.4 | 1061.6 | 247.9 | 133.3 | 6966.3 | 5391.3 | 22.2 | 20.5 |

**Table 3.3:** Encode and decode latency (ms) for video sequences.

#### 3.5.2.1 Encode-Decode Latency

Table 3.2 summarizes per-frame encode and decode times aggregated across all sequences within each dataset. We do this because, within a dataset (*e.g.*, HiFi4G), there is little variability in encode and decode times across sequences, since all sequences are person-centric. In Table 3.3, we report the actual encoding and decoding latencies for all sequences in the HiFi4G and N3DV datasets.

GS-NFS achieves $\sim$23 ms encode and $\sim$21 ms decode on N3DV, and $\sim$18 ms encode and $\sim$14 ms decode on HiFi4G—well under the 33 ms budget for 30 frames-per-second (fps).

LTS-Draco, the next fastest baseline, is **9–17$\times$ slower than GS-NFS for encoding** and **7–8$\times$ slower for decoding**. Draco encoding and decoding could potentially be made faster using coarse-grained CPU parallelism [45, 33, 30], but this trades off compressibility.

Though 2D video codecs are fast, V$^3$-2D's several hundred milliseconds to encode decode, due to the overhead of calling video encoder sequentially for each channel[‖]; It is **16–61$\times$ slower than GS-NFS for encoding** and **14–24$\times$ slower for decoding**. V$^3$-2D could offload H.264 encoding to an NVIDIA GPU

[‖]V$^3$-2D encodes and decode the full scene N3DV dataset faster than the HiFi4G person-centric dataset, because, on the former dataset, QUEEN does not produce degree-3 SHs.

| Sequence | V$^3$-2D | | MesonGS | | LTS-Draco | | G-PCC | |
|---|---|---|---|---|---|---|---|---|
| | **ΔPSNR** (dB) | **RCR** | **ΔPSNR** (dB) | **RCR** | **ΔPSNR** (dB) | **RCR** | **ΔPSNR** (dB) | **RCR** |
| Actor1 | 0.02 | 0.52 | -0.02 | 2.00 | -0.04 | 3.01 | -0.03 | 4.91 |
| Actor2 | 0.03 | 0.31 | -0.03 | 2.11 | -0.04 | 3.05 | -0.01 | 5.08 |
| Actor3 | -0.01 | 0.64 | 0.01 | 1.74 | -0.02 | 3.18 | 0.05 | 4.52 |
| Actor4 | -0.05 | 0.56 | -0.02 | 1.77 | -0.07 | 2.84 | 0.01 | 4.98 |
| Actor5 | 0.07 | 0.53 | 0.05 | 1.60 | -0.07 | 2.91 | 0.01 | 5.06 |
| Actor6 | -0.07 | 0.81 | -0.18 | 2.07 | -0.09 | 2.87 | 0.01 | 4.66 |
| Actor7 | 0.02 | 0.50 | -0.08 | 2.05 | -0.04 | 3.01 | 0.00 | 4.77 |
| Mean | 0.00 | 0.55 | -0.04 | 1.91 | -0.05 | 2.98 | 0.01 | 4.85 |
| cook_spinach | 0.05 | 0.48 | 0.04 | 1.68 | -0.19 | 4.27 | -0.02 | 6.46 |
| coffee_martini | 0.39 | 0.67 | -0.06 | 1.52 | 0.01 | 6.38 | -0.04 | 3.26 |
| cut_roasted_beef | 0.06 | 0.35 | -0.01 | 1.34 | -0.08 | 3.72 | -0.19 | 6.60 |
| flame_salmon | 0.74 | 0.61 | 0.09 | 1.44 | -0.02 | 8.17 | -0.01 | 3.40 |
| flame_steak | -0.08 | 0.38 | -0.03 | 1.30 | -0.16 | 4.04 | 0.39 | 7.35 |
| sear_steak | -0.11 | 0.31 | -0.01 | 1.45 | -0.17 | 4.04 | 0.20 | 5.54 |
| Mean | 0.18 | 0.47 | 0.00 | 1.46 | -0.10 | 5.10 | 0.06 | 5.44 |

**Table 3.4:** Per-frame PSNR difference and compression ratio difference between GS-NFS and the baseline. For every metric, higher is better for GS-NFS.

using `nvenc/nvdec` [84]; however, `nvenc` allows only 8 parallel encoders on a desktop-class GPU [82], and V$^3$-2D needs to encode 62 videos, so it is unlikely to achieve full-frame rate performance with offloading.

G-PCC has the next-highest encode and decode latencies; on both data sets, its encode and decode latencies are at least **260**× larger than GS-NFS's. Although GS-NFS's design borrows heavily from G-PCC elements, its GPU-acceleration results in a significant performance difference.

MesonGS is the slowest baseline. It also borrows heavily from G-PCC, and uses some GPU acceleration, but takes several seconds or minutes to encode and decode each 3DGS frame, mostly because of the time to learn the vector quantization codebook for AC coefficients [121]. Its decoding time is dominated by RAHT decoding which takes up to 200–600 ms.

Overall, GS-NFS can encode and decode *one or two orders of magnitude* faster than the baselines on our datasets.

#### 3.5.2.2 Quality and Compression Performance

Table 3.4 shows the ΔPSNR and RCR performance for each sequence from both datasets, across all baselines. To understand the results, recall that: (a) ΔPSNR captures the PSNR difference between GS-NFS and a baseline, and RCR the relative frame sizes after compression between the baseline and GS-NFS; (b) when we do this experiment, we find a setting for GS-NFS which has approximately the same PSNR on the first frame as the baseline's default setting; and (c) the values in the table average these quantities across measured frames. Finally, a positive value of ΔPSNR indicates that GS-NFS has better quality, and a value $> 1$ for RCR indicates that GS-NFS has better compression performance. Given this, we expect ΔPSNR to be close to zero for all sequences, for all approaches. This is because the sequences do not have significant scene changes, so a parameter setting from the first frame is likely to give mostly similar PSNR values for all other frames, for all baselines.

**V$^3$-2D.** For this approach, ΔPSNR is near zero for all sequences in HiFi4G. However, even though we roughly equalized PSNR, V$^3$-2D has nearly 0.4 dB lower PSNR in the *coffee_martini* sequence, and a 0.74 db lower PSNR in the *flame_salmon* sequence in the N3DV data set. V$^3$-2D has evaluated their approach on person-centric datasets, not on the larger scenes in this data set. We discuss this in greater detail in §3.5.3, where we show that this approach fails to achieve high quality on some N3DV sequences.

However, V$^3$-2D has better compression performance than GS-NFS. On HiFi4G, for example, GS-NFS's frame sizes can be **1.2–3**× larger than V$^3$-2D. On N3DV, GS-NFS's frame sizes can be **1.2–2.6**× larger than V$^3$-2D. On average, for both datasets, V$^3$-2D's frame size is about half that of GS-NFS. By using a 2D codec, V$^3$-2D can exploit inter-frame compression techniques in those codecs. In contrast, all other techniques, including GS-NFS only use intra-frame compression, so have lower compression performance.

**MesonGS.** For this approach, ΔPSNR is uniformly near zero for all sequences, as expected. However, it is less compression-efficient than GS-NFS. On the HiFi4G, data set, MesonGS's frame size is nearly 2× that of GS-NFS, and on N3DV it is 1.46×. MesonGS uses similar algorithms as ours, but with two important

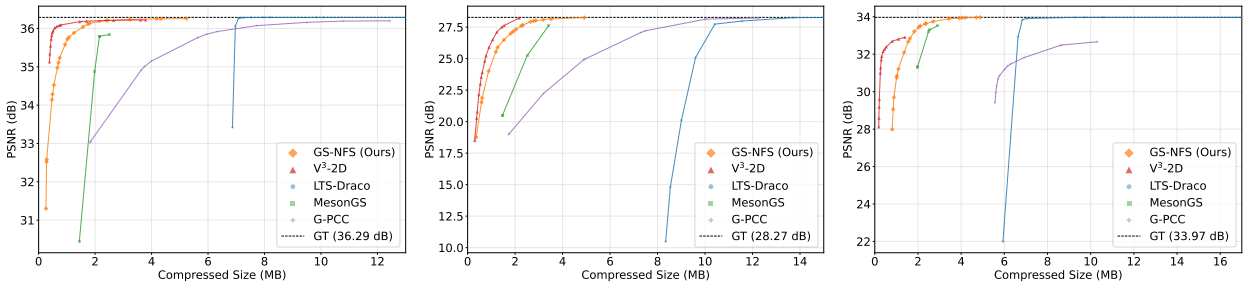



**Figure 3.6:** R-D curves for (Left) Actor1 from HiFi4G, (Middle) flame_salmon from N3DV, and (Right) sear_steak from N3DV.

---

differences. They use vector quantization for SH coefficients**, but we apply KLT (§3.4) and then quantize RAHT coefficients. They also use LZ77 for encoding attributes and positions, we use ANS and RLGR entropy encoders. We conjecture that these differences contribute to MesonGS's lower compression performance and to its higher encode-decode latencies (Table 3.2).

**LTS-Draco.** For this approach, ΔPSNR is near zero, except for a few sequences in N3DV, where LTS-Draco exhibits a **0.16–0.19** dB PSNR drop. In conducting this experiment, we sought settings for GS-NFS whose PNSR matched that of LTS-Draco, but this is not always possible, so we selected the configuration that provided the closest match. This is likely the reason for the discrepancy. LTS-Draco, however, has much worse compression performance; its frame sizes are nearly 3× GS-NFS's for HiFi4G and 5× for N3DV.

**G-PCC.** Of all the approaches, G-PCC has the highest RCR; its compression performance is 3.4× to 7.35× worse than GS-NFS. Although GS-NFS uses similar components, the adaptation [113] of G-PCC to 4DGS that we use compresses SH AC coefficients far less aggressively than GS-NFS. That adaptation also applies *lossless compression* to scales and rotations (that is, it does not quantize coefficients before entropy coding), while GS-NFS applies *lossy compression* by quantizing those attributes. These two factors contribute to the significant differences.

### 3.5.3 Rate-Distortion Curves

To understand quality and compression trade-offs of different designs, we can also use a *rate-distortion* curve or *R-D curve*. Such a curve plots *rate* on the x-axis, *quality* or distortion on the y-axis, and the curve represents the best quality one can obtain for a given rate. To obtain this curve, practitioners explore a large space of codec parameters, obtain the rate and quality for each parameter, and plot these as points on the R-D plot. The R-D curve is the *convex hull* of those points.

R-D curves can also help encode videos at a given bitrate $B$. To do this, we find the point on the convex hull whose rate is closest to $B$, and use its configuration to encode at the target bitrate. We also used R-D curves to find the GS-NFS configuration with the nearest PSNR in §3.5.2.2.

**Methodology.** We computed R-D curves for the first frame of *all* sequences, for all our baselines. To do this, we sweep the space of parameters of each baseline and GS-NFS, plot rate and distortion and compute the convex hull as discussed above. Here, we discuss the parameters we used for each baseline and for GS-NFS in generating the R-D curves. We perform an exhaustive parameter sweep for each baseline codec, per frame. For each parameter configuration, we compress and decompress the 3DGS representation of a single frame, render the decompressed Gaussians, and compute PSNR against the ground-truth renders (rendered from the uncompressed model). All renders use a resolution-downscale factor of 2. For $V^3$-2D, which operates on groups of frames, we compress a group of 20 consecutive frames starting from the target frame and report the per-frame average compressed size; PSNR and SSIM are measured only on the target frame. All other baselines compress and evaluate a single frame independently. For some approaches like MesonGS and G-PCC, we could not explore all parameter combinations because of their high encoding latency, and exploring the entire space would have taken hours or days. In contrast, for GS-NFS, we were able to evaluate 1000 combinations in about 12 minutes.

[**] MesonGS [121] was designed to encode static scenes and we have used it to encode 4DGS. Most video codecs would only retrain the vector quantization codebook every few frames, so our approach slightly over-estimates MesonGS frame sizes.

| Method | Parameter | Values | # Parameters |
|---|---|---|---|
| V$^3$-2D | QP | $\{0, 1, \ldots, 40\}$ | 41 |
| | Group size | 20 | |
| MesonGS | Octree depth | $\{8, 10, 12\}/\{12, 14, 16\}^*$ | 24 |
| | Num. bits | $\{8, 16\}$ | |
| | Blocks | $\{57, 66\}$ | |
| | Codebook | $\{2048, 4096\}$ | |
| LTS-Draco | `eg` (geom.) | $\{0, 8, 12, 16\}$ | 256 |
| | `eo` (opacity) | $\{0, 8, 12, 16\}$ | |
| | `et` (SH) | $\{0, 8, 12, 16\}$ | |
| | `es` (scale) | $\{0, 8, 12, 16\}$ | |
| | Comp. level | 10 | |
| G-PCC | Octree depth | $\{8–12\}/\{12–17\}^*$ | 225/ 270* |
| | $(q_r, q_d, q_o)^\dagger$ | 45 selected triples | |

* HiFi4G / N3DV. G-PCC total = octree depths × 45 QP triples.
† $q_r$: QP for remaining attributes, $q_d$: QP for DC coefficients, $q_o$: QP for opacity.

**Table 3.5:** Parameter sweep configurations for R-D curve generation.

The resulting (compressed size, quality) operating points are reduced to their upper convex hull to produce the final R-D curve. Table 3.5 lists the swept parameters and their ranges for each baseline. Dataset-specific settings are noted where applicable.

**R–D curves.** Fig. 3.6 shows the R–D curves for one HiFi4G sequence (*Actor1*) and two N3DV sequences (*flame_salmon_1* and *sear_steak*). Fig. 3.7 and Fig. 3.8 depict the R-D curves for the remaining sequences for the two datasets respectively. If the R-D curve for approach $A$ is entirely to the left of the curve for $B$, for a given sequence, we say that $A$ dominates $B$. If the two curves intersect, then one is better in some bitrate regimes, and worse in others.

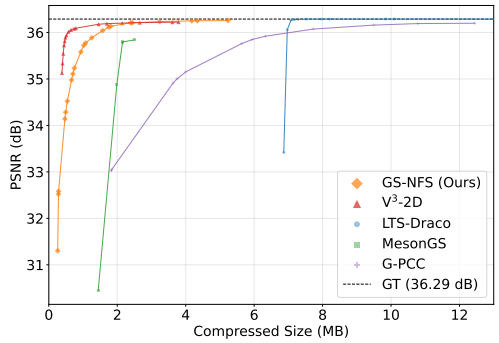
Consider the R-D curves for *Actor1*. Clearly, GS-NFS dominates MesonGS, Draco-LTS, and G-PCC. However, V$^3$-2D dominates GS-NFS: it can, for a given quality, encode at a lower rate than GS-NFS. This explains why its compression performance was better in our baseline comparisons (§3.5.2.2), and is likely due to its use of 2D codecs' ability to do inter-frame compression.

The same is true for *frame_salmon*, except for this full-scene sequence, V$^3$-2D's R-D curve is closer to, but still dominates GS-NFS's. For this sequence, both these approaches dominate MesonGS, Draco-LTS, and G-PCC.

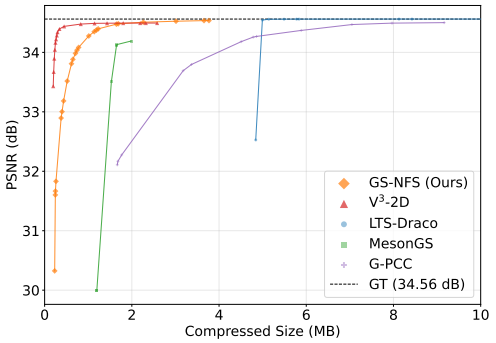
On the other hand, for *sear_steak*, V$^3$-2D does *not* dominate GS-NFS. In fact, for this sequence, and for *three other sequences* in N3DV out of a total of six (cook_spinach, cut_roasted_beef, and flame_steak, Fig. 3.8), V$^3$-2D's R-D curve shows that *there is no parameter setting for which its quality reaches close to the ground-truth PSNR* (we explored QP values from 1 to 40, where 1 is the best quality).

In each case, GS-NFS has a parameter setting that can provide **0.5–2 dB** higher PSNR for the same rate. Moreover, for all of these sequences, GS-NFS has parameter settings that can reach ground-truth (GT) quality. Put another way, if GS-NFS were used to encode videos for DASH-based 4DGS streaming, for these sequences, it would provide much better quality at higher bandwidth-availability.

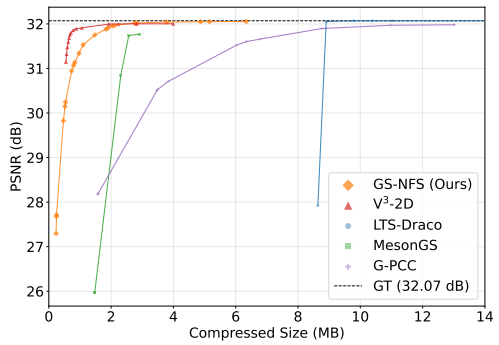
To our knowledge, V$^3$-2D has only been evaluated on person-centric datasets and not on full-scene ones. Now, V$^3$-2D must map 3D Gaussian positions to a 2D image. There are many possible 3D-to-2D mappings; V$^3$-2D applies a fixed criterion to determine where an attribute corresponding to a Gaussian in 3D should be located in the 2D. Consider two consecutive frames in 3D; the 3D-to-2D mapping algorithm is applied independently to each frame, which means that even if motion in the 3D scene is smooth, Gaussian attributes close to each other on the first 2D frame may not be close to each other in the second 2D frame. As a sequence, because the location of the same 3D attributes may change from one 2D frame to another, motion-compensated prediction in the 2D frames may not work well, potentially resulting in a higher rate than an intra-only approach. We conjecture that, for this reason, V$^3$-2D does not perform well in some N3DV sequences, where inter-frame coding becomes inefficient. On the other hand, GS-NFS does not exhibit this degradation because it uses a 3D codec.



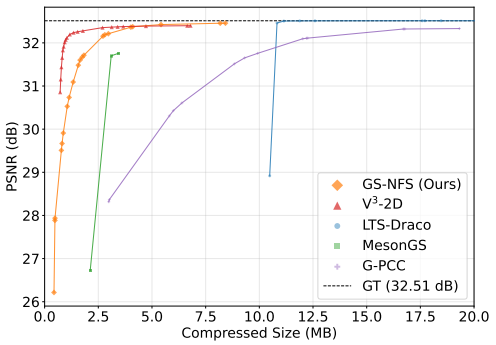

(A) Actor1_Greeting



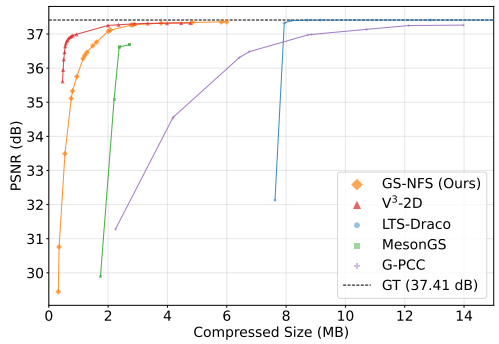

(B) Actor2_Dancing



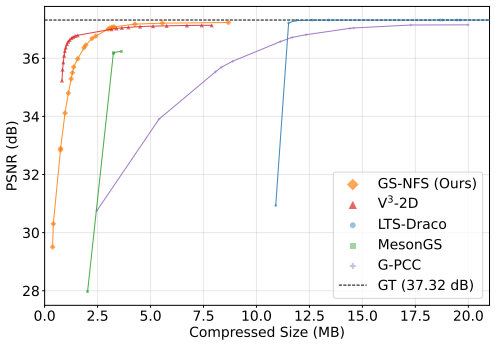

(C) Actor3_Violin



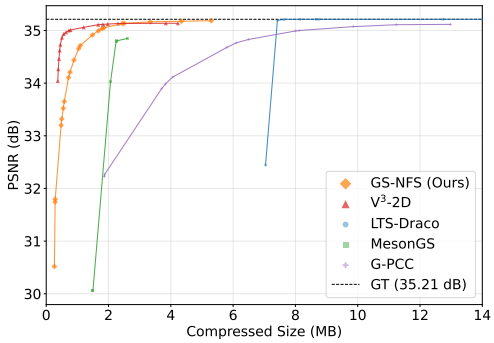

(D) Actor4_Dancing




(E) Actor5_Oil-paper_Umbrella

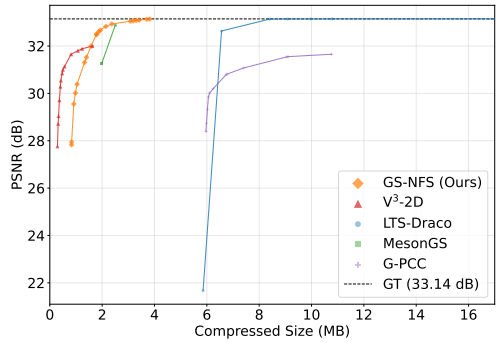



(F) Actor6_Changing_Clothes

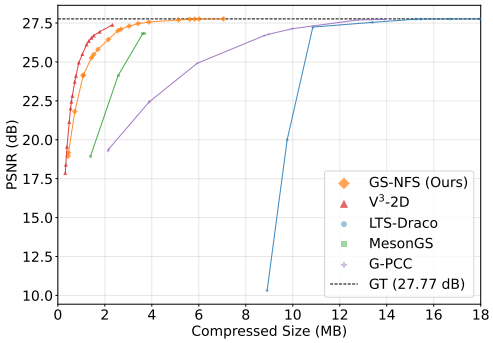



(G) Actor7_Nunchaku

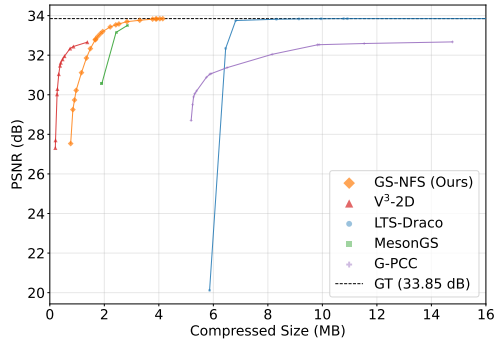
**Figure 3.7:** Rate-distortion curves for HiFi4G 4K sequences (frame 0).



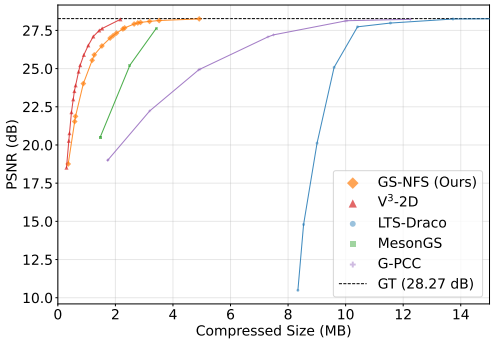

**(A)** cook_spinach



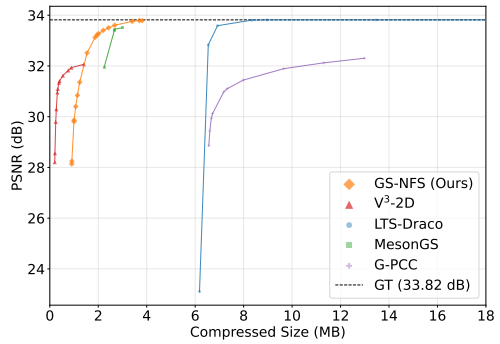

**(B)** coffee_martini



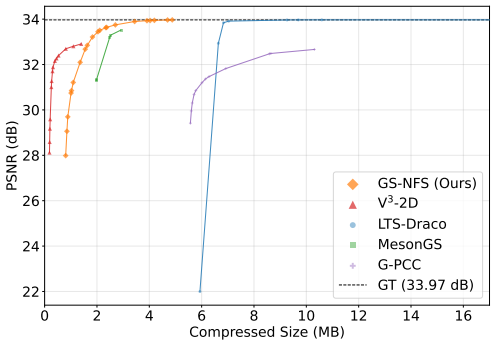

**(C)** cut_roasted_beef




**(D)** flame_salmon_1




**(E)** flame_steak




**(F)** sear_steak

**FIGURE 3.8:** Rate-distortion curves for N3DV sequences (frame 1).

| Sequence | SH degree | # channels | Decode latency (ms) |
|---|---|---|---|
| Actor1 | 0 | 11 | 40 |
| Actor1 | 3 | 56 | 57 |
| flame_salmon | 0 | 11 | 59 |
| flame_salmon | 2 | 35 | 120 |
| sear_steak | 0 | 11 | 49 |
| sear_steak | 2 | 35 | 113 |

**Table 3.6:** Decode latency on Jetson Orin, averaged across frames.

### 3.5.4 Mobile Performance

GS-NFS supports 4DGS playback on resource-constrained devices. To demonstrate this, we evaluate GS-NFS's decode performance on an NVIDIA Jetson Orin, a mobile GPU with compute comparable to those in in AR/VR headsets and edge devices [83]. Table 3.6 shows end-to-end decode latency on Jetson Orin. We observed that many 3DGS mobile implementations can support 30 fps 3DGS rendering for Gaussians with only SH DC coefficients (SH0), but not at higher SH degrees [114]. At SH0, GS-NFS achieves 40–59 ms across all sequences, allowing about 17–25 fps decoding. Latency scales *sub-linearly* with channel count: going from SH0 to SH3 (5.1× more channels) increases average decode latency across all sequences by $\sim 2\times$. At higher SH degrees, scene complexity dominates for full-scene content (more Gaussians): flame_salmon at SH2 (35 ch) is $2\times$ slower than Actor1 at SH3 (56 ch), despite having fewer channels.

### 3.5.5 Novel GS-NFS Use Cases

So far, we have discussed using GS-NFS to compress stored 4DGS videos. In this section, we demonstrate that GS-NFS can **compress, on-the-fly, live 4DGS videos** generated by feed-forward Gaussian generators and **point-clouds** as well, both at full frame-rate.

**Static 3DGS scene compression.** We evaluate GS-NFS on two large static outdoor scenes from the Deep Blending dataset (*drjohnson*, 3M Gaussians, 748 MB) and Tanks & Temples (*truck*, 2M Gaussians, 485 MB). Table 3.7 shows the results. GS-NFS achieves encode times of 91–165 ms, which is **31× faster** than

| Scene | Method | **Enc.** (ms) | **Dec.** (ms) | **PSNR** (dB) | **Size** (MB) |
|---|---|---|---|---|---|
| *drjohnson* | LTS-Draco | 5,084 | 2,610 | 35.5 | 312.8 |
| | MesonGS | 220,066 | 22,421 | 32.6 | 40.6 |
| | **GS-NFS** | **165** | **183** | 34.4 | 37.8 |
| *truck* | LTS-Draco | 3,385 | 1,792 | 23.1 | 204.0 |
| | MesonGS | 319,395 | 14,357 | 22.7 | 27.8 |
| | **GS-NFS** | **91** | **108** | 22.4 | 19.0 |

**Table 3.7:** Static 3DGS scene compression. PSNR, SSIM, and LPIPS computed on rendered test views.

LTS-Draco and **1,200–3,400× faster** than MesonGS. At quality comparable to LTS-Draco (<1 dB PSNR difference), GS-NFS produces compressed files that are 5–10× smaller.

**Live 4DGS Compression.** GPS-Gaussian [128] generates Gaussian at ∼25 fps (∼270K Gaussians per frame) using captures from 2 views of a person. We adapted GS-NFS to compress these frames on-the-fly. Specifically, our approach repeatedly takes 2 views of a person per frame (pre-loaded from disk), generates a 3DGS frame using GPS-Gaussian, compresses the frame using GS-NFS, and finally decompresses the frame to reconstruct the 3DGS frame. This pipeline can achieve **22 fps** and an end-to-end latency (from capture to after decode) of **75 ms**. While GS-NFS's encoder and decoder takes only 15 ms each, the GPS-Gaussian generator takes ∼43 ms, and is the bottleneck in this pipeline.

**Point cloud compression.** Since GS-NFS builds upon G-PCC, it can compress point clouds (3DGS with only DC attribute). So, we evaluate its performance on popular point cloud-based volumetric video datasets. We use six sequences from two datasets: four single person sequences from 8i [17] (∼700K–1M points/frame; ∼17–26 MB) and 2 multi-person full-scene sequences from CMU Panoptic [47] (∼700K–850K points/frame; ∼17–21 MB). Point clouds are voxelized at octree depth $J = 10$ and GS-NFS's codec uses a quantization step 0.1 for color attributes. GS-NFS can encode each frame in 12–14 ms (avg. 3.3 ms for geometry) and decode in 9–12 ms (avg. 2.6 ms for geometry) while achieving **lossless geometry** and color Y-PSNR of 31–37 dB; refer Table 3.8 for octree encoding/decoding. It can compress 8i sequences to 270–370 KB and CMU Panoptic sequences to 660–780 KB, **26–64**× smaller than the original size.

| Sequence | Method | Enc. (ms) | Dec. (ms) | Size (KB) |
|---|---|---|---|---|
| longdress | Draco (CPU) | 126 | 57 | 395 |
| | G-PCC (CPU) | 195 | 97 | 195 |
| | GS-NFS octree | **2.9** | **1.9** | 238 |
| soldier | Draco (CPU) | 170 | 79 | 533 |
| | G-PCC (CPU) | 268 | 136 | 273 |
| | GS-NFS octree | **3.4** | **2.4** | 332 |
| band2 | Draco (CPU) | 123 | 54 | 561 |
| | G-PCC (CPU) | 325 | 213 | 526 |
| | GS-NFS octree | **3.3** | **2.6** | 565 |
| pizza1 | Draco (CPU) | 137 | 59 | 617 |
| | G-PCC (CPU) | 343 | 222 | 572 |
| | GS-NFS octree | **3.3** | **2.6** | 616 |

**Table 3.8:** Position-only octree encoding on point-cloud sequences. Encode/decode in ms, size in KB, averaged per frame.

| Sequence | Compressed Size (MB) | | | Relative Ratio | |
|---|---|---|---|---|---|
| | RGB | YUV | KLT | RGB/KLT | YUV/KLT |
| flame_salmon | 11.94 | 11.37 | 8.35 | 1.43 | 1.36 |
| sear_steak | 9.29 | 8.99 | 6.78 | 1.37 | 1.33 |
| Actor1 | 10.97 | 10.43 | 9.90 | 1.11 | 1.05 |

**Table 3.9:** Impact of color decorrelation on compressed size (megabytes).

### 3.5.6 Ablations

**KLT color decorrelation.** GS-NFS decorrelates SH coefficients using a YUV transform followed by a per-channel KLT (§3.4). Table 3.9 shows the impact on compressed size. RGB inflates the attribute bitstream by **43%** compared to raw RGB, while YUV is larger than KLT by **36%** for flame_salmon. For others, the reductions are smaller, but still significant. The additional latency for computing and applying the KLT matrix is less than 1 ms per frame, a negligible overhead.

**CUDA vs. PyTorch RAHT decode on Jetson.** RAHT takes about $\sim$50% of the per frame decode latency, particularly on Jetson Orin. An efficient implementation of RAHT is crucial for mobile decode. GS-NFS's custom CUDA RAHT kernels achieve 2.0–3.9$\times$ speedup over a PyTorch baseline on Jetson, with larger gains for lower SH degrees (Table 3.10). We see a reduction of about 4–5$\times$ for SH0 and 2–4$\times$ for higher SH

| Sequence | SH degree | Tot. ch | PyTorch (ms) | CUDA (ms) | Speedup |
|---|---|---|---|---|---|
| Actor1 | 0 | 11 | 77 | 20 | 3.9× |
| Actor1 | 3 | 56 | 100 | 28 | 3.6× |
| flame_salmon | 0 | 11 | 101 | 31 | 3.3× |
| flame_salmon | 2 | 35 | 125 | 57 | 2.2× |
| sear_steak | 0 | 11 | 79 | 25 | 3.2× |
| sear_steak | 2 | 35 | 96 | 47 | 2.0× |

**Table 3.10:** RAHT decode latency on Jetson Orin (ms). CUDA vs. PyTorch implementation. Averaged across sampled frames.

| Variant | Block size | Enc. (ms) | Dec. (ms) | ΔSize |
|---|---|---|---|---|
| CPU | – | 143.8 | 207.1 | 0.0% |
| GPU | – | 494.2 | 447.4 | 0.0% |
| GPU | 8192 | 15.7 | 15.0 | 0.1% |
| GPU | 4096 | 8.0 | 7.6 | 0.2% |
| GPU | 2048 | 4.1 | 4.0 | 0.4% |
| GPU | 1024 | 2.9 | 2.4 | 0.8% |
| GPU | 512 | 2.5 | 1.9 | 1.5% |
| GPU | 256 | 2.5 | 2.1 | 2.5% |

**Table 3.11:** RLGR entropy coding latency: CPU vs. GPU with varying block sizes for flame_salmon.

degrees for RAHT decode, which is the main bottleneck for mobile decode; prelude has ∼2× of speedup, since it doesn't depend on the number of channels. These gains also reduce encode latency, since both forward and inverse RAHT perform the same work.

**RLGR parallelization.** GS-NFS parallelizes RLGR entropy coding by partitioning each attribute channel into fixed-size blocks and encoding them independently on GPU threads (§3.3.4). Table 3.11 compares the CPU baseline (sequential per-channel RLGR) against GPU variants with different block sizes. Parallel RLGR a block size of 512 achieves a **58× encode** and **109× decode speedup** on a desktop GPU over CPU, with only ∼1.5% increase in compressed size. On Jetson Orin, a block size of 512 achieves ∼6 ms decode time per frame, compared to ∼180 ms for the CPU implementation–a 30× speedup. Smaller block sizes below 512 starts having diminishing returns. Block size 2048–8192 provides a balanced tradeoff: 18–51× speedup in encode/decode latency with negligible size overhead.

| Sequence | Method | Enc. (ms) | Dec. (ms) | Comp Ratio |
|---|---|---|---|---|
| flame_salmon (400K Gauss) | Draco (CPU) | 98.0 | 37.5 | 10.9× |
| | G-PCC (CPU) | 228.6 | 127.5 | 11.3× |
| | Octree (w/o ANS) | 2.9 | 2.4 | 6.6× |
| | Octree (w/ ANS) | **3.2** | **2.7** | 11.2× |
| sear_steak (250K Gauss) | Draco (CPU) | 64.9 | 28.1 | 5.1× |
| | G-PCC (CPU) | 294.8 | 204.3 | 5.2× |
| | Octree (w/o ANS) | 2.9 | 3.0 | 2.5× |
| | Octree (w/ ANS) | **3.2** | **3.1** | 4.8× |
| Actor1 (130K Gauss) | Draco (CPU) | 30.7 | 13.0 | 6.7× |
| | G-PCC (CPU) | 127.8 | 91.2 | 6.8× |
| | Octree (w/o ANS) | 1.8 | 1.4 | 3.4× |
| | Octree (w/ ANS) | **1.9** | **1.6** | 6.1× |

**Table 3.12:** Octree compression: CPU-based methods vs GS-NFS's octree codec.

## 3.6 Related Work

**During-training 3DGS compression.** Recent research has focused on integrating compression directly into an optimization loop within training to enhance model sparsity. Compact3DGS [59] identifies prunes redundant Gaussians (as do [26, 70]) and replaces spherical harmonics with hash-based grids. CompGS [76] utilizes quantization-aware training and opacity regularization. 4D-GS [119] trains an MLP to predict motion across frames, while QUEEN [28] learns quantized attribute residuals and uses a gating module to sparsify positions. LapisGS [99] trains 3DGS in a layered progressive representation. Relative to GS-NFS, these methods incur significantly higher overhead for compression.

**Post-training 3DGS compression.** MesonGS [121] prunes insignificant Gaussians and then applies octree and RAHT. Some schemes [**gauspcgc**, 38] use learning based methods to find the compression parameters for position and attributes. Others [104, 113] use point cloud compression techniques to compress trained 3DGS. To address this, $V^3$ [114] encodes 4DGS in multiple 2D videos. GS-NFS encodes and decodes much faster than these techniques.

## 3.7 Conclusions

GS-NFS is a frame-rate GPU-accelerated encoder and decoder for 4DGS videos. To our knowledge, it is the first to demonstrate frame-rate encoding and decoding performance not just for 4DGS, but also for point clouds. It achieves this performance using novel parallelization algorithms for position and attributes. Its compression performance is also better than many state-of-the-art approaches due to a novel de-correlation approach. Future work can develop rate-adaptive codecs that can dynamically adapt compression levels in response to bandwidth changes, using GS-NFS's ability to quickly encode and decode frames.

# Bibliography


[1] Zahaib Akhtar, Yun Seong Nam, Ramesh Govindan, Sanjay Rao, Jessica Chen, Ethan Katz-Bassett, Bruno Ribeiro, Jibin Zhan, and Hui Zhang. "Oboe: Auto-tuning Video ABR Algorithms to Network Conditions". In: *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. SIGCOMM '18. Budapest, Hungary: ACM, Jan. 1, 2018, pp. 44–58. ISBN: 978-1-4503-5567-4. DOI: 10.1145/3230543.3230558. published.

[2] Evangelos Alexiou and Touradj Ebrahimi. "Towards a Point Cloud Structural Similarity Metric". In: *2020 IEEE International Conference on Multimedia & Expo Workshops (ICMEW)*. 2020, pp. 1–6. DOI: 10.1109/ICMEW46912.2020.9106005.

[3] *Azure Kinect DK*. https://www.microsoft.com/en-us/d/azure-kinect-dk/8pp5vxmd9nhq. 2024.

[4] *Azure Kinect DK hardware specifications*. https://learn.microsoft.com/en-us/azure/kinect-dk/hardware-specification. 2022.

[5] *Azure-Kinect-Sensor-SDK*. https://github.com/microsoft/Azure-Kinect-Sensor-SDK.

[6] Michael Bader. *Space-filling curves: an introduction with applications in scientific computing*. Vol. 9. Springer Science & Business Media, 2012.

[7] Seonghoon Ban and Kyung Hoon Hyun. "Pixel of Matter: New Ways of Seeing with an Active Volumetric Filmmaking System". In: *ACM SIGGRAPH 2020 Art Gallery*. SIGGRAPH '20. Virtual Event, USA: Association for Computing Machinery, 2020, pp. 434–437. ISBN: 9781450379526. DOI: 10.1145/3386567.3388576.

[8] Niklas Blum, Serge Lachapelle, and Harald Alvestrand. "WebRTC - Realtime Communication for the Open Web Platform: What was once a way to bring audio and video to the web has expanded into more use cases we could ever imagine." In: *Queue* 19.1 (Mar. 2021), pp. 77–93. ISSN: 1542-7730. DOI: 10.1145/3454122.3457587.

[9] *Boost C++ Libraries*. https://www.boost.org/.

[10] Gaetano Carlucci, Luca De Cicco, Stefan Holmer, and Saverio Mascolo. "Analysis and design of the google congestion control for web real-time communication (WebRTC)". In: *Proceedings of the 7th International Conference on Multimedia Systems*. MMSys '16. Klagenfurt, Austria: Association for Computing Machinery, 2016. ISBN: 9781450342971. DOI: 10.1145/2910017.2910605.

[11] Li-Heng Chen, Christos G. Bampis, Zhi Li, Joel Sole, and Alan C. Bovik. “Perceptual Video Quality Prediction Emphasizing Chroma Distortions”. In: *IEEE Transactions on Image Processing* 30 (2021), pp. 1408–1422. DOI: 10.1109/TIP.2020.3043127.

[12] Ruopeng Chen, Mengbai Xiao, Dongxiao Yu, Guanghui Zhang, and Yao Liu. “patchVVC: A Real-time Compression Framework for Streaming Volumetric Videos”. In: *Proceedings of the 14th Conference on ACM Multimedia Systems*. 2023, pp. 119–129.

[13] Austin Chronicle. *SXSW Panel Recap: The Biz of Being Human: Volumetric Humans at Scale*. https://www.austinchronicle.com/daily/sxsw/2023-03-14/sxsw-panel-recap-the-biz-of-being-human-volumetric-humans-at-scale/. 2023.

[14] *Cisco Annual Internet Report (2018–2023) White Paper*. https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html. Jan. 2022.

[15] Alvaro Collet, Ming Chuang, Pat Sweeney, Don Gillett, Dennis Evseev, David Calabrese, Hugues Hoppe, Adam Kirk, and Steve Sullivan. “High-quality streamable free-viewpoint video”. In: *ACM Transactions on Graphics (ToG)* 34.4 (2015), pp. 1–13.

[16] NVIDIA Corporation. *NVIDIA nvCOMP - A CUDA library for Fast Lossless Compression*. 2025. URL: https://developer.nvidia.com/nvcomp.

[17] E d’Eon, B Harrison, T Myers, and PA Chou. “8i Voxelized Full Bodies—A Voxelized Point Cloud Dataset, document WG11M40059/WG1M74006”. In: *ISO/IEC JTC1/SC29 Joint WG11/WG1 (MPEG/JPEG), Geneva, Switzerland* (2017).

[18] Ricardo L De Queiroz and Philip A Chou. “Compression of 3D point clouds using a region-adaptive hierarchical transform”. In: *IEEE Transactions on Image Processing* 25.8 (2016), pp. 3947–3956.

[19] *Depthkit Holoportation via Webcam*. https://www.depthkit.tv/tutorials/depthkit-holoportation-via-webcam. 2022.

[20] Sandesh Dhawaskar Sathyanarayana, Kyunghan Lee, Dirk Grunwald, and Sangtae Ha. “Converge: QoE-driven Multipath Video Conferencing over WebRTC”. In: *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM SIGCOMM ’23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 637–653. ISBN: 9798400702365. DOI: 10.1145/3603269.3604822.

[21] Mingsong Dou, Sameh Khamis, Yury Degtyarev, Philip Davidson, Sean Ryan Fanello, Adarsh Kowdle, Sergio Orts Escolano, Christoph Rhemann, David Kim, Jonathan Taylor, et al. “Fusion4d: Real-time performance capture of challenging scenes”. In: *ACM Transactions on Graphics (ToG)* 35.4 (2016), pp. 1–13.

[22] *Draco 3D Compression*. https://github.com/google/draco.

[23] *Eigen*. https://eigen.tuxfamily.org/.

[24] *FFmpeg*. https://ffmpeg.org/.

[25] Sadjad Fouladi, John Emmons, Emre Orbay, Catherine Wu, Riad S. Wahby, and Keith Winstein. "Salsify: Low-Latency Network Video through Tighter Integration between a Video Codec and a Transport Protocol". In: *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. Renton, WA: USENIX Association, Apr. 2018, pp. 267–282. ISBN: 978-1-939133-01-4. URL: https://www.usenix.org/conference/nsdi18/presentation/fouladi.

[26] Jiaye Fu, Qiankun Gao, Chengxiang Wen, Yanmin Wu, Siwei Ma, Jiaqi Zhang, and Jian Zhang. "ReCon-GS: Continuum-Preserved Gaussian Streaming for Fast and Compact Reconstruction of Dynamic Scenes". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2025.

[27] Ehab Ghabashneh, Chandan Bothra, Ramesh Govindan, Antonio Ortega, and Sanjay Rao. "Dragonfly: Higher Perceptual Quality For Continuous 360° Video Playback". In: *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM SIGCOMM '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 516–532. ISBN: 9798400702365. DOI: 10.1145/3603269.3604876.

[28] Sharath Girish, Tianye Li, Amrita Mazumdar, Abhinav Shrivastava, david luebke, and Shalini De Mello. "QUEEN: QUantized Efficient ENcoding for Streaming Free-viewpoint Videos". In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. 2024. URL: https://openreview.net/forum?id=7xhwE7VH4S.

[29] D. Graziosi, O. Nakagami, S. Kuma, A. Zaghetto, T. Suzuki, and A. Tabatabai. "An overview of ongoing point cloud compression standardization activities: video-based (V-PCC) and geometry-based (G-PCC)". In: *APSIPA Transactions on Signal and Information Processing* 9 (2020), e13. DOI: 10.1017/ATSIP.2020.12.

[30] Yongjie Guan, Xueyu Hou, Nan Wu, Bo Han, and Tao Han. "MetaStream: Live Volumetric Content Capture, Creation, Delivery, and Rendering in Real Time". In: *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*. New York, NY, USA: Association for Computing Machinery, 2023. ISBN: 9781450399906. URL: https://doi.org/10.1145/3570361.3592530.

[31] Serhan Gül, Sebastian Bosse, Dimitri Podborski, Thomas Schierl, and Cornelius Hellge. "Kalman Filter-based Head Motion Prediction for Cloud-based Mixed Reality". In: *Proceedings of the 28th ACM International Conference on Multimedia*. MM '20. Seattle, WA, USA: Association for Computing Machinery, 2020, pp. 3632–3641. ISBN: 9781450379885. DOI: 10.1145/3394171.3413699.

[32] Simon N. B. Gunkel, Rick Hindriks, Karim M. El Assal, Hans M. Stokking, Sylvie Dijkstra-Soudarissanane, Frank ter Haar, and Omar Niamut. "VRComm: an end-to-end web system for real-time photorealistic social VR communication". In: *Proceedings of the 12th ACM Multimedia Systems Conference*. MMSys '21. Istanbul, Turkey: Association for Computing Machinery, 2021, pp. 65–79. ISBN: 9781450384346. DOI: 10.1145/3458305.3459595.

[33] Bo Han, Yu Liu, and Feng Qian. "ViVo: Visibility-Aware Mobile Volumetric Video Streaming". In: *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. MobiCom '20. London, United Kingdom: Association for Computing Machinery, 2020. ISBN: 9781450370851. DOI: 10.1145/3372224.3380888.

[34] Raphael Hauser. “Line search methods for unconstrained optimisation”. In: *Lecture 8, Numerical Linear Algebra and Optimisation Oxford University Computing Laboratory* (2007).

[35] Haoran Hong, Eduardo Pavez, Antonio Ortega, Ryosuke Watanabe, and Keisuke Nonaka. “Fractional motion estimation for point cloud compression”. In: *2022 Data Compression Conference (DCC)*. 2022, pp. 369–378.

[36] Haoran Hong, Eduardo Pavez, Antonio Ortega, Ryosuke Watanabe, and Keisuke Nonaka. “Motion estimation and filtered prediction for dynamic point cloud attribute compression”. In: *2022 Picture Coding Symposium (PCS)*. 2022, pp. 139–143.

[37] Te-Yuan Huang, Ramesh Johari, Nick McKeown, Matthew Trunnell, and Mark Watson. “A Buffer-based Approach to Rate Adaptation: Evidence from a Large Video Streaming Service”. In: *Proceedings of the 2014 ACM Conference on SIGCOMM*. SIGCOMM ’14. Chicago, Illinois, USA: ACM, 2014, pp. 187–198. ISBN: 978-1-4503-2836-4. DOI: 10.1145/2619239.2626296.

[38] Yuning Huang, Jiahao Pang, Fengqing Zhu, and Dong Tian. “EntropyGS: An Efficient Entropy Coding on 3D Gaussian Splatting”. In: *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2026.

[39] Apple Inc. *Apple Introduces Spatial Video Capture on Iphone 15 Pro*. https://www.apple.com/newsroom/2023/12/apple-introduces-spatial-video-capture-on-iphone-15-pro/. 2023.

[40] Netflix Inc. *Per-title Encode Optimization*. https://netflixtechblog.com/per-title-encode-optimization-7e99442b62a2. 2015.

[41] *Intel RealSense Depth Camera D455*. https://www.intelrealsense.com/. 2024.

[42] *Intel® RealSense™ Depth Camera D455*. https://www.intelrealsense.com/depth-camera-d455/. 2020.

[43] Andrew Irlitti, Mesut Latifoglu, Qiushi Zhou, Martin N Reinoso, Thuong Hoang, Eduardo Velloso, and Frank Vetere. “Volumetric Mixed Reality Telepresence for Real-Time Cross Modality Collaboration”. In: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. CHI ’23. Hamburg, Germany: Association for Computing Machinery, 2023. ISBN: 9781450394215. DOI: 10.1145/3544548.3581277.

[44] Junchen Jiang, Vyas Sekar, and Hui Zhang. “Improving Fairness, Efficiency, and Stability in HTTP-based Adaptive Video Streaming with FESTIVE”. In: *Proceedings of the 8th International Conference on Emerging Networking Experiments and Technologies*. CoNEXT ’12. Nice, France: ACM, 2012, pp. 97–108. ISBN: 978-1-4503-1775-7. DOI: 10.1145/2413176.2413189.

[45] Tao Jin, Mallesham Dasa, Connor Smith, Kittipat Apicharttrisorn, Srinivasan Seshan, and Anthony Rowe. “MeshReduce: Scalable and Bandwidth Efficient 3D Scene Capture”. In: *2024 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*. 2024, pp. 20–30. DOI: 10.1109/VR58804.2024.00026.

[46] Hanbyul Joo, Tomas Simon, Xulong Li, Hao Liu, Lei Tan, Lin Gui, Sean Banerjee, Timothy Scott Godisart, Bart Nabbe, Iain Matthews, Takeo Kanade, Shohei Nobuhara, and Yaser Sheikh. "Panoptic Studio: A Massively Multiview System for Social Interaction Capture". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2017).

[47] Hanbyul Joo, Tomas Simon, Xulong Li, Hao Liu, Lei Tan, Lin Gui, Sean Banerjee, Timothy Scott Godisart, Bart Nabbe, Iain Matthews, Takeo Kanade, Shohei Nobuhara, and Yaser Sheikh. "Panoptic Studio: A Massively Multiview System for Social Interaction Capture". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2017).

[48] R. E. Kalman. "A New Approach to Linear Filtering and Prediction Problems". In: *Journal of Basic Engineering* 82.1 (Mar. 1960), pp. 35–45. ISSN: 0021-9223. DOI: 10.1115/1.3662552. eprint: https://asmedigitalcollection.asme.org/fluidsengineering/article-pdf/82/1/35/5518977/35_1.pdf.

[49] Tero Karras. "Maximizing parallelism in the construction of BVHs, octrees, and k-d trees". In: *Proceedings of the Fourth ACM SIGGRAPH / Eurographics Conference on High-Performance Graphics*. EGGH-HPG'12. Paris, France: Eurographics Association, 2012, pp. 33–37. ISBN: 9783905674415.

[50] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. "3D Gaussian Splatting for Real-Time Radiance Field Rendering". In: *ACM Transactions on Graphics* 42.4 (July 2023). URL: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/.

[51] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. "3D Gaussian Splatting for Real-Time Radiance Field Rendering". In: *ACM Transactions on Graphics* 42.4 (July 2023). URL: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/.

[52] Gunjoong Kim, Seonghoon Park, Jeho Lee, Chanyoung Jung, Hyungchol Jun, and Hojung Cha. "Vega: Fully Immersive Mobile Volumetric Video Streaming with 3D Gaussian Splatting". In: *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. ACM MOBICOM '25. Kerry Hotel, Hong Kong, Hong Kong, China: Association for Computing Machinery, 2025, pp. 1106–1120. ISBN: 9798400711299. DOI: 10.1145/3680207.3765267.

[53] Woo-Shik Kim, Antonio Ortega, PoLin Lai, and Dong Tian. "Depth Map Coding Optimization Using Rendered View Distortion for 3D Video Coding". In: *IEEE Transactions on Image Processing* 24.11 (2015), pp. 3534–3545. DOI: 10.1109/TIP.2015.2447737.

[54] Naimin Koh, Pradeep Kumar Jayaraman, and Jianmin Zheng. "Parallel Point Cloud Compression Using Truncated Octree". In: *2020 International Conference on Cyberworlds (CW)*. 2020, pp. 1–8. DOI: 10.1109/CW49994.2020.00009.

[55] Marek Kowalski, Jacek Naruniec, and Michal Daniluk. "Livescan3D: A Fast and Inexpensive 3D Data Acquisition System for Multiple Kinect v2 Sensors". In: *2015 International Conference on 3D Vision*. 2015, pp. 318–325. DOI: 10.1109/3DV.2015.43.

[56] Jason Lawrence, Danb Goldman, Supreeth Achar, Gregory Major Blascovich, Joseph G. Desloge, Tommy Fortes, Eric M. Gomez, Sascha Häberling, Hugues Hoppe, Andy Huibers, Claude Knaus, Brian Kuschak, Ricardo Martin-Brualla, Harris Nover, Andrew Ian Russell, Steven M. Seitz, and Kevin Tong. "Project Starline: A High-Fidelity Telepresence System". In: *ACM Trans. Graph.* 40.6 (Dec. 2021). ISSN: 0730-0301. DOI: 10.1145/3478513.3480490.

[57] Insoo Lee, Seyeon Kim, Sandesh Sathyanarayana, Kyungmin Bin, Song Chong, Kyunghan Lee, Dirk Grunwald, and Sangtae Ha. "R-FEC: RL-based FEC Adjustment for Better QoE in WebRTC". In: *Proceedings of the 30th ACM International Conference on Multimedia*. MM '22. Lisboa, Portugal: Association for Computing Machinery, 2022, pp. 2948–2956. ISBN: 9781450392037. DOI: 10.1145/3503161.3548370.

[58] Insoo Lee, Jinsung Lee, Kyunghan Lee, Dirk Grunwald, and Sangtae Ha. "Demystifying Commercial Video Conferencing Applications". In: *Proceedings of the 29th ACM International Conference on Multimedia*. MM '21. Virtual Event, China: Association for Computing Machinery, 2021, pp. 3583–3591. ISBN: 9781450386517. DOI: 10.1145/3474085.3475523.

[59] Joo Chan Lee, Daniel Rho, Xiangyu Sun, Jong Hwan Ko, and Eunbyung Park. *Compact 3D Gaussian Splatting for Static and Dynamic Radiance Fields*. 2024. arXiv: 2408.03822 [cs.CV]. URL: https://arxiv.org/abs/2408.03822.

[60] Kyungjin Lee, Juheon Yi, and Youngki Lee. "FarfetchFusion: Towards Fully Mobile Live 3D Telepresence Platform". In: *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*. New York, NY, USA: Association for Computing Machinery, 2023. ISBN: 9781450399906. URL: https://doi.org/10.1145/3570361.3592525.

[61] Kyungjin Lee, Juheon Yi, Youngki Lee, Sunghyun Choi, and Young Min Kim. "GROOT: A Real-Time Streaming System of High-Fidelity Volumetric Videos". In: *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. MobiCom '20. London, United Kingdom: Association for Computing Machinery, 2020. ISBN: 9781450370851. DOI: 10.1145/3372224.3419214.

[62] Boyan Li, Yongting Chen, Dayou Zhang, and Fangxin Wang. *DeformStream: Deformation-based Adaptive Volumetric Video Streaming*. 2024. arXiv: 2409.16615 [cs.CV]. URL: https://arxiv.org/abs/2409.16615.

[63] Lingzhi Li, Zhen Shen, Zhongshu Wang, Li Shen, and Ping Tan. "Streaming radiance fields for 3d video synthesis". In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 13485–13498.

[64] Zhuqi Li, Yaxiong Xie, Ravi Netravali, and Kyle Jamieson. "Dashlet: Taming Swipe Uncertainty for Robust Short Video Streaming". In: *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 1583–1599. ISBN: 978-1-939133-33-5. URL: https://www.usenix.org/conference/nsdi23/presentation/li-zhuqi.

[65] Zhuqi Li, Yaxiong Xie, Longfei Shangguan, Rotman Ivan Zelaya, Jeremy Gummeson, Wenjun Hu, and Kyle Jamieson. "Towards Programming the Radio Environment with Large Arrays of Inexpensive Antennas". In: *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. Boston, MA: USENIX Association, Feb. 2019, pp. 285–300. ISBN: 978-1-931971-49-2. URL: https://www.usenix.org/conference/nsdi19/presentation/lizhuqi.

[66] Zhicheng Liang, Junhua Liu, Mallesham Dasari, and Fangxin Wang. "Fumos: Neural Compression and Progressive Refinement for Continuous Point Cloud Video Streaming". In: *IEEE Transactions on Visualization and Computer Graphics* 30.5 (2024), pp. 2849–2859. DOI: 10.1109/TVCG.2024.3372096.

[67] Bangya Liu and Suman Banerjee. *SwinGS: Sliding Window Gaussian Splatting for Volumetric Video Streaming with Arbitrary Length*. 2024. arXiv: 2409.07759 [cs.MM]. URL: https://arxiv.org/abs/2409.07759.

[68] Xiangrui Liu, Xinju Wu, Shiqi Wang, Zhu Li, and Sam Kwong. *CompGS++: Compressed Gaussian Splatting for Static and Dynamic Scene Representation*. 2025. arXiv: 2504.13022 [cs.GR]. URL: https://arxiv.org/abs/2504.13022.

[69] Yu Liu, Bo Han, Feng Qian, Arvind Narayanan, and Zhi-Li Zhang. "Vues: practical mobile volumetric video streaming through multiview transcoding". In: *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*. MobiCom '22. Sydney, NSW, Australia: Association for Computing Machinery, 2022, pp. 514–527. ISBN: 9781450391818. DOI: 10.1145/3495243.3517027.

[70] Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. "Scaffold-gs: Structured 3d gaussians for view-adaptive rendering". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 20654–20664.

[71] H.S. Malvar. "Adaptive run-length/Golomb-Rice encoding of quantized generalized Gaussian sources with unknown statistics". In: *Data Compression Conference (DCC'06)*. 2006, pp. 23–32. DOI: 10.1109/DCC.2006.5.

[72] Donald Meagher. "Geometric modeling using octree encoding". In: *Computer Graphics and Image Processing* 19.2 (1982), pp. 129–147. ISSN: 0146-664X. DOI: https://doi.org/10.1016/0146-664X(82)90104-6.

[73] Gabriel Meynet, Yana Nehmé, Julie Digne, and Guillaume Lavoué. "PCQM: A Full-Reference Quality Metric for Colored 3D Point Clouds". In: *2020 Twelfth International Conference on Quality of Multimedia Experience (QoMEX)*. 2020, pp. 1–6. DOI: 10.1109/QoMEX48832.2020.9123147.

[74] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis". In: *ECCV*. 2020.

[75] MPEG. *GPCC - mpeg-pcc-tmc13*. https://github.com/MPEGGroup/mpeg-pcc-tmc13. 2023.

[76] K L Navaneet, Kossar Pourahmadi Meibodi, Soroush Abbasi Koohpayegani, and Hamed Pirsiavash. "CompGS: Smaller and Faster Gaussian Splatting with Vector Quantization". In: *Computer Vision - ECCV 2024: 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part XXXII*. Milan, Italy: Springer-Verlag, 2024, pp. 330–349. ISBN: 978-3-031-73410-6. DOI: 10.1007/978-3-031-73411-3_19.

[77] Ravi Netravali, Anirudh Sivaraman, Somak Das, Ameesh Goyal, Keith Winstein, James Mickens, and Hari Balakrishnan. "Mahimahi: Accurate Record-and-Replay for HTTP". In: *2015 USENIX Annual Technical Conference (USENIX ATC 15)*. Santa Clara, CA: USENIX Association, July 2015, pp. 417–429. ISBN: 978-1-931971-225. URL: https://www.usenix.org/conference/atc15/technical-session/presentation/netravali.

[78] Richard A Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J Davison, Pushmeet Kohi, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. "Kinectfusion: Real-time dense surface mapping and tracking". In: *2011 10th IEEE international symposium on mixed and augmented reality*. Ieee. 2011, pp. 127–136.

[79] Richard A. Newcombe, Dieter Fox, and Steven M. Seitz. "DynamicFusion: Reconstruction and tracking of non-rigid scenes in real-time". In: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 343–352. DOI: 10.1109/CVPR.2015.7298631.

[80] "Line Search Methods". In: *Numerical Optimization*. Ed. by Jorge Nocedal and Stephen J. Wright. New York, NY: Springer New York, 1999, pp. 34–63. ISBN: 978-0-387-22742-9. DOI: 10.1007/0-387-22742-3_3.

[81] *nvh265enc*. https://gstreamer.freedesktop.org/documentation/nvcodec/nvh265enc.html.

[82] NVIDIA. *Video Encode and Decode GPU Support Matrix*. https://developer.nvidia.com/video-encode-and-decode-gpu-support-matrix-new. 2024.

[83] NVIDIA Corporation. *NVIDIA Jetson AGX Orin Developer Kit*. Accessed: 2024-05-20. 2024. URL: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/.

[84] *NVIDIA Video Codec SDK*. https://developer.nvidia.com/video-codec-sdk.

[85] Néill O'dwyer, Emin Zerman, Gareth W. Young, Aljosa Smolic, Siobhán Dunne, and Helen Shenton. "Volumetric Video in Augmented Reality Applications for Museological Narratives: A User Study for the Long Room in the Library of Trinity College Dublin". In: *J. Comput. Cult. Herit.* 14.2 (May 2021). ISSN: 1556-4673. DOI: 10.1145/3425400.

[86] *OpenMP*. https://www.openmp.org/.

[87] Sergio Orts-Escolano, Christoph Rhemann, Sean Fanello, Wayne Chang, Adarsh Kowdle, Yury Degtyarev, David Kim, Philip L. Davidson, Sameh Khamis, Mingsong Dou, Vladimir Tankovich, Charles Loop, Qin Cai, Philip A. Chou, Sarah Mennicken, Julien Valentin, Vivek Pradeep, Shenlong Wang, Sing Bing Kang, Pushmeet Kohli, Yuliya Lutchyn, Cem Keskin, and Shahram Izadi. "Holoportation: Virtual 3D Teleportation in Real-Time". In: *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*. UIST '16. Tokyo, Japan: Association for Computing Machinery, 2016, pp. 741–754. ISBN: 9781450341899. DOI: 10.1145/2984511.2984517.

[88] *PanopticStudio Toolbox*. https://github.com/CMU-Perceptual-Computing-Lab/panoptic-toolbox.

[89] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. "PyTorch: an imperative style, high-performance deep learning library". In: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[90] Eduardo Pavez, Philip A. Chou, Ricardo L. de Queiroz, and Antonio Ortega. "Dynamic polygon clouds: representation and compression for VR/AR". In: *APSIPA Transactions on Signal and Information Processing* 7 (2018), e15. DOI: 10.1017/ATSIP.2018.15.

[91] Fabrizio Pece, Jan Kautz, and Tim Weyrich. "Adapting standard video codecs for depth streaming". In: *Proceedings of the 17th Eurographics Conference on Virtual Environments & Third Joint Virtual Reality*. EGVE - JVRC'11. Nottingham, UK: Eurographics Association, 2011, pp. 59–66. ISBN: 9783905674330.

[92] *Point Cloud Library*. https://pointclouds.org/.

[93] Feng Qian, Bo Han, Qingyang Xiao, and Vijay Gopalakrishnan. "Flare: Practical Viewport-Adaptive 360-Degree Video Streaming for Mobile Devices". In: *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking*. MobiCom '18. New Delhi, India: Association for Computing Machinery, 2018, pp. 99–114. ISBN: 9781450359030. DOI: 10.1145/3241539.3241565.

[94] K. Rao and N. Ahmed. "Orthogonal transforms for digital signal processing". In: *ICASSP '76. IEEE International Conference on Acoustics, Speech, and Signal Processing*. Vol. 1. 1976, pp. 136–140. DOI: 10.1109/ICASSP.1976.1170121.

[95] Sebastian Schwarz, Marius Preda, Vittorio Baroncini, Madhukar Budagavi, Pablo Cesar, Philip A. Chou, Robert A. Cohen, Maja Krivokuća, Sébastien Lasserre, Zhu Li, Joan Llach, Khaled Mammou, Rufael Mekuria, Ohji Nakagami, Ernestasia Siahaan, Ali Tabatabai, Alexis M. Tourapis, and Vladyslav Zakharchenko. "Emerging MPEG Standards for Point Cloud Compression". In: *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9.1 (2019), pp. 133–148. DOI: 10.1109/JETCAS.2018.2885981.

[96] Anil Shanbhag, Samuel Madden, and Xiangyao Yu. "A Study of the Fundamental Performance Characteristics of GPUs and CPUs for Database Analytics". In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. SIGMOD '20. Portland, OR, USA: Association for Computing Machinery, 2020, pp. 1617–1632. ISBN: 9781450367356. DOI: 10.1145/3318464.3380595.

[97] Taveesh Sharma, Tarun Mangla, Arpit Gupta, Junchen Jiang, and Nick Feamster. "Estimating WebRTC Video QoE Metrics Without Using Application Headers". In: *Proceedings of the 2023 ACM on Internet Measurement Conference*. IMC '23. Montreal QC, Canada: Association for Computing Machinery, 2023, pp. 485–500. ISBN: 9798400703829. DOI: 10.1145/3618257.3624828.

[98] Jianxin Shi, Miao Zhang, Linfeng Shen, Jiangchuan Liu, Yuan Zhang, Lingjun Pu, and Jingdong Xu. "Towards Full-scene Volumetric Video Streaming via Spatially Layered Representation and NeRF Generation". In: *Proceedings of the 34th Edition of the Workshop on Network and Operating System Support for Digital Audio and Video*. NOSSDAV '24. Bari, Italy: Association for Computing Machinery, 2024, pp. 22–28. ISBN: 9798400706134. DOI: 10.1145/3651863.3651879.

[99] Yuang Shi. "3D Gaussian-based Immersive Media Streaming in Networked Extended Reality". In: *Proceedings of the 16th ACM Multimedia Systems Conference*. MMSys '25. Stellenbosch, South Africa: Association for Computing Machinery, 2025, pp. 356–360. ISBN: 9798400714672. DOI: 10.1145/3712676.3719673.

[100] Tetsuri Sonoda and Anders Grunnet-Jepsen. *Depth image compression by colorization for Intel® RealSense™ Depth Cameras*. https://dev.intelrealsense.com/docs/depth-image-compression-by-colorization-for-intel-realsense-depth-cameras.

[101] Kevin Spiteri, Rahul Urgaonkar, and Ramesh K Sitaraman. "BOLA: Near-optimal bitrate adaptation for online videos". In: *IEEE INFOCOM 2016-The 35th Annual IEEE International Conference on Computer Communications*. IEEE. 2016, pp. 1–9.

[102] Gary J. Sullivan and Shijun Sun. "On dead-zone plus uniform threshold scalar quantization". In: *Visual Communications and Image Processing 2005*. Ed. by Shipeng Li, Fernando Pereira, Heung-Yeung Shum, and Andrew G. Tescher. Vol. 5960. International Society for Optics and Photonics. SPIE, 2005, p. 596033. DOI: 10.1117/12.631550.

[103] Jiakai Sun, Han Jiao, Guangyuan Li, Zhanjie Zhang, Lei Zhao, and Wei Xing. "3DGStream: On-the-Fly Training of 3D Gaussians for Efficient Streaming of Photo-Realistic Free-Viewpoint Videos". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2024, pp. 20675–20685.

[104] Yuan-Chun Sun, Yuang Shi, Cheng-Tse Lee, Mufeng Zhu, Wei Tsang Ooi, Yao Liu, Chun-Ying Huang, and Cheng-Hsin Hsu. "LTS: A DASH Streaming System for Dynamic Multi-Layer 3D Gaussian Splatting Scenes". In: *Proceedings of the 16th ACM Multimedia Systems Conference*. MMSys '25. Stellenbosch, South Africa: Association for Computing Machinery, 2025, pp. 136–147. ISBN: 9798400714672. DOI: 10.1145/3712676.3714445.

[105] Yuan-Chun Sun, Yuang Shi, Wei Tsang Ooi, Chun-Ying Huang, and Cheng-Hsin Hsu. "Multi-frame Bitrate Allocation of Dynamic 3D Gaussian Splatting Streaming Over Dynamic Networks". In: *Proceedings of the 2024 SIGCOMM Workshop on Emerging Multimedia Systems*. EMS '24. Sydney, NSW, Australia: Association for Computing Machinery, 2024, pp. 1–7. ISBN: 9798400707117. DOI: 10.1145/3672196.3673394.

[106] Yuan-Chun Sun, Yuang Shi, Wei Tsang Ooi, Chun-Ying Huang, and Cheng-Hsin Hsu. "Multi-frame Bitrate Allocation of Dynamic 3D Gaussian Splatting Streaming Over Dynamic Networks". In: *Proceedings of the 2024 SIGCOMM Workshop on Emerging Multimedia Systems*. EMS '24. Sydney, NSW, Australia: Association for Computing Machinery, 2024, pp. 1–7. ISBN: 9798400707117. DOI: 10.1145/3672196.3673394.

[107] Hari Sundar, Rahul S. Sampath, and George Biros. "Bottom-Up Construction and 2:1 Balance Refinement of Linear Octrees in Parallel". In: *SIAM Journal on Scientific Computing* 30.5 (2008), pp. 2675–2708. DOI: 10.1137/070681727. eprint: https://doi.org/10.1137/070681727.

[108] *Synchronize multiple Azure Kinect DK devices*. https://learn.microsoft.com/en-us/azure/kinect-dk/multi-camera-sync. 2022.

[109] GStreamer Team. *Gstreamer: open source multimedia framework*. https://gstreamer.freedesktop.org/.

[110] Dong Tian, Hideaki Ochimizu, Chen Feng, Robert Cohen, and Anthony Vetro. "Geometric distortion metrics for point cloud compression". In: *2017 IEEE International Conference on Image Processing (ICIP)*. 2017, pp. 3460–3464. DOI: 10.1109/ICIP.2017.8296925.

[111] Hanzhang Tu, Ruizhi Shao, Xue Dong, Shunyuan Zheng, Hao Zhang, Lili Chen, Meili Wang, Wenyu Li, Siyan Ma, Shengping Zhang, Boyao Zhou, and Yebin Liu. "Tele-Aloha: A Telepresence System with Low-budget and High-authenticity Using Sparse RGB Cameras". In: *ACM SIGGRAPH 2024 Conference Papers*. SIGGRAPH '24. Denver, CO, USA: Association for Computing Machinery, 2024. ISBN: 9798400705250. DOI: 10.1145/3641519.3657491.

[112] The Verge. *Volumetric video technology enables video game-like broadcasts*. https://www.theverge.com/2022/3/16/22982243/nets-mavericks-espn-nba-volumetric-video-canon. 2022.

[113] Chenjunjie Wang, Shashank N. Sridhara, Eduardo Pavez, Antonio Ortega, and Cheng Chang. "Adaptive Voxelization for Transform Coding of 3D Gaussian Splatting Data". In: *2025 IEEE International Conference on Image Processing (ICIP)*. 2025, pp. 2414–2419. DOI: 10.1109/ICIP55913.2025.11084522.

[114] Penghao Wang, Zhirui Zhang, Liao Wang, Kaixin Yao, Siyuan Xie, Jingyi Yu, Minye Wu, and Lan Xu. "V^ 3: Viewing Volumetric Videos on Mobiles via Streamable 2D Dynamic Gaussians". In: *ACM Transactions on Graphics (TOG)* 43.6 (2024), pp. 1–13.

[115] Shengze Wang, Ziheng Wang, Ryan Schmelzle, Liujie Zheng, YoungJoong Kwon, Roni Sengupta, and Henry Fuchs. "Learning View Synthesis for Desktop Telepresence with Few RGBD Cameras". In: *IEEE Transactions on Visualization and Computer Graphics* (2024), pp. 1–13. DOI: 10.1109/TVCG.2024.3411626.

[116] Yizong Wang, Dong Zhao, Huanhuan Zhang, Chenghao Huang, Teng Gao, Zixuan Guo, Liming Pang, and Huadong Ma. "Hermes: Leveraging Implicit Inter-Frame Correlation for Bandwidth-Efficient Mobile Volumetric Video Streaming". In: *Proceedings of the 31st ACM International Conference on Multimedia*. 2023, pp. 9185–9193.

[117] Zhe Wang and Yifei Zhu. "Towards Real-Time Neural Volumetric Rendering on Mobile Devices: A Measurement Study". In: *Proceedings of the 2024 SIGCOMM Workshop on Emerging Multimedia Systems*. EMS '24. Sydney, NSW, Australia: Association for Computing Machinery, 2024, pp. 8–13. ISBN: 9798400707117. DOI: 10.1145/3672196.3673399.

[118] Andrew D. Wilson. "Fast Lossless Depth Image Compression". In: *Proceedings of the 2017 ACM International Conference on Interactive Surfaces and Spaces*. ISS '17. Brighton, United Kingdom: Association for Computing Machinery, 2017, pp. 100–105. ISBN: 9781450346917. DOI: 10.1145/3132272.3134144.

[119] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. "4D Gaussian Splatting for Real-Time Dynamic Scene Rendering". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2024, pp. 20310–20320.

[120] Yixuan Wu, Kaiyuan Hu, Qian Shao, Jintai Chen, Danny Z. Chen, and Jian Wu. " TeleOR: Real-time Telemedicine System for Full-Scene Operating Room ". In: *proceedings of Medical Image Computing and Computer Assisted Intervention – MICCAI 2024*. Vol. LNCS 15006. Springer Nature Switzerland, Oct. 2024.

[121] Shuzhao Xie, Weixiang Zhang, Chen Tang, Yunpeng Bai, Rongwei Lu, Shijia Ge, and Zhi Wang. "MesonGS: Post-training Compression of 3D Gaussians via Efficient Attribute Transformation". In: *European Conference on Computer Vision*. Springer. 2024.

[122] Mengyu Yang, Zhenxiao Luo, Miao Hu, Min Chen, and Di Wu. "A Comparative Measurement Study of Point Cloud-Based Volumetric Video Codecs". In: *IEEE Transactions on Broadcasting* 69.3 (2023), pp. 715–726. DOI: 10.1109/TBC.2023.3243407.

[123] Qi Yang, Zhan Ma, Yiling Xu, Zhu Li, and Jun Sun. "Inferring Point Cloud Quality via Graph Similarity". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.6 (2022), pp. 3015–3029. DOI: 10.1109/TPAMI.2020.3047083.

[124] *YUV*. https://en.wikipedia.org/wiki/YUV.

[125] Anlan Zhang, Chendong Wang, Bo Han, and Feng Qian. "{YuZu}:{Neural-Enhanced} volumetric video streaming". In: *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 2022, pp. 137–154.

[126] Kai Zhang, Kaibo Wang, Yuan Yuan, Lei Guo, Rubao Lee, and Xiaodong Zhang. "Mega-KV: a case for GPUs to maximize the throughput of in-memory key-value stores". In: *Proc. VLDB Endow.* 8.11 (July 2015), pp. 1226–1237. ISSN: 2150-8097. DOI: 10.14778/2809974.2809984.

[127] Z. Zhang. "A flexible new technique for camera calibration". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.11 (2000), pp. 1330–1334. DOI: 10.1109/34.888718.

[128] Shunyuan Zheng, Boyao Zhou, Ruizhi Shao, Boning Liu, Shengping Zhang, Liqiang Nie, and Yebin Liu. “GPS-Gaussian: Generalizable Pixel-wise 3D Gaussian Splatting for Real-time Human Novel View Synthesis”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024.

[129] Zihan Zheng, Zhenlong Wu, Houqiang Zhong, Yuan Tian, Ning Cao, Lan Xu, Jiangchao Yao, Xiaoyun Zhang, Qiang Hu, and Wenjun Zhang. *4DGCPro: Efficient Hierarchical 4D Gaussian Compression for Progressive Volumetric Video Streaming*. 2025. arXiv: 2509.17513 [cs.CV]. URL: https://arxiv.org/abs/2509.17513.

[130] Junquan Zhong, Haodan Zhang, Quanlu Jia, Jiangkai Wu, Peiheng Wang, Haoyang Wang, Liming Liu, Xinggong Zhang, and Zongming Guo. “Low-bitrate Volumetric Video Streaming with Depth Image”. In: *Proceedings of the 2024 SIGCOMM Workshop on Emerging Multimedia Systems*. EMS ’24. Sydney, NSW, Australia: Association for Computing Machinery, 2024, pp. 39–44. ISBN: 9798400707117. DOI: 10.1145/3672196.3673397.

[131] Boyao Zhou, Shunyuan Zheng, Hanzhang Tu, Ruizhi Shao, Boning Liu, Shengping Zhang, Liqiang Nie, and Yebin Liu. “GPS-Gaussian+: Generalizable Pixel-wise 3D Gaussian Splatting for Real-Time Human-Scene Rendering from Sparse Views”. In: *arXiv preprint arXiv:2411.11363* (2024).

[132] Kun Zhou, Minmin Gong, Xin Huang, and Baining Guo. “Data-Parallel Octrees for Surface Reconstruction”. In: *IEEE Transactions on Visualization and Computer Graphics* 17.5 (2011), pp. 669–681. DOI: 10.1109/TVCG.2010.75.

[133] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. “Open3D: A Modern Library for 3D Data Processing”. In: *arXiv:1801.09847* (2018).